PHYSICS-BASED AND DIRECTABLE SIMULATIONS OF LARGE-DEFORMATION PROCESSES FOR ANIMATION AND ENGINEERING

By

Chang-Yong Song

Dissertation

Submitted to the Faculty of the

Graduate School of Vanderbilt University

in partial fulfillment of the requirements

for the degree of

DOCTOR OF PHILOSOPHY

in

Computer Science

November, 2026

Nashville, Tennessee

TABLE OF CONTENTS

Page

LIST OF TABLES . . . . . **vi**

LIST OF FIGURES . . . . . **vii**

1 Introduction **1**

1.1 Motivation: Controllable Deformation for Animation and Engineering 1
1.2 Challenges of Conventional Numerical Methods 1
1.3 The Material Point Method and Its Extensions 2
1.4 From Forward Simulation to Differentiable Control and Physical Grounding 2
1.5 Research Contributions 3
1.5.1 Physics-Based Shape Morphing 3
1.5.2 Render-Guided Volumetric Morphing 3
1.5.3 Physics-Grounded Robotic Cutting and VLA Benchmarking 4

2 Related Work **6**

2.1 Physics-Based Shape Morphing 6
2.1.1 Material Point Method (MPM) 6
2.1.2 Shape Morphing 7
2.1.3 Optimization and Control 7
2.1.4 Differentiable Simulation 8
2.2 Render-Guided Morphing 8
2.2.1 Gaussian Splatting and Dynamic Scenes 8
2.2.2 Differentiable Physics Coupled with Rendering 9
2.3 Physics-Grounded Robotic Cutting 9
2.3.1 Vision–Language–Action Policies 9
2.3.2 Large-Scale Robot Learning Datasets 10

3 Physics-Based Shape Morphing **11**

3.1 Overview 11
3.1.1 Differentiable Simulation 12
3.2 Deformation Gradient Control 13
3.2.1 Automatic Per-Particle Deformation Gradient Updates 14
3.2.2 Preliminary Shape Loss Functions 14
3.2.3 Log-Based Mass Loss Function 15
3.3 Morphing Method 16
3.3.1 Simulation Optimization 17
3.3.2 Control Layers 17
3.3.3 Multiple Optimization Passes 18
3.3.4 Chaining Multiple Simulation Networks 19
3.4 Examples 20
3.5 Chapter Summary 30

4 Render-Guided Volumetric Morphing with Differentiable Physics **32**

4.1 Overview 32

4.2 The Simulated Body . . . 33
4.3 Two Controls: Bulk and Surface . . . 34
4.3.1 What $\delta \mathbf{F}_c$ Does: The Bulk . . . 35
4.3.2 What u Does: The Surface . . . 35
4.4 Objective . . . 35
4.4.1 What the Physics Terms Ask . . . 36
4.4.2 What the Render Loss Asks For . . . 36
4.5 Morphing Method . . . 36
4.5.1 Settled Windows . . . 37
4.5.2 How the Render Reaches $\delta \mathbf{F}_c$ and u . . . 37
4.5.3 Gradients and the Render Weight . . . 38
4.5.4 A Second Mode: The Render Keeps Its Weight . . . 38
4.5.5 Step, Commit and Stop . . . 39
4.5.6 What the Body Conserves . . . 39
4.5.7 Bodies in a Scene . . . 39
4.6 What the Render Does . . . 40
4.6.1 Setup . . . 40
4.6.2 Where the Render Acts . . . 40
4.6.3 Under the Default Rule: Arriving, Then Polishing . . . 40
4.6.4 What the Render Can Write . . . 44
4.6.5 The Letters Are in the Material . . . 46
4.7 Examples and Comparisons . . . 47
4.7.1 From a Sphere to Five Targets . . . 47
4.7.2 The 40k Gallery . . . 47
4.7.3 A Gallery of 300k Morphs . . . 48
4.7.4 Materials and External Forces . . . 48
4.7.5 Ablations . . . 49
4.7.6 Stress Control against Force Control . . . 51
4.7.7 Comparison with Other Methods . . . 51
4.7.8 Runtime . . . 51
4.7.9 Thrown and Dropped Bodies . . . 52
4.8 Measured Costs and Next Steps . . . 52
4.9 Chapter Summary . . . 53

5 CulinaryCut: Physics-Grounded Cutting for Vision–Language–Action Models . . . 55

5.1 Overview . . . 55
5.1.1 Research Motivation and Contributions . . . 56
5.2 Background and Related Work . . . 57
5.2.1 Vision–Language–Action Policies and Geometric Supervision . . . 57
5.2.2 Manipulation Datasets and Cutting Simulators . . . 57
5.2.3 Material Simulation and the Visual/Physical Domain Gap . . . 58
5.3 IntSim: Physics-Grounded Cutting Simulation . . . 59
5.3.1 Robot–Material Simulation Coupling . . . 59
5.3.2 MLS-MPM Formulation . . . 59
5.3.2.1 Conservation Laws and Constitutive Response . . . 59
5.3.2.2 Particle-to-Grid Transfer . . . 60
5.3.2.3 Grid Evolution and Tool Contact . . . 60
5.3.2.4 Grid-to-Particle Transfer and Deformation Update . . . 61
5.3.3 Damage-Gated Separation and Numerical Stability . . . 61
5.3.4 Implementation Diagnostics and Resolution Treatment . . . 62
5.3.5 Material Parameters and Simulation Settings . . . 62
5.3.6 Execution-Time Velocity Guardrail . . . 64
5.4 CulinaryCut Benchmark and Training Protocol . . . 64

5.4.1 Task Design and Geometric Grounding . . . 64
5.4.2 Instruction Representation and Ambiguity Resolution . . . 67
5.4.3 Trajectory Generation and Dataset Composition . . . 68
5.4.4 Dataset Preview: Robotic Observations and Particle-Level Cutting . . . 68
5.4.5 VLA Baselines and Optimization Interface . . . 72
5.4.6 Evaluation Metrics . . . 73
5.5 Experiments and Results . . . 73
5.5.1 Visual and Physical Validation of IntSim . . . 73
5.5.2 Geometry-Gap Evaluation . . . 75
5.5.2.1 Object-Scale Sensitivity and Multi-Object Scenes . . . 75
5.5.2.2 Continuous Ratios and Directional Reference . . . 77
5.5.2.3 Sequential Split Cuts . . . 77
5.5.3 Physics Gap and Real-Robot Transfer . . . 78
5.5.4 Ablation: Importance of Topology Updates . . . 80
5.5.5 Ablation: Execution-Time Velocity Guardrail . . . 82
5.5.6 Ablation: Food–Board Friction . . . 83
5.6 Discussion and Limitations . . . 83
5.6.1 Interpreting the Geometry and Physics Gaps . . . 83
5.6.2 What the Benchmark Demonstrates about Simulation . . . 83
5.6.3 Limitations and Future Directions . . . 83
5.7 Chapter Summary . . . 84

6 Limitations and Future Directions . . . 85

6.1 Limitations of Physics-Based Shape Morphing . . . 85
6.1.1 Time Integration and Optimization Cost . . . 85
6.1.2 Limited Optimization Horizons . . . 85
6.1.3 Dependence on the Objective and Smoothing Parameters . . . 85
6.2 Limitations of Render-Guided Volumetric Morphing . . . 86
6.2.1 The Render Term's Authority after Arrival . . . 86
6.2.2 Surface Detail below the Grid . . . 86
6.2.3 Rendering Representation versus Physical State . . . 86
6.3 Limitations of Physics-Grounded Robotic Cutting . . . 87
6.3.1 Geometric Generalization and Sequential Actions . . . 87
6.3.2 Approximate Cutting Mechanics and Material Calibration . . . 87
6.3.3 Limits of Physical Validation and Sim-to-Real Transfer . . . 87
6.4 Shared Challenges in Efficiency and Reproducibility . . . 88
6.5 Future Research Directions . . . 88
6.5.1 More Efficient and Stable Simulation-Based Optimization . . . 88
6.5.2 Lasting Rendering Feedback and Finer Surface Control . . . 88
6.5.3 Measured Material Properties and Broader Robotic Validation . . . 88
6.5.4 Bridging Simulation Objectives and Downstream Task Success . . . 89
6.6 Concluding Remarks . . . 89

A MLS-MPM Details and Notation . . . 90

B Render-Guided Morphing Details . . . 93

B.1 The Simulated Body . . . 93
B.1.1 Position Updates below the Grid . . . 94
B.2 Objective Terms . . . 94
B.2.1 Cleanup Terms . . . 95
B.2.2 Render Terms . . . 96
B.3 Window, Step and Selection . . . 96

B.4 Multi-Body Scenes . . . 97
B.5 Implementation and Runtime . . . 98
B.6 Measures . . . 98
B.7 Additional Results . . . 99
B.7.1 The Render's Effect . . . 99
B.7.2 The 40k Gallery . . . 101
B.7.3 The 300k Gallery . . . 102
B.7.4 Materials and Gravity . . . 102
B.7.5 Ablations . . . 103
B.7.6 Stress against Force Control . . . 103
B.7.7 Comparison with Other Methods . . . 104
B.7.8 Thrown and Dropped Bodies . . . 105
B.7.9 The Render with Its Weight Held . . . 105
B.7.10 Further Measured Costs . . . 107

C IntSim Cutting Implementation Details . . . 108

C.1 Contact-Strength Normalization and Resistance Update . . . 108
C.2 Resolution-Dependent Damage Thresholds and Time-Step Constraint . . . 108

References . . . 109

LIST OF TABLES

Table		Page
3.1	Varying the number of optimization passes and optimization iterations per pass	19
3.2	All example pairs used in our morphing experiments	20
3.3	Performance for various shape morphing sequences	23
3.4	Number of particles for our complex examples	27
4.1	The full method against its $\lambda = 0$ twin, read against the exact target mesh	43
5.1	Capabilities emphasized by representative cutting and manipulation datasets	58
5.2	Representative IntSim numerical and observational settings	63
5.3	Representative material parameters in CulinaryCut	64
5.4	Representative language instructions in CulinaryCut	67
5.5	Fine-tuning settings for the three VLA baselines	72
5.6	Geometric generalization across evaluation settings	75
5.7	RDT step-wise split-cut evaluation	78
5.8	Effect of physics-grounded demonstrations for RDT orange cutting	78
5.9	RDT topology-update ablation	80
5.10	Velocity-guardrail ablation	82
A.1	A table of variables used in our MPM simulation formulation	90
B.1	Wall-clock time of a 300k morph on one RTX 6000 Ada	98
B.2	The six 300k meshes at the end, full method / twin	100
B.3	Ablations at 300k (bunny / dragon, three seeds)	103
B.4	Every method on the six 300k targets	104

LIST OF FIGURES

Figure		Page
3.1	Simulation overview	11
3.2	An example of mass ejections occurring under the naive nodal mass loss	15
3.3	Morphing method	16
3.4	Sphere to bunny morphing animation	21
3.5	Visualizing our loss function while morphing a sphere to a bunny	22
3.6	Morphing Duck (Bob) to Cow	24
3.7	Pairwise shape morphing results among five distinct geometries	25
3.8	Comparison of loss curves under different optimizers (Adam, GD, Lion)	26
3.9	Parallel scaling	27
3.10	Morphing example with letters	28
3.11	Morphing dragon example	29
3.12	Morphing under physical external forces	30
3.13	Morphing behavior under vertical compression across varying material stiffness	31
4.1	Pipeline overview	33
4.2	Surface proximity and the outer layer	34
4.3	How the render loss reaches the two controls	37
4.4	The render gradient on the deforming 300k bunny, early (top) and late (bottom)	41
4.5	What the render loss changes at the end of a 300k morph	42
4.6	The render term switched at 6 s	44
4.7	Lettering that exists only in the target pictures	45
4.8	Placing material by its appearance	46
4.9	Where the render acts when it keeps its weight	47
4.10	The lettering is in the material	48
4.11	From a sphere to five targets at 300k particles	49
4.12	Twenty-four 300k morphs, each shown at its start, halfway and end	50
4.13	Two thrown bodies, each becoming a bunny	52
5.1	Overview of physics-grounded robotic food cutting	56
5.2	CulinaryCut data generation	57
5.3	Blade-localized damage gate	62
5.4	Representative cutting motions	66
5.5	CulinaryCut dataset gallery (1/2)	70
5.6	CulinaryCut dataset gallery (2/2)	71
5.7	RDT fine-tuning learning curve	72
5.8	IntSim material response during orange cutting	74
5.9	Representative force validation against a real Franka robot	74
5.10	Real–simulation force comparison	75
5.11	Single-object cutting across food categories	76
5.12	Transfer to an unseen food object category	76
5.13	Multi-object geometric grounding	76
5.14	Continuous-ratio cutting	77
5.15	Directional and ratio transfer	77
5.16	Real-world comparison of trajectory sources	79
5.17	Real-world orange cutting	79
5.18	Qualitative impact of topology updates	81
5.19	Cutting motion and force with the velocity guardrail	82
5.20	Exploratory visual enhancement with Cosmos	84
B.1	The end of the run, full method against its $\lambda = 0$ twin on six meshes	100

B.2 Sensitivity of the particle silhouette IoU 101
B.3 The 40k gallery against the exact meshes 102

CHAPTER 1

Introduction

1.1 Motivation: Controllable Deformation for Animation and Engineering

Deformable objects are central to computer animation, interactive graphics, and engineering applications involving contact and material response. Unlike rigid-body motion, deformation may change an object's shape, local stress, and even its connectivity. Examples range from transforming a character into a different shape to cutting a soft object with a robotic tool. Producing useful simulations of these processes requires more than plausible images: the evolving motion should also reflect the modeled material properties, imposed controls, and contact conditions.

These requirements introduce several coupled challenges. Large deformations can lead to numerical instability or the loss of fine geometric features; changes in topology complicate surface representations; and material-dependent forces influence whether a commanded interaction produces its intended outcome. Moreover, a physically reasonable particle configuration does not necessarily yield a visually accurate rendering of a target shape. In robotics, a geometrically valid tool trajectory likewise does not guarantee successful material separation. Physical behavior, visual appearance, and task-level success must therefore be considered as related but distinct objectives.

This dissertation investigates how physics-based particle simulation can support three complementary goals: *directing* deformable motion toward a desired shape, *using rendering feedback* to improve the visual outcome of that motion, and *generating physically grounded data* for evaluating contact-rich robotic manipulation. Together, these problems motivate a common simulation foundation without assuming that every application uses an identical optimization or feedback architecture.

1.2 Challenges of Conventional Numerical Methods

A variety of numerical representations have been developed for deformable simulation. Mass–spring systems are straightforward to implement and can be efficient for animation, but their spring parameters do not automatically correspond to continuum constitutive properties, and stiff systems may require careful time integration (Terzopoulos et al., 1987; Baraff and Witkin, 1998; Bridson et al., 2003). The Finite Element Method (FEM) provides a well-established discretization of continuum mechanics and supports accurate stress and deformation analysis. However, severe element distortion and fracture may require specialized remeshing, cohesive interfaces, or other topology-management techniques (O'Brien et al., 2002). The Discrete Element Method (DEM) is well suited to interactions among separate grains or blocks, although it is not, by itself, a continuum deformation model (Cundall and Strack, 1979).

These methods are valuable in their respective settings; none should be regarded as universally unsuitable for

large deformation or contact. For the applications considered here, however, it is advantageous to represent persistent material state with particles while evaluating interactions on a temporary computational grid. This motivates the Material Point Method (MPM), which combines Lagrangian material tracking with Eulerian grid computations.

1.3 The Material Point Method and Its Extensions

MPM was introduced for history-dependent materials (Sulsky et al., 1994) and has since been extended to elastoplastic animation, fracture, and coupled material interactions (Stomakhin et al., 2013; Wolper et al., 2019; de Vaucorbeil et al., 2020). Its material points carry mass, velocity, deformation gradients, and constitutive history. At each time step, particle information is transferred to a background grid, grid momenta and forces are updated, and the resulting velocities are transferred back to the particles. Because the grid is rebuilt rather than permanently attached to the material, large changes in shape need not distort a persistent finite-element mesh.

The particle representation also makes it possible to represent evolving connectivity and material separation, although realistic fracture still requires suitable constitutive damage and contact treatments. Two developments are especially relevant to this dissertation:

- **Affine Particle-in-Cell (APIC).** APIC augments particle velocity with a local affine velocity matrix $\mathbf{C}_p$. Its transfer operators reduce the numerical dissipation of conventional PIC while retaining linear- and angular-momentum-conserving properties under the assumptions of the transfer scheme (Jiang et al., 2015, 2017).

- **Moving Least Squares MPM (MLS-MPM).** MLS-MPM provides an efficient particle–grid discretization, including an MLS-based formulation of the transfer and stress-divergence terms. Its associated compatible particle-in-cell treatment can represent displacement discontinuities and tool–material coupling (Hu et al., 2018).

These developments make MPM a suitable starting point for the morphing and cutting problems studied here. Nevertheless, the numerical accuracy and stability of a complete simulation still depend on the time integrator, material model, contact handling, spatial resolution, and any stabilization used in a particular application.

1.4 From Forward Simulation to Differentiable Control and Physical Grounding

Forward simulation predicts how material states evolve under specified initial conditions, constitutive laws, and interactions. Inverse problems instead ask which controls will drive the system toward a desired outcome. Differentiable simulation enables gradient-based approaches to such problems by computing derivatives through the time-stepping process (Hu et al., 2019b; Coros et al., 2021; Li et al., 2023). For example, a target shape can define an objective whose gradient is propagated backward through MPM to optimize internal deformation controls.

A further challenge arises when the desired outcome is defined in image space. If simulation states are converted into display meshes and rendered only as a separate post-processing operation, pixel-level errors do not automatically

provide gradients to the underlying particle controls. A differentiable connection between physical states and rendered observations can instead make visual appearance part of the optimization objective.

Not every physics-based application, however, requires a differentiable policy or simulator at inference time. In contact-rich robotics, a forward simulator can provide physically consistent demonstrations and evaluation signals even when an existing policy predicts actions only from images, language, and robot state. This distinction is central to the organization of this dissertation: Chapters 3 and 4 study simulation-based optimization, whereas Chapter 5 investigates physical grounding at the data-generation and evaluation stages of robotic cutting.

1.5 Research Contributions

The dissertation makes three complementary contributions to the simulation and use of large-deformation processes.

1.5.1 Physics-Based Shape Morphing

The first contribution develops a directable shape-morphing method based on differentiable MLS-MPM. Rather than prescribing a separate trajectory for every particle or designing external forces by hand, the method optimizes per-particle *control deformation gradients*. The resulting internal stresses drive the material from a source configuration toward a target shape while maintaining a continuum-mechanics-based evolution.

A correspondence-free, log-based nodal-mass objective provides shape supervision and reduces the mass-ejection artifacts observed with a naive grid-mass discrepancy. The method also incorporates iterative gradient-based optimization, multiple optimization passes, and chained simulation segments to manage long animation sequences. Experiments in Chapter 3 demonstrate morphing between geometrically distinct objects, including examples with changing topology and fine features, as well as behavior under gravity, twisting, and changes in material stiffness. These results establish deformation-gradient control as a practical way to direct MPM-based animation.

1.5.2 Render-Guided Volumetric Morphing

The second contribution addresses the gap between particle-based physical simulation and image-space supervision. Conventional visualization pipelines often reconstruct a surface from simulation particles using a separate meshing stage, for example with sparse volume representations or marching-cubes extraction (Museth, 2013; Lorensen and Cline, 1987). When this stage is not differentiated through, rendering losses cannot directly guide the particle simulation. Such a limitation is specific to the chosen pipeline; it does not imply that all surface-extraction methods are intrinsically non-differentiable.

Chapter 4 makes the rendered picture part of the control. Silhouette and shading terms are evaluated on the body's *exterior*, the surface of a field built from its own particles, and their gradients reach two controls: the stress increment of Chapter 3, which moves the bulk of the body through the grid, and a rigid-free offset of the outer particle layer,

which shapes the surface below the grid. The physics objective becomes a Sinkhorn transport between the body and the target, and the morph is optimized in *settled windows*, each scored only after the controls have been released, so that a state the controls hold only while they push cannot win. The controls still enter the material only as a stress and a surface offset, and the body keeps its mass exactly.

Each part of the method is measured against an otherwise identical run with the render weight set to zero. The render term redirects material while the body is arriving and afterwards mainly polishes its outline, leaving the end consistently but slightly closer to the target. Chapter 4 presents the method, morphs toward a wide range of targets, comparisons with other methods, and the steps that remain, chief among them keeping the render term's authority after the body has arrived.

1.5.3 Physics-Grounded Robotic Cutting and VLA Benchmarking

The third contribution examines robotic food cutting, a contact-rich task in which placing a knife at the intended location is not sufficient for success. A cutting policy must also induce the material-dependent interaction needed to deform and separate the food. This motivates two complementary evaluation questions: whether a policy correctly grounds spatial instructions in object geometry (the *geometry gap*), and whether its actions produce successful material separation (the *physics gap*).

To study these questions, Chapter 5 introduces *IntSim*, an integrated environment coupling an MLS-MPM food-material simulator with ManiSkill robot execution. The material simulation represents deformation, tool contact, damage-dependent weakening, and separation, while contact impulses provide force profiles associated with the executed trajectories. IntSim is used to generate *CulinaryCut*, a benchmark containing 21,000 base cutting trajectories across 15 food categories. Visual-scene and language augmentation expand these trajectories to approximately 1.575 million observation–instruction instances.

We evaluate three existing Vision–Language–Action (VLA) models—RDT, Octo, and OpenVLA—without modifying their underlying policy architectures. Importantly, the simulated contact forces and damage states are used for generating and assessing the data; they are *not* provided as force-feedback inputs to the policies. This design distinguishes the contribution from a new force-conditioned controller.

Experiments reveal weaknesses in ratio- and direction-based grounding, multi-object selection, held-out-object transfer, and sequential cutting. In the physical-severing evaluation, RDT trained on IntSim-grounded demonstrations improves orange-cutting success from 23% to 59% relative to training on geometry-based ManiSkill demonstrations. Preliminary real-robot trials show the same direction of improvement, from 1/10 to 3/10 successful cuts. A separate comparison of simulated and measured contact forces during banana cutting provides evidence of physical plausibility for a representative task, without establishing complete force fidelity across all food materials. These results support physics-grounded simulation as a source of training data and evaluation criteria for deformable robotic manipulation.

Dissertation Organization. These contributions share MPM-based deformation modeling, but address different objectives: physical shape control, render-guided control, and physically grounded robotic interaction. They should not be interpreted as one end-to-end differentiable system deployed unchanged across all three studies. The remaining chapters are organized as follows:

- **Chapter 2** reviews prior work on MPM and deformable shape morphing, differentiable simulation and rendering, and vision–language–action learning for manipulation.
- **Chapter 3** develops physics-based shape morphing through differentiable MLS-MPM and deformation-gradient control, with experiments on geometric and physical variations.
- **Chapter 4** develops render-guided volumetric morphing, in which silhouette and shading terms on the body's exterior steer the stress and surface controls of the differentiable MPM simulation.
- **Chapter 5** presents CulinaryCut and IntSim, evaluates geometry and physics gaps in VLA food cutting, and examines simulated and real robotic interactions.
- **Chapter 6** discusses the limitations of the methods and identifies directions for future work.

CHAPTER 2

Related Work

2.1 Physics-Based Shape Morphing

2.1.1 Material Point Method (MPM)

The Material Point Method (MPM) was introduced by Sulsky et al. (1994) to solve solid mechanics problems involving large deformations, impact, and penetration. As a hybrid method that couples Lagrangian particles with an Eulerian grid, MPM effectively handles severe material deformation and topology-changing phenomena.

Early MPM implementations that relied on standard PIC/FLIP transfers suffered from noticeable numerical dissipation and, in particular, poor conservation of angular momentum. To mitigate this, the **APIC** (Affine Particle-in-Cell) scheme (Jiang et al., 2015, 2017) was proposed; by transporting an affine velocity term (**C**), APIC better preserves angular momentum and shear response, thereby reducing artificial damping. Building on this idea, **MLS-MPM** (Hu et al., 2018) uses Moving Least Squares shape functions to derive APIC-consistent transfers and a faster stress-divergence discretization, and its compatible particle-in-cell extension supports cutting and two-way rigid-body coupling.

Recent work has substantially extended MPM in stability, accuracy, and scope. Implicit time-integration techniques (Hyde et al., 2020; Wang et al., 2020a) increase robustness for stiff materials and strong surface tension and allow larger time steps, while high-fidelity frictional contact algorithms (Kakouris et al., 2025) are now available for scenarios with repeated contact and separation, such as the cutting task considered in this thesis.

The repertoire of material models has likewise broadened from simple elasticity to anisotropic media (e.g., meats and various fruits) (Wolper et al., 2020), complex plasticity (e.g., snow) (Stomakhin et al., 2013), and fracture coupled with topological change (Wolper et al., 2019; Wang et al., 2019). MPM has also been extended to challenging geometries, including thin-shell dynamics (Guo et al., 2018).

In parallel, performance-oriented research has accelerated MPM substantially: GPU acceleration with automatic differentiation (e.g., JAX-based pipelines) (Du and He, 2026), massively parallel MPM on multiple GPUs (Wang et al., 2020b), and sparse, memory-efficient GPU-MPM frameworks for large-scale simulations (Chen et al., 2025a) enable much larger problem sizes.

Throughout this thesis, we build on these developments. In particular, Chapter 3 will adopt APIC's momentum-preserving transfers together with the stability of MLS-MPM to form a reliable computational core, which later chapters leverage for differentiable control.

2.1.2 Shape Morphing

Shape morphing algorithms have historically been explored under a variety of geometric representations. Implicit-surface approaches were demonstrated by Turk and O'Brien (1999) and Breen and Whitaker (2001); Bansal and Tatu (2018) proposed interpolation via Lie body representations of triangle meshes; and Kilian et al. (2007) re-cast shapes as points in a Riemannian shape space, enabling natural interpolation operations.

Physically motivated approaches, which are closest in spirit to our goals, have also gained traction. For example, Bao et al. (2005) morphed point-sampled surfaces with a meshless physical simulation, obtaining more physically plausible results than linear interpolation, while Müller et al. (2004) leveraged point samples for both surfaces and volumes to animate elastoplastic objects. Although Chapter 3 shares this high-level objective, it *critically differs* by employing an MPM-based physics simulation and by leveraging differentiable simulation and modern optimization capabilities that were not present in the work of Müller et al. (2004).

The widely used As-Rigid-As-Possible (ARAP) energy (Alexa et al., 2000) and its derivatives deform triangle/tetrahedral meshes via energy minimization for interpolation tasks. Subsequent methods have pursued physical plausibility (Hu et al., 2004; Eisenberger and Cremers, 2020), and solving a Poisson equation on the domain mesh (Xu et al., 2005) reduces artifacts such as shrinkage. Further, Liu et al. (2011) reduced reliance on consistent tetrahedral meshes by operating on triangle surface meshes, broadening practical applicability.

More recently, data-driven methods have converged with geometry processing: Gao et al. (2013) constructed local shape spaces from model databases and posed morphing as a global shortest-path optimization, while neural-field representations have enabled new routes to geometric editing (Yang et al., 2021). Given the growing interplay between learning and simulation (see, e.g., Gibou et al., 2019), *Chapter 3 will present a physics-based methodology that is designed to integrate with learning components in future work.*

2.1.3 Optimization and Control

Shape morphing is closely related to spatiotemporal optimization (Witkin and Kass, 1988), which directly informs our approach. Within this domain, prior work has investigated inverse control of physical systems:

- **Fluid control:** McNamara et al. (2004) minimized a keyframe-based objective by computing gradients via the adjoint method, aligning simulated fluid motion with target states.
- **Deformable-body control:** Coros et al. (2012) optimized nonlinear, time-dependent objectives with SQP, using rest-shape adaptation to control deformable characters.
- **Amorphous-material manipulation:** Zhang et al. (2020) learned controllers for amorphous materials via reinforcement learning with externally controlled rigid contacts.

Chapter 3 builds on this trajectory by framing shape morphing as an inverse control problem. Using differentiable MPM, we directly optimize physically meaningful parameters (e.g., deformation gradients) to reach a prescribed target state, in line with established spatiotemporal optimization principles.

2.1.4 Differentiable Simulation

The approach in Chapter 3 is built on a differentiable simulation framework. Differentiable simulation (see, e.g., Coros et al., 2021) has attracted significant interest in the learning and graphics communities due to its flexibility and efficiency. Its core is the ability to compute **gradients** across the entire simulation via **backpropagation**, which also allows it to be embedded in neural networks (Liang and Lin, 2020).

This paradigm has been applied to soft-robot control (Hu et al., 2019b, 2020), material parameter estimation (Liang et al., 2019; Qiao et al., 2021), and accelerated reinforcement learning (Zamora Mora et al., 2021; Xu et al., 2022). Among various discretizations, the **Material Point Method (MPM)** (Sulsky et al., 1994; Jiang et al., 2016) has seen successful differentiable formulations and implementations (e.g., Hu et al., 2019b), which we leverage as a core component.

Recent work has also addressed **collision handling** in complex dynamical systems within differentiable frameworks (Qiao et al., 2020). Our methodology likewise maintains physically consistent constraints such as external forces. For instance, our morphing results remain elastically plausible under gravitational disturbances, underscoring the necessity and advantage of an MPM-based approach (see Section 3.4).

*Chapter 3 will further extend the applicability of differentiable MPM (including MLS-MPM; Hu et al., 2018) to shape morphing with complex, **topology-aware** configurations—without the intricate mesh setups required by Zhu et al. (2017). Ultimately, this combination of simulation and animation techniques enables both smooth deformations and dynamic topology changes, as demonstrated in our results.*

2.2 Render-Guided Morphing

2.2.1 Gaussian Splatting and Dynamic Scenes

3D Gaussian Splatting (3DGS) (Kerbl et al., 2023) represents a scene as a set of anisotropic 3D Gaussians and renders it with a differentiable, visibility-aware rasterizer, and it has become a standard tool for real-time novel view synthesis. Its success has led to many dynamic variants. 4DGS (Wu et al., 2024) and Deformable 3D Gaussians (Yang et al., 2024) learn deformation fields or per-Gaussian trajectories that reproduce observed non-rigid motion, and SplaTAM (Keetha et al., 2024) reconstructs static scenes online for SLAM. The motion of the dynamic methods is *kinematic*: it fits what the cameras saw, without mass, forces or a material law behind it.

Physics-based variants place a simulator under the Gaussians. PhysGaussian (Xie et al., 2024) drives 3DGS with forward MPM, so the picture follows the physics, but nothing in the picture flows back to it. OmniPhysGS (Lin

et al., 2025) keeps MPM but *learns* the constitutive model of each Gaussian from a video-diffusion prior, and PhysDreamer (Zhang et al., 2024) fits a material field and initial velocities to video priors through differentiable MPM and Gaussian rendering. In these works the physics serves the picture of a given scene; none of them uses a rendering loss to steer a body into a new shape.

2.2.2 Differentiable Physics Coupled with Rendering

Differentiable renderers for points (Yifan et al., 2019), meshes (Laine et al., 2020) and neural fields (Wu et al., 2025) recover geometry and appearance from images, but the shapes they optimize are not physical bodies. In a morphing pipeline that first simulates and then renders, the step between the two is usually a surface reconstruction, such as a sparse volume or a marching-cubes mesh (Museth, 2013; Lorensen and Cline, 1987), that is not differentiated, so a rendering loss cannot reach the controls of the simulation.

Chapter 4 closes this loop for the morphing of Chapter 3. Its silhouette and shading terms are read on the body's own exterior, the surface of a field built from the particles (Zhu and Bridson, 2005), and their gradients reach the same stress control as before together with a rigid-free offset of the outer particle layer. The physics objective becomes a Sinkhorn transport between the body and the target (Feydy et al., 2019), and every optimization window is scored after the controls have been released, so the picture cannot be bought with a state that springs back. The chapter also measures what the render term changes, against an otherwise identical run without it.

2.3 Physics-Grounded Robotic Cutting

2.3.1 Vision–Language–Action Policies

Large language models gained visual input when pretrained image encoders were attached to them: GPT-4 accepts images alongside text (OpenAI et al., 2023), and open models such as LLaVA and MiniGPT-4 connect a CLIP-style encoder (Radford et al., 2021) to a language model through a small learned projection (Liu et al., 2023b; Zhu et al., 2024). Vision–language–action (VLA) policies carry the same idea to robots, mapping a camera image and an instruction to motor commands. RT-2 trains a vision–language model on web data and robot trajectories together and emits actions as discrete tokens (Brohan et al., 2023). OpenVLA is an open-source policy of this kind built on the Prismatic vision–language model (Kim et al., 2025; Karamcheti et al., 2024), Octo is an open generalist policy trained on many robot datasets (Octo Model Team et al., 2024), and RDT-1B is a diffusion foundation model for bimanual manipulation (Liu et al., 2025).

Other work changes what such a policy sees or how its actions are encoded. LLARVA predicts two-dimensional visual traces of the end effector as an auxiliary target (Niu et al., 2025), TraceVLA prompts the policy with the traces of past motion (Zheng et al., 2025), and LAPA and Moto pretrain on videos with latent actions or motion tokens learned without action labels (Ye et al., 2025; Chen et al., 2025b). ForceVLA adds force sensing to the policy input (Yu et al.,

2025). These methods change the policy or its inputs; the demonstrations they learn from are not generated under the dynamics of a deformable material. Chapter 5 takes the opposite route. It fine-tunes three existing policies, RDT-1B, Octo and OpenVLA, without changing their architectures, measures where they fail to ground an instruction in the geometry of the food, and compares RDT trained on demonstrations generated and selected under MPM material dynamics with RDT trained on geometry-only demonstrations. The simulated forces are recorded with the data but are not given to the policies.

2.3.2 Large-Scale Robot Learning Datasets

Community efforts have consolidated broad, open datasets for imitation and VLA-style policies, but almost none target deformable cutting. *Open X-Embodiment (RT-X)* aggregates over one million real-robot trajectories spanning 22 embodiments by pooling 60 prior datasets from 34 labs into a standardized RLDS/`tfrecord` format, and additionally releases pre-trained RT-X checkpoints to study cross-embodiment transfer (Open X-Embodiment Collaboration et al., 2024). Beyond X-Embodiment, *RoboNet* pioneered sharing multi-robot experience (about 15M video frames across seven platforms) for vision-based manipulation (Dasari et al., 2020). *BridgeData V2* contributes 60,096 trajectories across 24 environments on a low-cost platform, enabling goal/language conditioning and cross-institution generalization (Walke et al., 2023). *DROID* scales in-the-wild teleoperation to 76k demonstrations ($\sim$350 hours) over 564 scenes and 86 tasks (Khazatsky et al., 2024). For evaluation, *LIBERO* provides language-conditioned manipulation suites, but does not include deformable cutting (Liu et al., 2023a). Cutting-specific resources are largely simulation-centric: *DiSECt* offers a differentiable cutting simulator with small real datasets for calibration (Heiden et al., 2021); *SliceIt!* proposes a dual high-fidelity cutting+robot simulator with limited real slicing for real2sim2real transfer (Beltran-Hernandez et al., 2024); and the recent *TopoCut* introduces a benchmark for multi-step cutting with spectral rewards and diffusion policies (Wang et al., 2025). The general datasets thus offer scale without a model of material separation, while the cutting simulators model separation without the language-conditioned, multi-view robot data that a VLA policy is trained on. Chapter 5 builds such data in simulation: its IntSim environment couples an MLS-MPM material solver to a ManiSkill robot (Gu et al., 2023), and the resulting CulinaryCut benchmark pairs cutting trajectories over fifteen foods with language instructions, multi-view images and contact-force records, which are used to generate and evaluate the demonstrations.

CHAPTER 3

Physics-Based Shape Morphing

This chapter is adapted from Xu et al. (2025), published in IEEE Transactions on Visualization and Computer Graphics, with co-authors Michael Xu, David Levin, and David Hyde.

3.1 Overview

The basis for the present work is the Material Point Method (MPM), which is a particle-in-cell method for simulating continuum materials. MPM's hybrid particle-grid formulation requires no fixed connectivity between the discretized elements of the material domain (the material points). This naturally allows for large topology-changing deformations when compared to mesh-based methods like the Finite Element Method. In the more than three decades since its introduction in 1994 (Sulsky et al., 1994), various formulations and improvements have been made to the original MPM, e.g. (Hu et al., 2018; Wolper et al., 2019; Fang et al., 2020; Bardenhagen and Kober, 2004; Sołowski et al., 2021; de Vaucorbeil et al., 2020).

To provide background, we review the Moving Least Squares MPM (MLS-MPM) variant that we use in this work (Hu et al., 2018).

The method starts with a continuum discretized into material points that carry physical properties of its domain, such as mass m_p, volume V_p, position $\mathbf{x}_p$, velocity $\mathbf{v}_p$, and deformation gradient $\mathbf{F}_p$. We use Affine Particle-in-Cell (APIC) transfer kernels (Jiang et al., 2017), which further requires an affine matrix $\mathbf{C}_p$ to be stored on the particles.

In MPM, updates to particle states are generally computed on a background grid. On each timestep, the background grid interpolates material point quantities like velocity to grid nodes, computes updates for quantities like momentum,

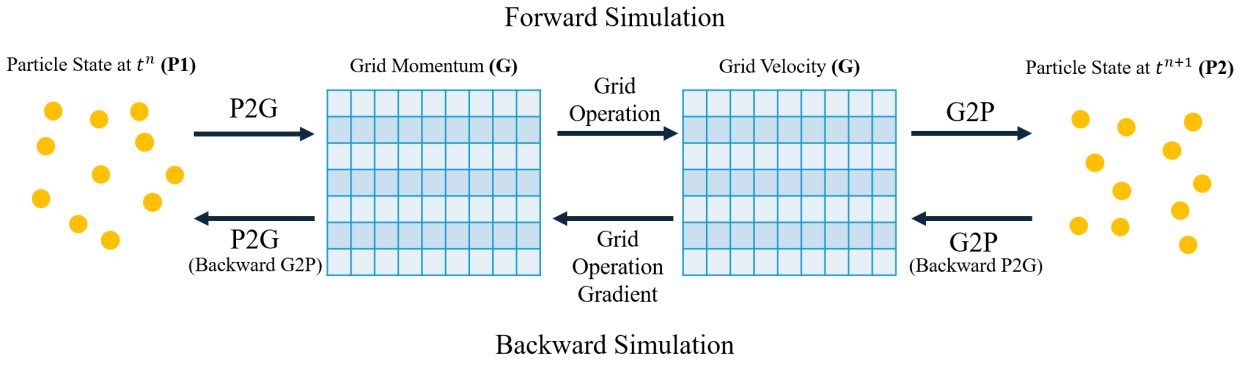


Figure 3.1: **Simulation overview.** A timestep in MPM consists of five major computations: **P1** → **P2G** → **G** → **G2P** → **P2**. Particles store their own physical properties but interact with each other through the background simulation grid.

then interpolates updated quantities back to the particles. The use of a regular background grid enables using efficient, scalable solvers that have been optimized for grids.

Each timestep of the MLS-MPM algorithm we implement can be split into five major steps, as seen in Figure 3.1:

1. **P1 (Particle Stress Calculation)**. For each particle, calculate its Piola-Kirchhoff stress tensor from its deformation gradient.
2. **P2G (Particle to Grid)**. For each particle, interpolate its mass and momentum to the grid with APIC transfer kernels (Jiang et al., 2015, 2017).
3. **G (Grid Velocity Calculation)**. For each grid node, solve for its updated momentum (incorporating any desired external forces) and velocities. Any time integration is performed using symplectic Euler.
4. **G2P (Grid to Particle)**. For each particle, interpolate its updated velocity and affine matrix from the grid.
5. **P2 (Particle Kinematic Update)**. For each particle, update its deformation gradient and position using the updated particle quantities and timestep.

We refer the reader to Hu et al. (2018) and to the appendix for more detailed explanations of the MLS-MPM algorithm.

3.1.1 Differentiable Simulation

We implemented MLS-MPM, which is naturally differentiable; analytical gradients have been derived by Hu et al. (2019b). We manually implemented the necessary analytical gradients to have more control over the implementation and avoid potential issues with software-based automatic differentiation (Johnson et al., 2025). In our method, these manually derived gradients were then used in conjunction with the non-stochastic version (i.e., all particles are used for each iteration) of the Adam optimizer (Kingma and Ba, 2015) to perform the optimization, ensuring stable and efficient convergence. In addition to optimization, a differentiable simulator naturally requires storing the feedforward simulation network to allow for the backpropagation of derivatives. We implemented this network by storing the computation graph of each timestep layer $\mathbf{T}^n$, where a layer performs each step of an MPM timestep:

$$\mathbf{T}^n = \mathbf{P1}^n \rightarrow \mathbf{P2G}^n \rightarrow \mathbf{G}^n \rightarrow \mathbf{G2P}^n \rightarrow \mathbf{P2}^n. \tag{3.1}$$

Backpropagation can then be used on the simulation network to compute gradients via the chain rule:

$$\mathbf{T}^1 \leftarrow \cdots \leftarrow \mathbf{T}^n \leftarrow \mathscr{L}. \tag{3.2}$$

We refer to Hu et al. (2019b) for the derivation of the analytical derivatives used in the backward pass. Our method involves controlling particles' deformation gradients (Section 3.2), so we introduce a control deformation gradient

term separate from the time-evolved deformation gradients. The exact modification to the MPM algorithm can be seen in the appendix. These control deformation gradients are added in a way such that the loss gradient with respect to them is the same as the loss gradient with respect to the time-evolved deformation gradients.

3.2 Deformation Gradient Control

In continuum mechanics, an object has an initial domain in space denoted as Ω_0. A point belonging to the object can be defined by its initial position $\mathbf{X} \in \Omega_0$. As the object moves through space and time, the world coordinate of a point $\mathbf{x}$ at some time t can be represented as $\mathbf{x} = \phi(\mathbf{X},t) \in \Omega_t$, where ϕ is known as a deformation map. The deformation gradient $\mathbf{F}$ is defined as the Jacobian of the deformation map:

$$\mathbf{F}(\mathbf{X},t) = \frac{\partial \phi}{\partial \mathbf{X}}(\mathbf{X},t). \tag{3.3}$$

It is common for elastic constitutive models to be defined on the deformation gradient $\mathbf{F}$ such that internal energy is only built up when $\mathbf{F}$ represents a non-rigid transformation between the rest state $\mathbf{X}$ and the world state $\mathbf{x}$. Our elastic energy function of choice is fixed corotated elasticity (Stomakhin et al., 2012; Jiang et al., 2016), whose Piola–Kirchhoff stress tensor is:

$$\mathbf{P(F)} = 2\mu(\mathbf{F} \text{ - } \mathbf{R}) + \lambda(J-1)J\mathbf{F}^{-T}, \tag{3.4}$$

where J is the determinant of $\mathbf{F}$ and $\mathbf{R}$ is its rotational component, which can be computed via the polar decomposition of $\mathbf{F}$ (Gast et al., 2016).

Using this elastic model, we can manually control internal forces in our MPM simulation through the particle deformation gradients (we are inspired by the preliminary work of Xu and Levin (2023)). An example of this would be manually setting deformation gradients to the identity matrix $\mathbf{F} = \mathbf{I}$ after the shape has deformed. This corresponds to setting the new rest state of the object to the current state. Another way to control internal forces through deformation gradients would be setting expansion ($\alpha > 1$) and contraction ($\alpha < 1$) deformation gradients $\mathbf{F} = \alpha\mathbf{I}$.

One might alternatively consider an approach that applies external forces to drive shape changes. However, if performing a shape morph using external forces, one needs to know precisely where to apply those forces on every timestep, and possibly extrinsic geometric surface properties like curvature; but as MPM represents objects with particles, it is not obvious where the surface of an object is at a given time step and what its local surface curvature is (especially when dealing with fine-scale/sub-grid geometric details). Recent works have demonstrated the importance of and necessary care when dealing with MPM boundary representations and curvature, in simulating surface tension (Hyde et al., 2020; Chen et al., 2021). In contrast, directly optimizing the per-particle deformation gradient $\mathbf{F}$ eschews the modeling questions and numerical issues of dealing with external forces and takes advantage of the inherent properties of MPM. By focusing on $\mathbf{F}$ rather than external forces, we preserve physical consistency and achieve more

stable, realistic morphing results in the context of MPM.

Of course, to actually drive a shape morph, an automatic method for per-particle deformation gradient control is required. We achieve this through the optimization of the feedforward simulation network, which requires a loss function to be defined.

3.2.1 Automatic Per-Particle Deformation Gradient Updates

A key detail in our method is how the control deformation gradient $\tilde{\mathbf{F}}_p$ is optimized and accumulated into the particle deformation gradient $\mathbf{F}_p$ over time. At each control timestep, we apply a direct update to $\tilde{\mathbf{F}}_p$ through gradient-based optimization, and accumulate it into $\mathbf{F}_p$. The complete formulation is provided in Appendix A11, which describes how the update is applied. This mechanism ensures that shape changes are persistent and physically consistent, rather than acting as temporary impulses. To further improve stability and smoothness of the optimization, we apply a smoothing factor when updating $\tilde{\mathbf{F}}_p$.

3.2.2 Preliminary Shape Loss Functions

To morph a shape from an input to a target geometry, we initially considered two simple loss function choices:

Particle positions. Given a simulation point cloud of size N, where each material point has a unique index $p \in [1,N]$, we require an equally sized target point cloud. A position-based loss function can then be defined as the sum of the squared distances between points $\mathbf{x}_p$ in the simulation point cloud and points $\mathbf{x}_p^*$ in the target point cloud:

$$\mathscr{L} = \sum_p \frac{1}{2} ||\mathbf{x}_p - \mathbf{x}_p^*||^2. \tag{3.5}$$

In our experiments, this loss function is suitable for locomotion tasks or simple shape changes such as expansion or shearing; however, since it merely penalizes average position errors, using this loss function can leave outlier points in the morph and generally does not provide visually adequate morphs in the presence of fine-scale features or topology changes. Another downside of using the position loss function is the requirement of an equally sized target point cloud with point-to-point correspondences. There are techniques available for computing the point-to-point correspondences of two different shapes (van Kaick et al., 2011), but this places a constraint on the optimal positions of individual material points.

Nodal mass. In MPM, we have the luxury of a background grid, whose nodes can store interpolated quantities such as mass from the particles. Using a nodal mass loss function instead of a point position loss function can alleviate many requirements on our input geometry, such as the need to have identical numbers of particles in the input and target geometries.

We can then compute the sum of the squared differences between the nodal masses m_i and m_i^* of the interpolated input masses and the interpolated target masses, respectively:

$$\mathscr{L} = \sum_i \frac{1}{2}(m_i - m_i^*)^2. \tag{3.6}$$

This loss function is not easy to optimize with general optimizers and can result in "mass ejections," where clumps of material points are violently propelled outward. Figure 3.2 illustrates such a scenario, where regions of the domain with large mass discrepancies cause particles to be ejected. We believe that mass ejections are a consequence of converging to local minima of the nodal mass loss function, in which these ejections lead to the quickest decrease of the loss value. This problem becomes more noticeable with larger differences between the initial and target shapes. In such cases, a stronger force is applied in the parts of the domain with less overlap between input and target geometries (hence, a large difference in mass), resulting in even greater mass ejection.

Knowing this, we designed a small modification to the nodal mass loss function to eliminate mass ejections.

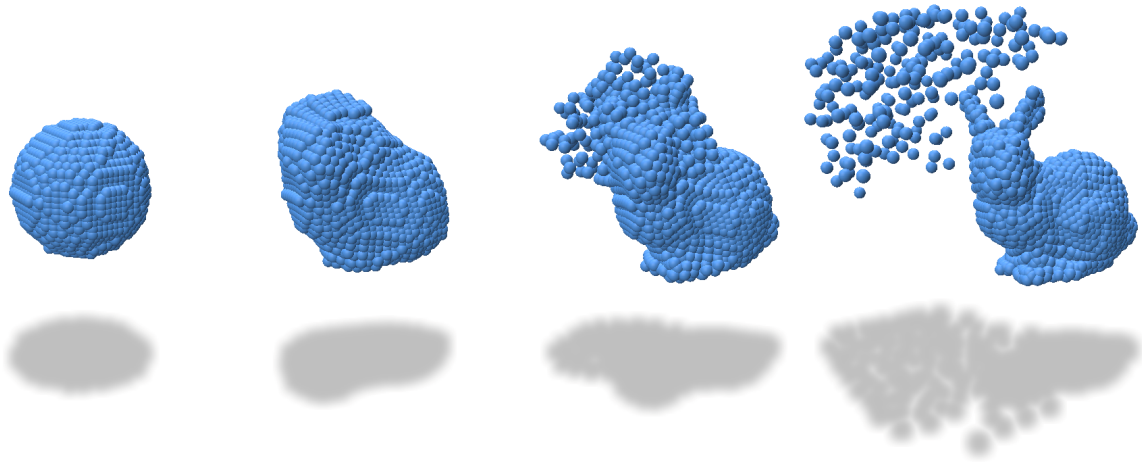


Figure 3.2: **An example of mass ejections occurring under the naive nodal mass loss.** Particles are forcefully ejected outward, leading to unstable or unrealistic behavior.

3.2.3 Log-Based Mass Loss Function

To address these limitations, we propose a novel loss formulation that simultaneously improves numerical stability and enforces physical plausibility. Specifically, we aim to construct a smoother loss landscape that is less sensitive to outliers and promotes stable optimization. To this end, we introduce a log-based mass loss function, defined as:

$$\mathscr{L} = \mathscr{L}_{\text{mass}} + \mathscr{L}_{\text{penalty}}, \tag{3.7}$$

$$\mathscr{L}_{\text{mass}} = \sum_i \frac{1}{2}(\ln(m_i + 1 + \varepsilon) - \ln(m_i^* + 1 + \varepsilon))^2, \tag{3.8}$$

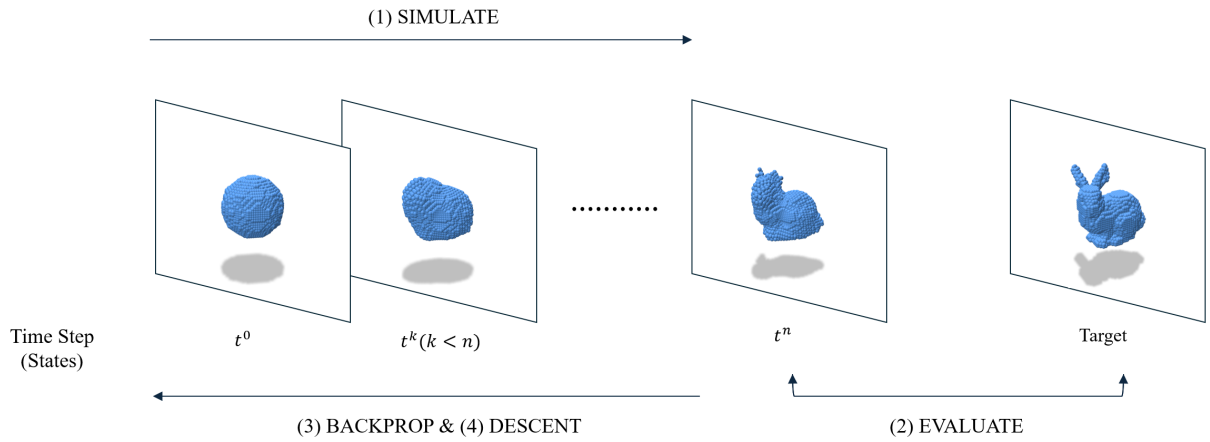


Figure 3.3: **Morphing method.** Step-by-step optimization process of the simulation networks, aligned according to the causality principle. The Adam optimizer for the control variables is implemented in order for each time step layer of the simulation and includes steps such as forward calculation (SIMULATE), loss function calculation (EVALUATE), backward calculation (BACKPROP), and control variable update (DESCENT). This optimization process can be repeated until the norm of the loss gradient falls below a certain tolerance.

$$\mathscr{L}_{\text{penalty}} = \sum_i 1_{\{m_i < m_{\min}\}} \cdot w \cdot (m_{\min} - m_i)^2. \tag{3.9}$$

To ensure numerical stability during the log transformation, we add a small constant ε as well as 1 to the mass values. We use a fixed value of $\varepsilon = 1 \times 10^{-4}$ throughout our experiments to ensure stable and consistent behavior.

Additionally, to prevent physically implausible mass loss, we introduce a penalty term $\mathscr{L}_{\text{penalty}}$ for cases where the predicted mass m_i falls below a predefined minimum threshold $m_{\min}$. This penalty is scaled quadratically with the difference and weighted by a factor w, which we set to 10.0 in all our experiments. The magnitude of w plays an important role in the optimization dynamics. Increasing w makes the result closer to the target mesh but can introduce oscillations near the optimum. Conversely, decreasing w reduces such oscillations but allows the result to deviate more from the target mesh.

This loss function design allows us to mitigate the impact of outliers while softly constraining unrealistic mass depletion. As a result, we are able to effectively suppress mass ejection artifacts during simulation, as demonstrated in Section 3.4.

3.3 Morphing Method

Given our simulation framework and loss function, we can now describe our proposed morphing algorithm. The main idea is based on Adam optimization of control deformation gradients.

3.3.1 Simulation Optimization

In our simulation network in Figure 3.3, the ordering of operations is governed by the principle of causality. This means that any control or intervention applied at earlier stages has a direct impact on the inputs of subsequent layers, and consequently, on their outputs. In this context, "control" refers to any manipulation or condition imposed on the system, while "layers" represent the sequential stages of the simulation process. We outline the steps to perform an iteration of optimization on the control variables for a single timestep layer $\mathbf{T}^n$ of the simulation network:

1. **SIMULATE**. Compute the forward pass starting from the control layer to the final layer.
2. **EVALUATE**. Compute the loss function via the final layer.
3. **BACKPROP**. Compute the backward pass to get the gradient of the loss function with respect to the control variables in the control layer.
4. **DESCENT**. Update the control variables by the loss function with Adam optimizer using the origin stepsize α, which can be fixed or computed via line search. In our case, we use a bisection line search method (Algorithm 2) to find a step size that sufficiently decreases loss.

This process of optimization can be repeated until convergence, which can be defined as the norm of the loss gradient being below a specified tolerance. Once the optimization for a single layer has been satisfactorily completed, the optimization can move on to the next layer.

3.3.2 Control Layers

To save on computation time, and to allow time for the effects of previously controlled deformation gradients to propagate, we only compute control deformation gradients on a subset of the timesteps. We denote the layers where we compute control deformation gradients as "control layers." Our simulation uses a timestep size of $\Delta t = 1/120$ seconds, and we apply control gradients every 10 timesteps.

We outline in pseudocode our algorithm for computing the control deformation gradients using one optimization pass in Algorithm 1. To make the notation concise, we let $\tilde{\mathbf{F}}^n$ represent the control deformation gradients at timestep n. We also let Loss($\tilde{\mathbf{F}}^n$) represent forward simulation from timestep n to N, followed by computing the loss between the material points at timestep N and the target shape. We note that the partial derivative of the loss is first computed per grid node with respect to the nodal grid mass m_i, which will in general be distinct for different grid nodes. Based on this quantity, we can then use MLS-MPM interpolatory weights to compute the derivative of loss with respect to a particle's location $\mathbf{x_p}$, and then apply the chain rule to compute derivatives of the loss with respect to per-particle control deformation gradients, the set of which is $\tilde{\mathbf{F}}^n$. The optimization step in Algorithm 1 can then update each

particle's control deformation gradient. We emphasize that with this algorithm, each particle will, in general, end up with distinct control deformation gradients.

Algorithm 1 Control Deformation Gradient Computation

Require: N, the final simulation timestep
Require: Δn, the control period
Require: $i_{\max}$, the max number of iterations per control timestep
Require: κ, the convergence tolerance for Adam optimizer
Require: α, the original Adam optimizer step size
Require: β_1, β_2, the decay rates for the moment estimates
Require: $\varepsilon_{\text{adam}}$, a constant to avoid division by zero in Adam

$\tilde{\mathbf{F}}^{1:N} \leftarrow \mathbf{0}$ ▷ Initialize control deformation gradients
$m^{1:N} \leftarrow \mathbf{0}$ ▷ First moment vector
$v^{1:N} \leftarrow \mathbf{0}$ ▷ Second moment vector
$t \leftarrow 0$
$n \leftarrow 1$
while $n < N$ **do**
  $i \leftarrow 0$
  while $i < i_{\max}$ **do**
    $L \leftarrow \text{Loss}(\tilde{\mathbf{F}}^n)$ ▷ Loss after sim from n to N
    $\mathbf{g} \leftarrow \dfrac{\partial L}{\partial \tilde{\mathbf{F}}^n}$ ▷ Gradient
    $m^n \leftarrow \beta_1 m^n + (1-\beta_1)\,\mathbf{g}$
    $v^n \leftarrow \beta_2 v^n + (1-\beta_2)\,(\mathbf{g} \odot \mathbf{g})$ ▷ Hadamard product
    $\hat{m}^n \leftarrow \dfrac{m^n}{1-\beta_1^{t+1}}$ ▷ Bias correction (use $t+1$ after update)
    $\hat{v}^n \leftarrow \dfrac{v^n}{1-\beta_2^{t+1}}$
    $\alpha \leftarrow \text{ComputeStepSize}\big(L, \tilde{\mathbf{F}}^n, \hat{m}^n, \alpha\big)$
    $\tilde{\mathbf{F}}^n \leftarrow \tilde{\mathbf{F}}^n - \alpha \dfrac{\hat{m}^n}{\sqrt{\hat{v}^n} + \varepsilon_{\text{adam}}}$
    if $\|\mathbf{g}\| < \kappa$ **then**
      break
    end if
    $i \leftarrow i+1$
    $t \leftarrow t+1$
  end while
  $n \leftarrow n + \Delta n$
end while
return $\tilde{\mathbf{F}}^{1:N}$

3.3.3 Multiple Optimization Passes

As described in the prior subsection, we use the routine of Algorithm 1 to compute optimal control deformation gradients for each control layer. However, this strategy is suboptimal: for instance, when we compute optimal control deformation gradients for the first control layer, we are using values of 0 for the control deformation gradients at all subsequent control layers, when we know that we will later optimize those to also have non-zero values. Accordingly, to obtain improved control deformation gradient values, we can utilize a multi-pass optimization scheme where, after

Algorithm 2 ComputeStepSize (L_0, $\mathbf{x}$, $\Delta\mathbf{x}$, α)

Require: L_0, the original loss
Require: $\mathbf{x}$, the original vector argument
Require: $\Delta\mathbf{x}$, the step direction vector
Require: α, the original step size
1: $L \leftarrow \text{Loss}(\mathbf{x} + \alpha\Delta\mathbf{x})$
2: **while** $L > L_0$ **do**
3: $\quad \alpha \leftarrow \alpha/2$
4: $\quad L \leftarrow \text{Loss}(\mathbf{x} + \alpha\Delta\mathbf{x})$
5: **end while**
6: **return** α

optimizing control deformation gradients at all control layers, we can go through and re-run the optimization routine of Algorithm 1 using the computed $\tilde{\mathbf{F}}^{1:N}$ as initial/improved guesses for the optimization.

For a clear analysis of the tradeoffs, we evaluated the impact of the number of passes and the number of gradient iterations on performance based on the data shown in Table 3.1. This ablation study provides useful insights into the relationship between loss values and execution time. For example, there is a significant improvement in loss between 1 pass and 3 passes, while the execution time remains the same. This suggests that multi-pass optimization can improve the loss without degrading performance.

Additionally, our analysis of the limitations of increasing the number of passes shows that beyond 6 or 12 passes, the loss values do not improve and may even increase. This indicates the limits of optimization, highlighting that a larger number of passes does not always guarantee better results.

Lastly, we observed that despite varying the number of passes and gradient iterations, the execution time remains nearly constant. This is a significant finding, as it emphasizes that further optimization can reduce the loss without negatively impacting performance.

Table 3.1: **Varying the number of optimization passes and optimization iterations per pass.**

Passes	# of Gradient Iter	Final Loss	Elapsed Time [s]
1	12	92.3826	187
3	4	91.0895	187
6	2	93.4914	187
12	1	93.1483	186

3.3.4 Chaining Multiple Simulation Networks

The amount of memory required for a simulation network is directly proportional to the number of simulation timesteps. This can make long animations require an infeasible amount of memory. Furthermore, backpropagation through longer networks can result in accumulating floating point errors and in problems like vanishing gradients (Hochreiter, 1998).

Therefore, for longer morph animations, we can divide the morph into different segments and optimize one segment at a time. We construct a new simulation network using the final layer of the previous optimized network. The animation produced by the previous network is saved to disk before the memory of the previous network is freed. The new network is then optimized, and this process repeats. In our experiments, we split the optimization for long animations into networks of 10 timesteps each, i.e., 0.083-second segments at 120 FPS. By chaining optimized simulation networks together, we can produce indefinitely longer animations without any increased memory requirements and with reduced computational cost.

3.4 Examples

Table 3.2: **All example pairs used in our morphing experiments are listed below.** We report the number of simulation steps (time steps), particle density, smoothing factor γ, and the number of particles for both the initial and target meshes. For all examples, we used a fixed timestep size $\Delta t = 1/120\,\text{s}$, performed 4 gradient descent iterations (3 optimization passes), and employed a 32^3 simulation grid. *Note:* Due to the small timestep size selected for high-resolution visualization, the resulting simulations effectively represent slow-motion morphing sequences.

Start Mesh	End Mesh	# of Timesteps	Density	Smoothing factor γ	Target # of Particles	Initial # of Particles
Sphere	Bunny	420	75	0.955	1.116×10^4	0.92×10^4
	Bob	600	60	0.955	1.116×10^4	1.06×10^4
	Cow	400	60	0.955	1.116×10^4	0.76×10^4
	Car	450	60	0.955	1.116×10^4	1.21×10^4
Bob	Sphere	450	70	0.950	1.06×10^4	1.11×10^4
	Bunny	600	75	0.955	1.06×10^4	0.92×10^4
	Cow	500	75	0.955	1.06×10^4	0.76×10^4
	Car	600	60	0.940	1.06×10^4	1.06×10^4
Bunny	Sphere	500	100	0.965	0.92×10^4	1.116×10^4
	Bob	630	120	0.96	0.92×10^4	1.116×10^4
	Cow	400	110	0.955	0.92×10^4	0.76×10^4
	Car	850	75	0.955	0.92×10^4	1.21×10^4
Cow	Sphere	400	75	0.955	0.76×10^4	1.116×10^4
	Bunny	400	75	0.955	0.76×10^4	0.92×10^4
	Bob	600	75	0.955	0.76×10^4	1.06×10^4
	Car	600	75	0.955	0.76×10^4	1.21×10^4
Car	Sphere	600	75	0.950	1.21×10^4	1.116×10^4
	Bunny	600	75	0.930	1.21×10^4	0.92×10^4
	Bob	300	60	0.955	1.21×10^4	1.06×10^4
	Cow	600	75	0.955	1.21×10^4	0.76×10^4

We demonstrate our methodology on a wide range of physically-based morphing animations in three-dimensional space. Leveraging MPM, our objects can seamlessly undergo topological morphs driven by internal elastic forces. We use a modified explicit time integration scheme for MLS-MPM (see the appendix), which helps prevent instability when taking larger timesteps. A full list of variables involved in our MPM implementation is also found in the appendix.

The pairwise morphing results across five distinct meshes are shown in Figure 3.7, and the corresponding experimental parameters are summarized in Table 3.2. The table lists the number of particles in both the initial and target meshes. While the number of particles does not change during the morphing process—the particle count used for each animation is fixed to the number of particles in the initial mesh—the number of particles in the target mesh can also

Figure 3.4: **Sphere to bunny morphing animation.** Key stages show the sphere evolving into a detailed bunny form, highlighting our method's precision in capturing sharp features like the ears.

affect performance (so we report this number as well). This is because the target mesh particles are used in computations of the various derivatives of the loss function used during the backpropagation step of our algorithm. For all our examples, MPM point cloud visualization was performed using Polyscope (Sharp et al., 2019), and rendering was conducted using Houdini. We ran all experiments on a machine with an Intel i9-13950HX CPU, 32GB RAM, and an

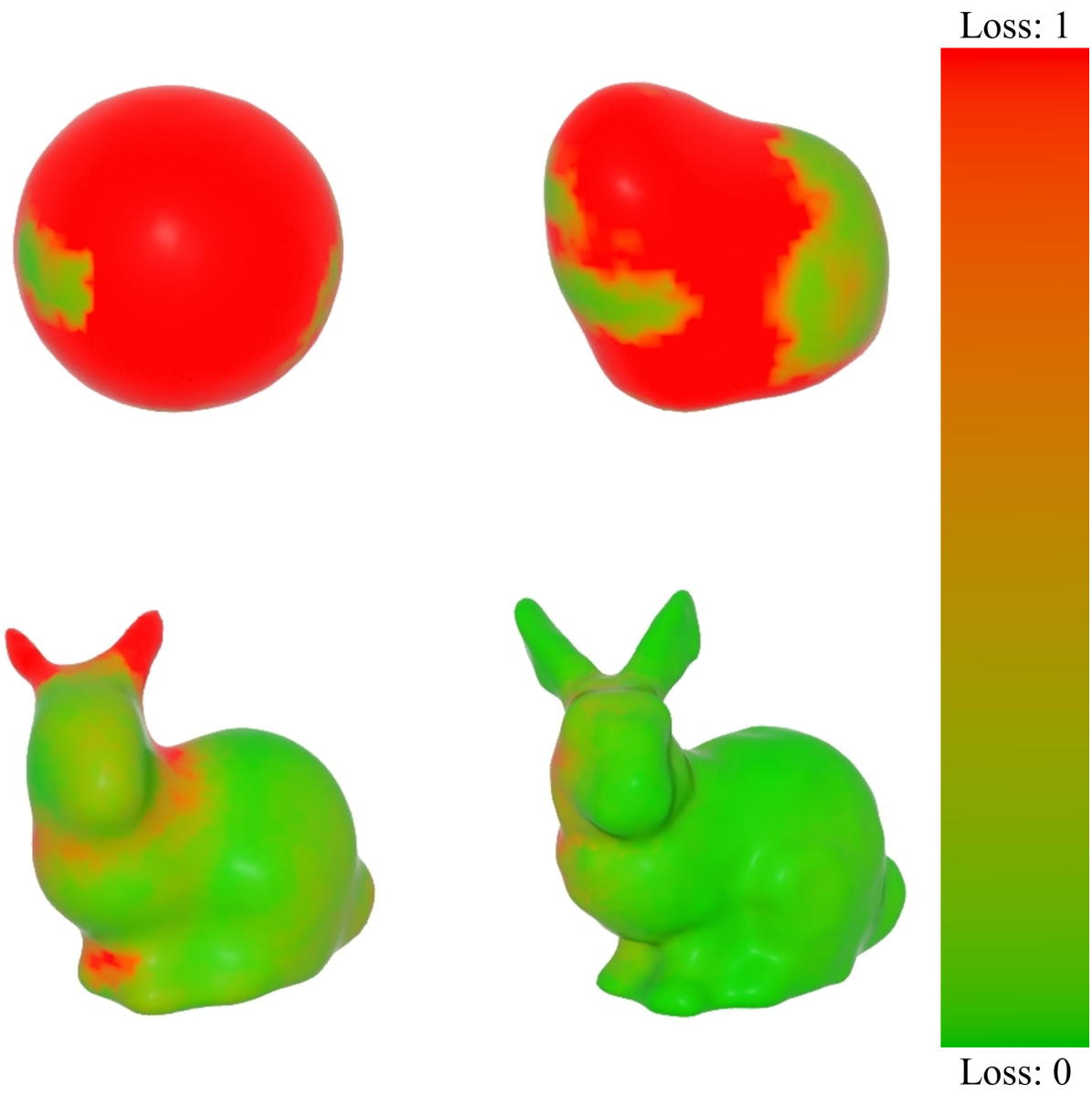


Figure 3.5: **Visualizing our loss function while morphing a sphere to a bunny.** Loss is normalized from 0 to 1 for ease of visualization. We observe a smooth decrease in the loss, including in the finer regions of the target geometry towards the end of the morph.

NVIDIA RTX 4090 GPU.

Sphere to Bunny. In our first example, we present a morphing animation from a sphere to a bunny, as depicted in Figure 3.4. This demonstrates a successful shape morph using our log-based nodal mass loss function (see supplemental video). While using other loss functions might result in phenomena like mass ejection, our method, as shown in the figure, allows for smooth transformation without ejected particles or other unpleasant artifacts.

One of the most notable aspects of this example is the representation of the bunny's ears. Our methodology allows

us not only to match the bulk volume of a target mesh but also to faithfully reproduce its sharp and thin features.

We also plot the loss function, interpolated to the surface, as the geometry deforms; see Figure 3.5. The loss function is normalized from 0 to 1 in this figure for ease of visualization. We observe that the loss function smoothly decreases as the morph proceeds, with fine-scale details of the target mesh generally being the last parts of the morph to reach an acceptable state.

Duck (Bob) to Cow. In the second example, we illustrate an animation transitioning from a duck model ("Bob") to a cow model ("Cow") in Figure 3.6. This case is interesting due to the changing topology between the two geometries. Our method achieves a smooth morph of the geometry and topology simultaneously, which is a natural benefit of our underlying MPM framework.

Table 3.3: **Performance for various shape morphing sequences, including accuracy (Final Loss is bolded for emphasis).**

Start Mesh	End Mesh	Time [s]	Loss (Initial → Final)	Acc [%]
Sphere	Bunny	187	5302.69 → **91.09**	98.28
	Cow	172	7685.97 → **124.43**	98.38
	Car	188	8163.73 → **104.36**	98.72
	Bob	290	11600.0 → **300.53**	97.41
Bob	Sphere	173	12018.7 → **388.20**	96.77
	Bunny	332	12055.2 → **166.26**	98.62
	Cow	195	12218.3 → **206.33**	98.31
	Car	266	11516.7 → **368.94**	96.80
Bunny	Sphere	223	6104.12 → **119.75**	98.04
	Bob	278	14184.2 → **608.07**	95.71
	Cow	165	4469.88 → **70.51**	98.42
	Car	415	11945.2 → **870.81**	92.71
Cow	Sphere	123	8606.47 → **465.71**	94.59
	Bunny	120	3694.31 → **116.35**	96.85
	Bob	178	12218.3 → **575.58**	95.29
	Car	204	14316.9 → **685.36**	95.21
Car	Sphere	333	9980.66 → **122.11**	98.78
	Bunny	335	11945.2 → **108.53**	99.09
	Bob	309	11516.7 → **398.10**	96.54
	Cow	333	14316.9 → **167.97**	98.83

Morphing Between Various Mesh Pairs. To further test our method, we explored pairwise deformations across five distinct meshes (Figure 3.7) under varying geometric conditions, with the experimental configurations summarized in Table 3.2. All attempted morphs were successful, demonstrating the robustness and adaptability of our approach. We also explored how varying smoothing factors and optimization algorithms affected the results:

Smoothing Factor. An additional element in achieving a high success rate was the application of a simple interpolation step during post-processing:

$$\mathbf{F}^{n+1} \leftarrow (1-\gamma)\,\mathbf{F}^{n+1} + \gamma \mathbf{F}^{n},$$

where $\gamma \in [0,1]$ is a smoothing factor. Larger γ values effectively suppress abrupt changes, leading to a smoother morph, whereas smaller γ values preserve faster convergence but may induce pronounced oscillations. In practice, we observed that setting γ beyond 0.5 made little difference visually, yet using $\gamma = 1$ often prevented the morph from

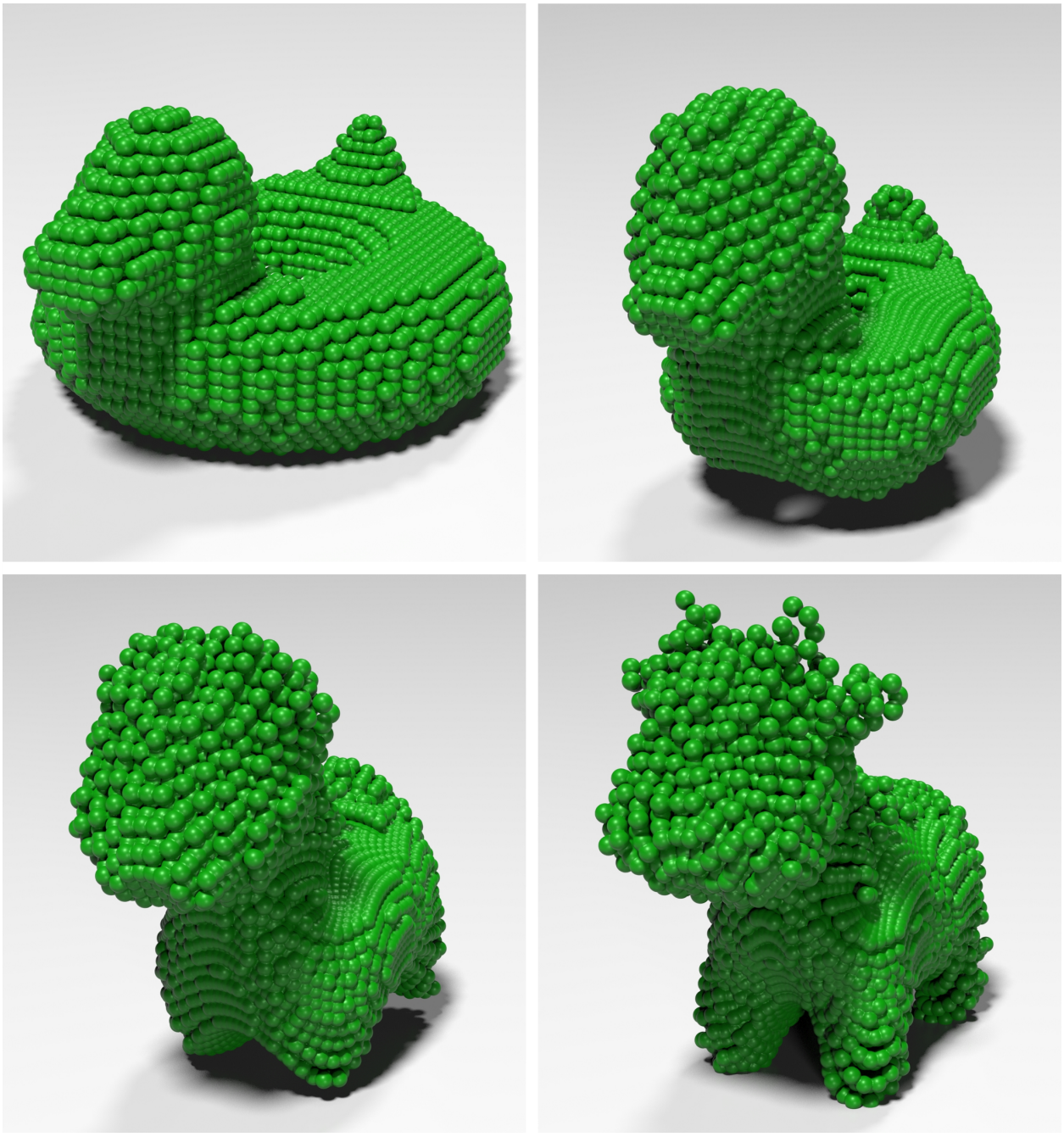


Figure 3.6: **Morphing Duck (Bob) to Cow.** As the model deforms into a cow, we observe a smooth topology change as the center hole of the inflatable duck closes. This example evidences our method's ability to handle simultaneously evolving geometry and topology.

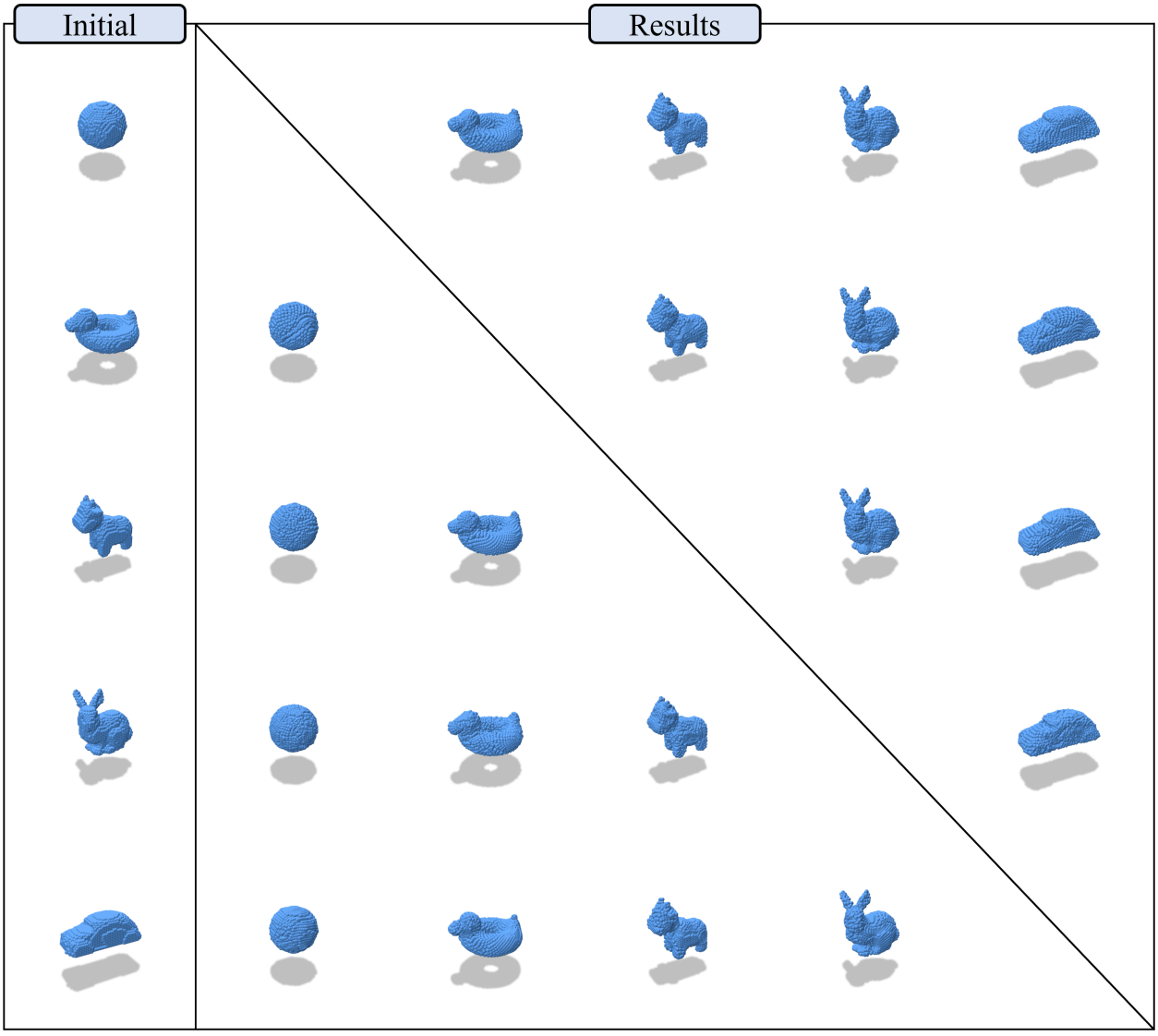


Figure 3.7: **Pairwise shape morphing results among five distinct geometries.** The left column shows the five initial meshes (sphere, duck/Bob, cow, bunny, and car); the grid on the right shows the resulting shapes for different targets. Self-morphs along the diagonal are omitted.

completing correctly. We primarily fine-tuned γ within the range of 0.95 to 0.98, striking a good balance between speed and stability. All specific γ values employed in our experiments are reported in Table 3.2. They were hand-tuned per example but were all generally set around 0.955. A more detailed demonstration of the the effects of varying γ (and its relative robustness above 0.5) can be found in our supplementary video.

Optimizer Comparisons. We also performed a series of experiments to compare different optimizers, including Adam, Gradient Descent (GD), and the recently introduced Lion (Evolved Sign Momentum) (Chen et al., 2023), highlighting the trade-off between convergence speed and stability. Figure 3.8 shows loss curves for each optimizer. Adam consistently achieved lower loss in fewer iterations but exhibited significant oscillations near the optimum. In contrast, GD and Lion converged more steadily, though requiring more steps overall. By integrating the smoothing factor into Adam updates, we were able to retain much of Adam's fast convergence advantage while notably reducing the high-frequency vibrations in the morphing result. For a visual demonstration of how smoothing affects Adam's oscillations, we again refer to our supplementary video.

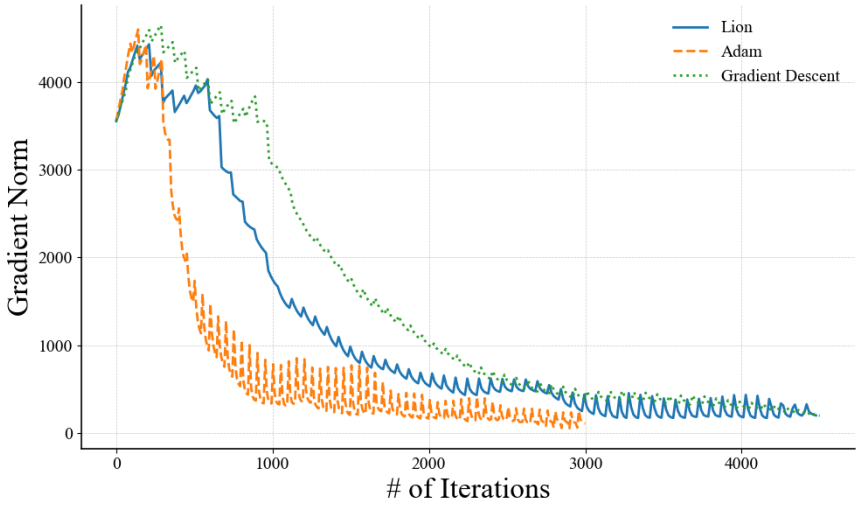


Figure 3.8: **Comparison of loss curves under different optimizers (Adam, GD, Lion) during the D to Dragon morphing task ($\beta_1 = 0.90$; learning rates: Lion/Adam = 1×10^{-3}, GD = 0.1).** Adam converges rapidly but suffers from strong oscillations near the optimum; GD and Lion converge more smoothly at a slower pace. Incorporating a smoothing factor (γ) into Adam's update reduces these oscillations, striking a balance between speed and stability.

Performance and parallelization. Another benefit of our method is that the majority of our algorithm is parallelizable, due to our foundation on MPM. We produced a CPU-parallel implementation using OpenMP, leading to significantly shorter runtimes than a serial code.

The performance of our parallel code on the various examples is reported in Table 3.3. We also demonstrate the strong scaling of our method in Figure 3.9; while our algorithm is not fully parallelizable and hence we do not expect perfect linear scaling, we still observe useful scaling from 1 to 24 cores (over 5x speedup).

Complex Object Animation. In our complex object animation examples (see Figures 3.10 and 3.11), we demon-

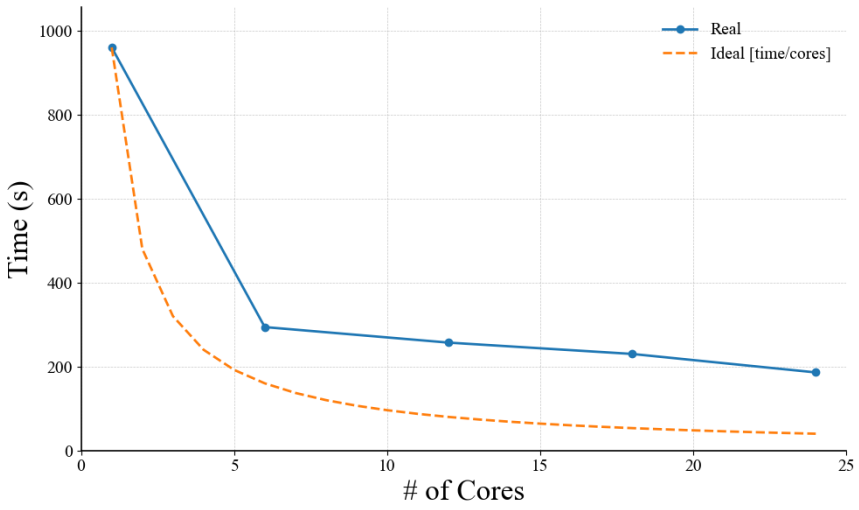


Figure 3.9: **Parallel scaling.** As we run an example of our method with more cores (up to 24 cores), we observe nearly linear scaling. The solid blue line indicates actual time, and theoretical time under perfect linear scaling is drawn with the yellow dashed curve. Our method is not fully parallelized, so we do not expect linear scaling.

Table 3.4: **Number of particles for our complex examples.**

Example	**# of Timesteps**	**Target # of Particles**	**Initial # of Particles**
Sphere to T	840	1.99×10^4	3.76×10^4
Teapot to V	840	2.48×10^4	2.98×10^4
Bunny to C	840	3.14×10^4	1.75×10^4
Bob to G	840	1.25×10^4	1.75×10^4
D to Dragon	950	3.92×10^4	2.46×10^4

strate that our approach can generate smooth morphing animations for complex surfaces and dynamic topology scenarios. In these examples, we test both letter models and a dragon model, which possess curves and topologies that could pose challenges for traditional shape morphing methods. The letter models showcase our method's ability to handle sharp edges and changing topologies, while the dragon model tests its capability to preserve intricate details during the morphing process. For the dragon model, we utilized a larger number of particles to better capture details throughout the morphing process (see Table 3.4).

We rendered these animations using the *VDB from Particles* node in Houdini with a varying voxel size and applied ten iterations of Laplace smoothing to the resulting mesh on each frame for better visual fidelity. Nonetheless, minor artifacts were observed in some of the letter models during the mesh generation process. These artifacts may be attributed to the complex topological transitions present in such models and could be mitigated in future work through enhanced interpolation methods or refined post-processing techniques.

Morphing Under Physical Disturbances. To evaluate the physical robustness of our MPM-based morphing framework, we designed and conducted experiments under various disturbance conditions. These experiments go beyond

Figure 3.10: **Morphing example with letters.** This example demonstrates the capability of our morphing technique to capture topological changes and fine details during a morph. The sequence maintains accurate detail as each form transitions into the next, showing the potential for making practical animations with our method.

static shape matching and assess system stability under external forces and material variability, reflecting more realistic physical environments.

Gravity-induced deformation. A constant vertical gravitational force of $3.5\,\mathrm{m/s^2}$ was applied throughout the morphing process. This force induced elastic sagging in the lower portion of the object, replicating stress experienced

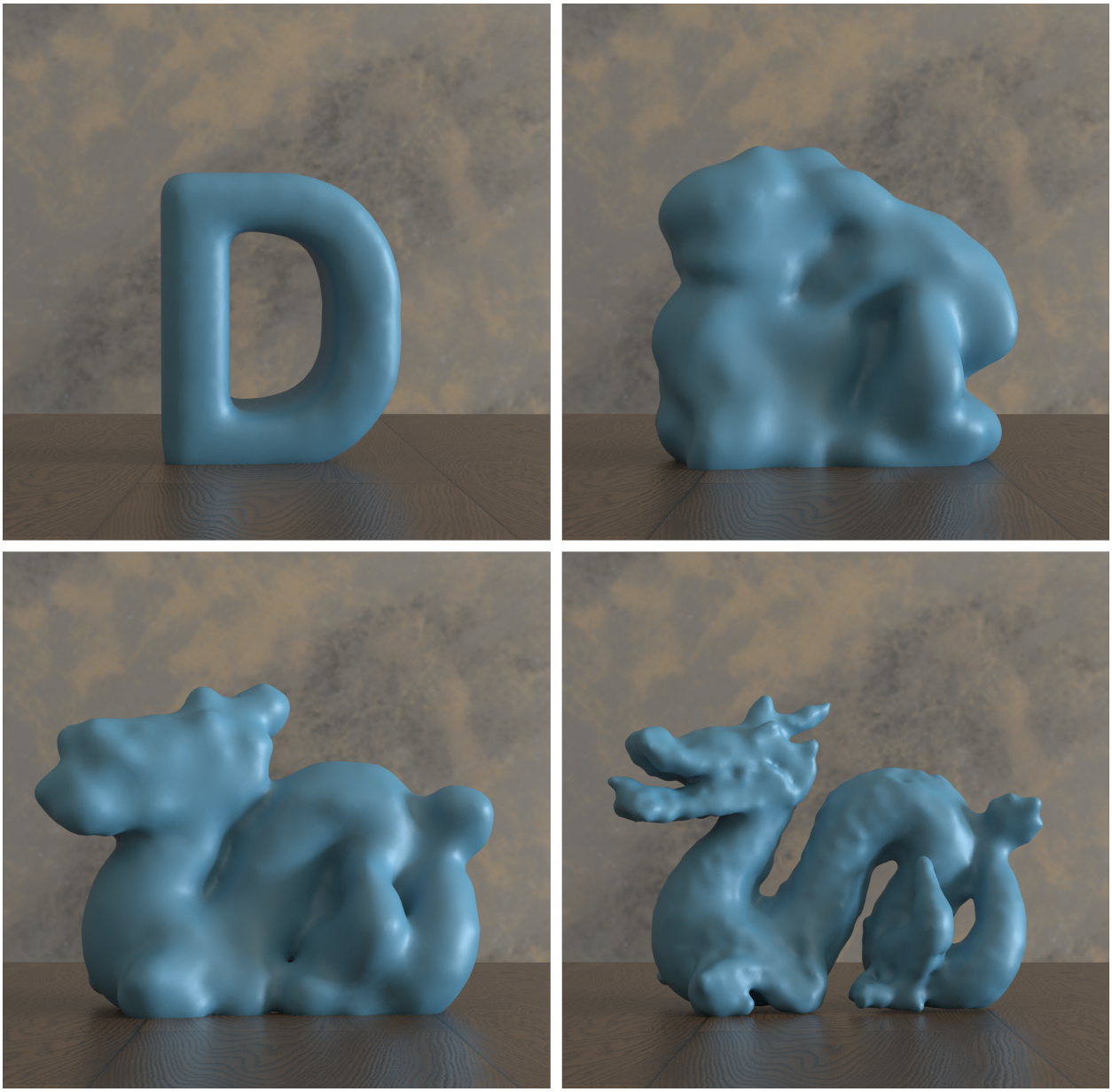


Figure 3.11: **Morphing dragon example.** This example demonstrates the transformation from the letter 'D' into a dragon model. Our method effectively handles the complex topology changes that occur during the morph, preserving both surface details and structural coherence.

by real-world structures under load. The object exhibited realistic deformation while gradually converging toward the target shape, maintaining its elastic properties throughout (see top of Figure 3.12).

Twist-induced stress. One end of the object was fixed while a torque was applied to the other end, resulting in a non-uniform stress field. Strong twisting occurred near the center of rotation during the initial steps, but the system successfully absorbed the stress and progressed toward the target. The final configuration preserved a physically plausible twist while completing the morph (see bottom of Figure 3.12).

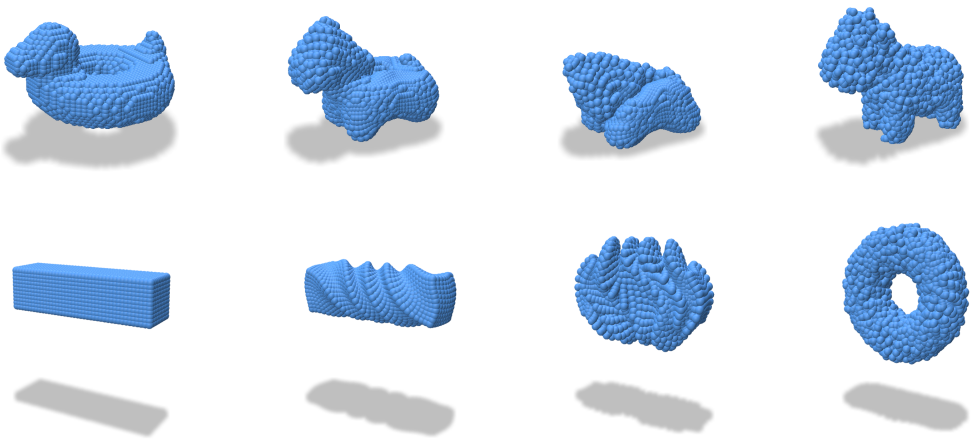


Figure 3.12: **Morphing under physical external forces. Top:** Under a constant vertical gravitational force of $3.5\,\mathrm{m/s^2}$, the object experiences elastic sagging yet steadily converges to the target shape. **Bottom:** When torque is applied to one end while fixing the other, the object undergoes twist deformation, but the method robustly handles the stress and completes the morph.

Material stiffness variation. We varied the Young's modulus E from 2.5×10^4 to 1.0×10^5, with the Poisson's ratio ν fixed at 0.3. In both free morphing and vertical compression scenarios, lower stiffness values produced larger deformations and slower convergence, while higher stiffness led to more rigid responses and faster convergence. Nonetheless, all cases ultimately closely reached the target shape.

These results demonstrate that our MPM-based differentiable simulation can handle physically realistic disturbances, including external forces and material variations, without introducing instability or failing to reach the target shape.

3.5 Chapter Summary

This chapter presented a physics-based shape morphing framework built upon differentiable Moving Least Squares Material Point Method (MLS-MPM). Rather than prescribing particle trajectories or applying manually designed external forces, our approach directly controls internal deformation through optimized per-particle control deformation gradients. By differentiating through the simulation and optimizing these controls, the framework enables target-driven shape transformations while retaining the underlying material dynamics.

To improve optimization stability, we introduced a log-based nodal mass loss function with an additional penalty term, which mitigates mass ejection artifacts without requiring explicit point-to-point correspondences between source and target geometries. We further employed gradient-based optimization with Adam, multiple optimization passes, and chained simulation networks to support complex and long-duration morphing sequences.

Through a variety of morphing experiments, we demonstrated the ability of our framework to handle substantial

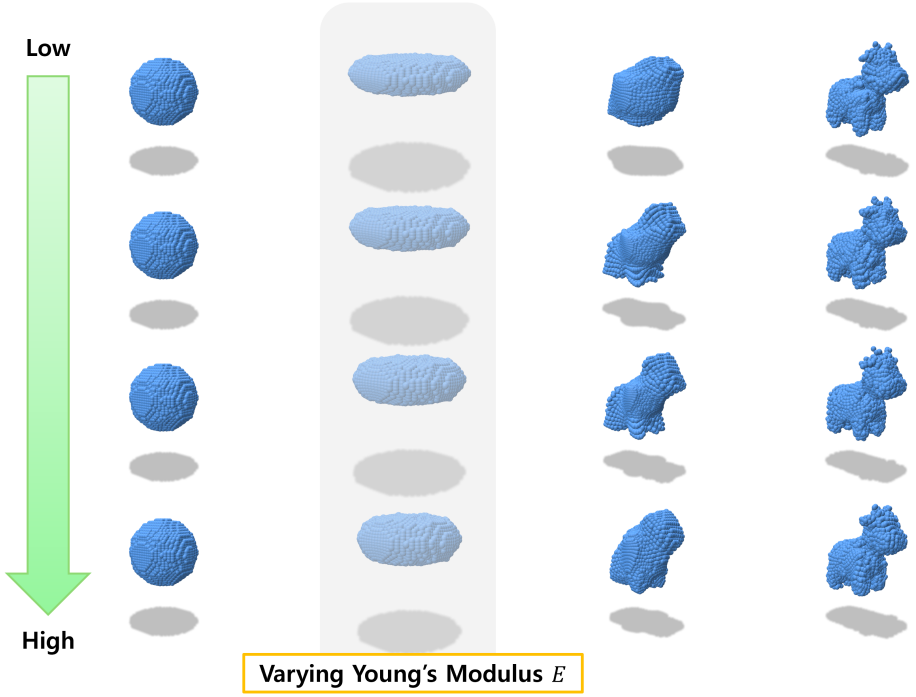


Figure 3.13: **Morphing behavior under vertical compression across varying material stiffness.** The highlighted middle column illustrates the deformation differences observed for varying Young's modulus values ($E = 2.5 \times 10^4$ to 1.0×10^5, with $\nu = 0.3$ fixed). Softer materials undergo more pronounced compression, while stiffer materials retain their shape better and tend to converge more quickly to the target configuration.

geometric deformations, topology changes, and fine-scale shape features. Additional experiments involving gravitational forces, twisting, and variations in material stiffness demonstrated the flexibility of deformation-gradient-based control under different physical conditions. Together, these results establish differentiable MLS-MPM as a practical foundation for directing deformable-object animations through physics-based optimization.

CHAPTER 4

Render-Guided Volumetric Morphing with Differentiable Physics

This chapter extends preliminary work that was presented as a poster at the ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA 2026); a preprint is available as Song and Hyde (2025).

4.1 Overview

Chapter 3 morphed one shape into another by optimizing per-particle control deformation gradients in a differentiable Moving Least Squares Material Point Method (MLS-MPM) simulation. The control enters the material only as an internal stress, so the morph conserves mass and momentum and can change topology. Its objective, however, was a mass loss on the background grid, and its appearance was produced afterwards, when the particles were meshed and smoothed for rendering. Nothing a viewer sees, neither the silhouette of the body nor the shading of its surface, took part in the control.

In this chapter we make the picture part of the control. We optimize the same kind of physical body so that it also *looks* like the target: differentiable rendering terms, evaluated on a surface built from the body's own particles, steer the simulation through its controls, while the controls still enter the material only as a stress and as a rigid-free offset of its surface. Three observations shape the method. A control that is scored while it is still pushing can win a state that springs back once it lets go. A picture asks for changes at the scale of a pixel, below the MPM cell, which a control acting only through the grid averages away over the many particles of a cell. And the pixels of a rendered particle cloud are not those of the surface a viewer sees. Our pipeline answers each of them:

- **Settled windows.** The body is optimized window by window, and each window is scored only after the controls have been released and the body has settled (Section 4.5).
- **Two controls on two scales.** A capped stress increment $\delta\mathbf{F}_c$ moves the bulk through the grid, as in Chapter 3; a rigid-free normal offset u of the outer particle layer shapes the surface below the grid (Section 4.3).
- **A volume-conserving body** whose particles carry the volume their motion makes (Section 4.2).
- **Render terms on the body's exterior**, the surface of the particles' own field, weighted against a transport-based physics objective (Section 4.4).

The chapter answers five questions in turn. *What does $\delta\mathbf{F}_c$ do?* It is the bulk control: a capped stress increment that acts through the grid, cannot change the momentum, and moves material at the scale of a cell, so it carries the transport (Section 4.3.1). *What does u do?* It is the surface control: a rigid-free offset of the outer particle layer along

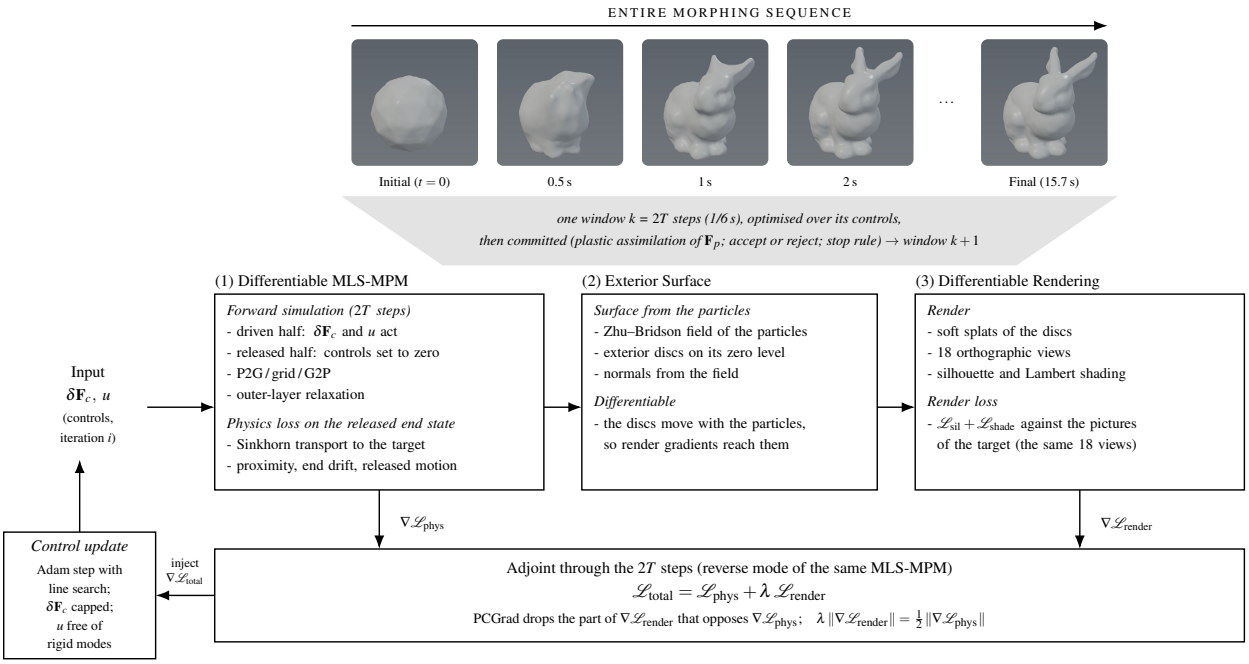


Figure 4.1: **Pipeline overview.** Each window simulates driven and released steps, scores the released end by the physics terms and by the silhouette and shading of the body's exterior from 18 views, and updates both controls through one adjoint (the two cleanup terms of Appendix B.2.1 are omitted).

its normals, below the grid, that shapes relief finer than a cell (Section 4.3.2). *What does the render loss ask for?* What a viewer sees: the body's own exterior, drawn from 18 views and compared with the target pictures in silhouette and shading (Section 4.4.2). *How does it reach the controls?* Through the surface discs to the outer particles, then directly to u below the grid, and to $\delta\mathbf{F}_c$ through the adjoint of the driven steps (Section 4.5.2). *And what does it change?* Under the default rule of its weight, it redirects material while the body arrives and then polishes the rims of thin parts, so the end is closer to the exact target by fractions of a particle spacing. Given its authority after arrival, it writes what only the pictures carry. It carves lettering that exists only in the target pictures into a plaque whose 3D target is smooth, and the lettering is in the material: the plaque prints it into a soft slab. It also moves coloured material to where the pictures paint it, a layout the physics objective cannot see (Section 4.6). That authority costs the body's rest, which Section 4.8 takes up. We measure each part against a $\lambda = 0$ twin, the same run with the render weight set to zero, and then show the method on a wider set of targets and against other methods (Section 4.7). The constants, the per-step updates and the longer tables are collected in Appendix B.

4.2 The Simulated Body

We build on the MLS-MPM simulator of Chapter 3, with cubic B-spline weights, Affine Particle-in-Cell (APIC) transfers (Jiang et al., 2015, 2016) and the fixed corotated elastic response (Stomakhin et al., 2012). The source (a sphere) and the target mesh are each sampled into N particles of equal mass, with the source rescaled to the target's

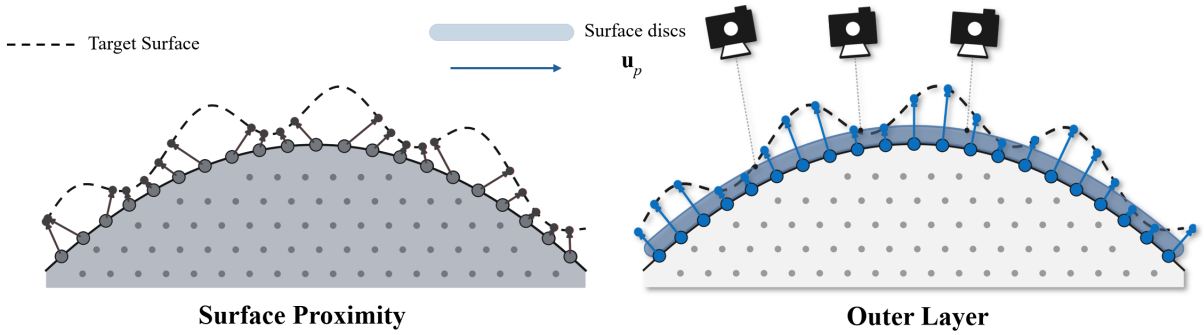


Figure 4.2: **Surface proximity and the outer layer.** Schematic cross-sections. Left: the surface proximity term pulls each surface particle toward the target surface (dashed), only as finely as the particle sample resolves it. Right: the layer offset moves each outer-layer particle along its normal by u_p, below the grid and free of rigid modes; the render terms read the surface discs over this layer from 18 views.

volume, and we call $a = (V/N)^{1/3}$, the edge of the volume per particle, the *sampling pitch*. The MPM cell is set by the shape, not by N, so at 300k particles a cell holds about 183 of them. This is why so much of what a picture asks for lies below the grid. In a driven step the control enters the elastic deformation,

$$\mathbf{F}_p^{\mathrm{E}} = \left(\mathbf{F}_p + \delta\mathbf{F}_{c,p}\right)\left(\mathbf{F}_p^{\mathrm{P}}\right)^{-1}, \qquad \tau_p = \mathbf{P}\left(\mathbf{F}_p^{\mathrm{E}}\right)\mathbf{F}_p^{\mathrm{E}\top}, \tag{4.1}$$

and the Kirchhoff stress τ_p enters the particle-to-grid transfer as in Chapter 3.

Three additions keep the body's volume and its surface in order (Appendix B.1). Each particle carries the volume its motion makes, so that the smoothing of the deformation gradient used in Chapter 3 no longer lets thin parts in transit dilate. At every commit, a plastic assimilation turns half of the elastic stretch, volume included, into rest shape, so the end of a morph is the body's own rest. And three position updates act below the grid: a minimum spacing that pushes too-close particles apart with equal and opposite pushes, a rigid-free relaxation of the outer layer toward the target's relief, and bonds that keep detached fragments with their neighbours.

4.3 Two Controls: Bulk and Surface

As in Chapter 3, we drive the morph through the material's internal deformation rather than through prescribed trajectories or external forces. A rendering term, however, asks for changes at two scales: the bulk of the body has to travel to the target, which the grid can carry, and its surface has to take the target's outline and relief, much of which lies below one MPM cell. We therefore give the optimizer two controls (Figure 4.2).

4.3.1 What $\delta\mathbf{F}_c$ Does: The Bulk

The stress increment is the bulk control: it moves material at the scale of an MPM cell and carries the transport. In each of the T driven steps every particle has a 3×3 increment $\delta\mathbf{F}^n_{c,p}$, the control deformation gradient of Chapter 3, capped per particle and step. It enters only through Eq. (4.1). Because the stress enters the grid as $\tau_p(\mathbf{x}_i-\mathbf{x}_p)$ and the B-spline weights satisfy $\sum_i w_{ip}(\mathbf{x}_i-\mathbf{x}_p)=\mathbf{0}$, the control cannot change the total linear momentum, and since τ_p is symmetric, APIC keeps the angular momentum as well (Jiang et al., 2017). One might instead drive the body with optimized external forces; Section 4.7.6 makes that replacement and measures what changes.

4.3.2 What u Does: The Surface

The layer offset is the surface control: it shapes relief finer than a cell, which a control acting through the grid would average away. Every outer-layer particle has one scalar u_p per window, a displacement along its normal applied in T equal parts over the driven steps:

$$\mathbf{x}_p \leftarrow \mathbf{x}_p + \Big(\tfrac{1}{T}\, g_p\, u_p + s'_p\Big)\mathbf{n}_p, \tag{4.2}$$

where s'_p is the relaxation step of Appendix B.1.1 and $g_p\in\{0,1\}$ a gate. Its adjoint is the identity times the physics' response, so the render gradient reaches it without the grid's low-pass. Three rules keep it a surface control rather than a hand edit. It acts only on particles whose remaining transport is within one MPM cell, so it shapes an arriving surface rather than carrying material. It is projected free of the six rigid modes of the layer after every step of the optimizer, so it moves the body neither along nor about any axis. And it is bounded by one spacing per window. Whether the physics holds what u moves is decided by the released half of the window, in which u does not act.

4.4 Objective

The objective has eight terms in three groups,

$$\mathscr{L} = \underbrace{\sigma\big(\mathscr{L}_{\mathrm{OT}}+\mathscr{L}_{\mathrm{prox}}+\mathscr{L}_{\mathrm{drift}}\big)+\mathscr{L}_{\mathrm{rel}}}_{\mathscr{L}_{\mathrm{phys}}} + w_c\underbrace{\big(\mathscr{L}_{\mathrm{spray}}+\mathscr{L}_{\mathrm{near}}\big)}_{\mathscr{L}_{\mathrm{clean}}} + \lambda\underbrace{\big(\mathscr{L}_{\mathrm{sil}}+\mathscr{L}_{\mathrm{shade}}\big)}_{\mathscr{L}_{\mathrm{render}}}, \tag{4.3}$$

whose formulas are given in Appendix B.2. We kept a term only if switching it off changed a measured quantity on a 19-mesh gallery and at 300k particles. The geometry and settling terms carry no weight of their own: σ is one constant, measured once at the source, that gives the geometry energy the gradient norm of a log-mass grid loss of the kind used in Chapter 3, and so fixes the unit of the cleanup weight w_c. There is no regularizer of the control or of the volume.

4.4.1 What the Physics Terms Ask

The *transport* $\mathscr{L}_{\mathrm{OT}}$ is the debiased Sinkhorn divergence (Feydy et al., 2019) between the body's mass and the target's on a loss grid. The choice matters for the morph. The nodal mass residual of Chapter 3 has its gradient only where the density jumps at the surface, so it cannot see a filled region that is merely in the wrong place; the transport's gradient is a transport field over the whole body, and it is indifferent to topology. Because the transport blurs positions within a loss cell, the *surface proximity* $\mathscr{L}_{\mathrm{prox}}$ asks, at every outer point of the target, whether body material is there. Two settling terms ask for an end at rest: the *end drift* $\mathscr{L}_{\mathrm{drift}}$ charges the velocity left at the released end, and the *released motion* $\mathscr{L}_{\mathrm{rel}}$ charges all motion during the released half; without the latter, runs stop while the body is still moving. The *cleanup* group is optional. Its two local terms, Eqs. (B.7) and (B.8), only tidy what the transport cannot resolve below a loss cell: particles sprayed off the body, and particles in a thin band just off the target surface. We use $w_c = 0.2$ throughout, and $w_c = 0$ removes the group. In the leave-one-out tests that fixed the objective, removing the spray pull left the shape unchanged but left more isolated particles and lengthened the runs, and removing the near band left more thin target points uncovered at 40k particles (Appendix B.2.1).

4.4.2 What the Render Loss Asks For

The render loss asks for what a viewer sees, and it reads the surface of the body, not its particle cloud. The body's *exterior* is the zero set of Zhu and Bridson's field (Zhu and Bridson, 2005) of the particles, covered by surface discs that follow the particles. The *silhouette* term compares the discs' coverage in 18 orthographic views with the target's, penalizing missing coverage inside the target more than excess outside it; the *shading* term compares headlight-lit images of the discs. The target pictures are drawn by the same operator from the exterior of the target sample, averaged over eight independent samples of the mesh so that the reference carries no single sample's noise (Appendix B.2.2). Reading the render terms on the exterior rather than on the particle cloud is what makes them act on the surface a viewer sees: read on the cloud, they improve the cloud's own silhouette and leave the displayed surface where the twin has it (Section 4.7.5). The images in this chapter draw the same exterior with surface discs in a studio look; every image is a committed simulated state.

4.5 Morphing Method

Given the body, its controls and the objective, the morph is an optimization over a sequence of short simulation networks, as in Chapter 3. Here each network is a settled window, and a window is committed only after the body has been released and scored.

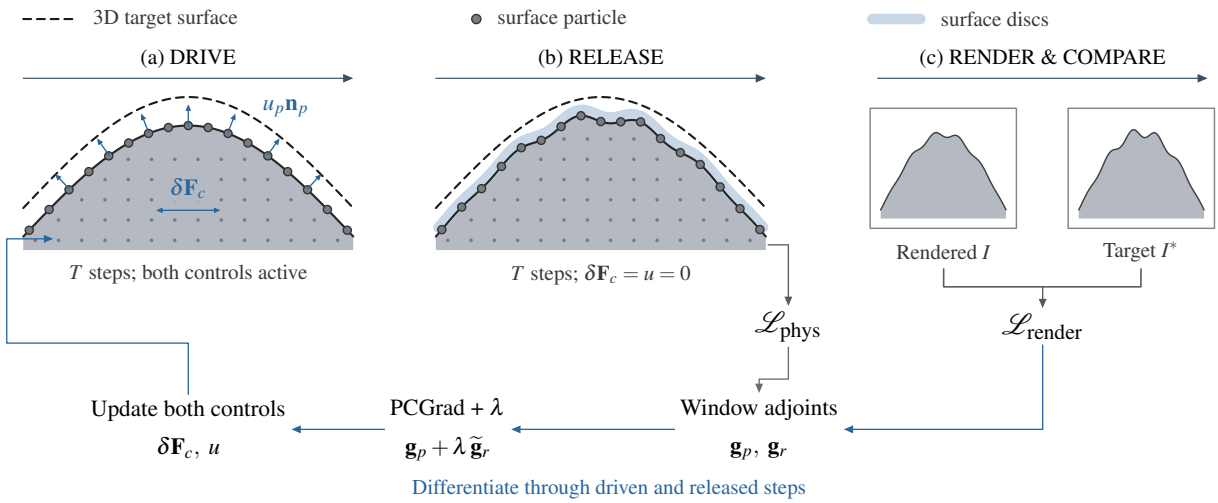


Figure 4.3: **How the render loss reaches the two controls.** (a) Bulk stress $\delta \mathbf{F}_c$ and normal offsets u act for T driven steps. (b) Physics continues for T released steps with both controls zero; surface discs follow the released body. (c) Rendering compares this exterior with target images. Each loss is differentiated through the entire window, and its control-space gradients are combined using PCGrad and λ before updating both controls. The dashed curve is the 3D placement target; image targets may supply additional content. Geometry and image panels are schematic; cleanup gradients are omitted.

4.5.1 Settled Windows

A window simulates T driven steps with the controls, then T released steps with the controls at zero while the physics keeps running. Every term of the objective is evaluated on the released end state and the released velocities. Rebound, ringing and anything else that the control holds only while it pushes therefore cost the control directly. The optimizer runs a fixed number of iterations per window; the window is then committed, and the next one starts from the committed state (Algorithm 3).

4.5.2 How the Render Reaches $\delta \mathbf{F}_c$ and u

The render loss is a function of pictures, and the pictures are drawn from discs that sit on the outer particles. Its gradient therefore lands first on the discs and, through the exterior's field, on the positions of the outer particles at the released end (Figure 4.3). On its way to the controls it is combined with the physics gradient: the part that opposes the physics is removed, and the rest is scaled by the render weight λ (Section 4.5.3). It then reaches the two controls by two paths. The short path goes to u: the offset moves those same particles along their normals, so the gradient arrives below the grid, without the grid's low-pass, and can carve relief finer than a cell. The long path runs backward through the released and the driven steps of the window, through the transfers between particles and grid, to every $\delta \mathbf{F}_c^n$, and reshapes the body at the scale of a cell. The render has no other route to the material: no external force, and no edit of positions outside the two controls. Section 4.6.2 measures how the two paths share the work.

4.5.3 Gradients and the Render Weight

The physics, cleanup and render terms are differentiated separately through the same $2T$-step adjoint back to every $\delta\mathbf{F}_c^n$ and to u. Let $\mathbf{g}_p$ be the gradient of the physics core and $\mathbf{g}_r$ that of the render term. The render gradient loses its component against the physics gradient when the two conflict, as in PCGrad (Yu et al., 2020),

$$\tilde{\mathbf{g}}_r = \mathbf{g}_r - \frac{\min(0, \langle \mathbf{g}_p, \mathbf{g}_r \rangle)}{\|\mathbf{g}_p\|^2}\, \mathbf{g}_p, \tag{4.4}$$

and the render weight is set at each window's first gradient by

$$\lambda\, \|\tilde{\mathbf{g}}_r\| = \tfrac{1}{2}\, \|\mathbf{g}_p\|, \tag{4.5}$$

so that the render term holds one third of the first gradient in every window. The step direction is $\mathbf{g} = \mathbf{g}_p + \lambda \tilde{\mathbf{g}}_r + \mathbf{g}_{\text{clean}}$.

This rule keeps the render's *share* of a step constant, but not its *authority*. While the body arrives, the physics gradient falls by orders of magnitude, and λ falls with it. The render term therefore acts strongly while the body is on its way and only weakly once it has arrived, a consequence we measure in Section 4.6.3 and take up again in Section 4.8.

4.5.4 A Second Mode: The Render Keeps Its Weight

The rule of Eq. (4.5) ties the render's weight to the physics residual. On the 300k bunny, λ falls from 0.32 at the first window to below 0.003 once the body has arrived, and the render's share of the outer layer's pull settles near a fifth. The render then has no authority left exactly when there is something only it could decide: which material ends where, and relief finer than the 3D target resolves. The physics objective cannot see either, because every physics term reads only the set of particle positions.

In a second operating mode, the render keeps its first-window weight: λ is calibrated once by Eq. (4.5) and then held, or only the weight of the term that carries the picture-only content, such as a colour term, is held while the other render terms follow the rule. The purpose of this mode is content that only the pictures carry: relief that exists in the target pictures alone, and the placement of material by its appearance. After arrival the fixed-weight cleanup terms are among the forces that push back against the render, so $w_c = 0$ is a natural companion of this mode, although the runs of Section 4.6.4 keep it. The mode has a cost: a held weight does not vanish once the body has arrived, so the body is still moving when the run stops, and the finest shape metrics relax slightly. A weight that keeps the render's authority where it has something to decide and lets the body come to rest is ongoing work (Section 4.8).

Algorithm 3 Settled-window morph

Require: source and target samples, T driven and T released steps, $i_{\max}$ iterations per window
$\mathbf{s} \leftarrow$ the source body at rest; $\delta\mathbf{F}_c^{\text{prev}} \leftarrow \mathbf{0}$
while the stop rule has not fired **do**
  $\delta\mathbf{F}_c \leftarrow \frac{1}{2}\delta\mathbf{F}_c^{\text{prev}}$ if that beats zero, else $\mathbf{0}$; $u \leftarrow 0$
  for $i = 1, \ldots, i_{\max}$ **do**
    simulate T driven steps ($\delta\mathbf{F}_c$, u act) and T released steps from $\mathbf{s}$
    evaluate $\mathscr{L}_{\text{phys}}$, $\mathscr{L}_{\text{clean}}$, $\mathscr{L}_{\text{render}}$ on the released end
    $\mathbf{g}_p, \mathbf{g}_{\text{clean}}, \mathbf{g}_r \leftarrow$ the adjoint through the $2T$ steps
    $\tilde{\mathbf{g}}_r \leftarrow \text{PCGrad}(\mathbf{g}_r, \mathbf{g}_p)$; at $i = 1$: $\lambda \leftarrow \frac{1}{2}\|\mathbf{g}_p\|/\|\tilde{\mathbf{g}}_r\|$
    Adam step on $\mathbf{g}_p + \lambda\tilde{\mathbf{g}}_r + \mathbf{g}_{\text{clean}}$ with a backtracking line search
    cap $\delta\mathbf{F}_c$; bound u and remove its rigid modes
  end for
  replay the accepted candidate; assimilate half the elastic stretch into $\mathbf{F}^{\text{P}}$
  if the merit rose by more than 5 % **then**
    reject: keep $\mathbf{s}$
  else
    commit: $\mathbf{s} \leftarrow$ the released end state; $\delta\mathbf{F}_c^{\text{prev}} \leftarrow \delta\mathbf{F}_c$
  end if
end while
return the trajectory up to the best window

4.5.5 Step, Commit and Stop

The step is Adam's (Kingma and Ba, 2015) with a backtracking line search, as in Chapter 3; after each trial $\delta\mathbf{F}_c$ is capped and u is bounded and made rigid-free. The moments of u carry from one window to the next, so that its first step in a window does not reverse the last one. A window's result is committed together with the plastic assimilation, rejected if it raises the objective markedly, and the run stops when the objective no longer improves (Appendix B.3).

4.5.6 What the Body Conserves

Mass is exact by construction. The stress control cannot change the linear or angular momentum, the minimum spacing's push is pairwise and central, and the relaxation and u are free of the rigid modes of the layer. The one exception is a weak ambient drag, which is not angular-momentum consistent, so we report rotation as a kinematic quantity. Over whole 300k morphs the centre of mass moves by less than 10^{-4} pitches, and the body's physical volume ends within half a percent of the target's.

4.5.7 Bodies in a Scene

A single world-fixed objective cannot let a body fly, spin or land. For scenes with several bodies, contact and gravity, each body deposits into its own grid field, so that bodies meet with a momentum-conserving contact rule instead of sticking, and each body's objective is read in its own shape-matching frame, so that flight and tumbling cost nothing. Every window is committed, and the targets move along their transport so that bodies meet half-formed. The single-body path is unchanged by these extensions (Appendix B.4).

4.6 What the Render Does

4.6.1 Setup

Every morph of the main evaluation starts from a sphere and targets one of six meshes at $N = 300{,}000$: the bunny, the dragon, the teapot, bob, the armadillo and homer, three of them with handles. A 19-mesh gallery runs at $N = 40{,}000$, and twenty further targets and shape pairs run at 300k. We name the compared arms by what they contain: *the full method* ($\delta\mathbf{F}_c$ + layer u + render loss); *its* $\lambda = 0$ *twin* ($\delta\mathbf{F}_c$ + layer u); $\delta\mathbf{F}_c$ *only*; and *the F-control of Chapter 3*.

No result is scored against the sample it was optimized toward. We read each end against the exact target mesh on views the loss never sees, using standard silhouette, surface and image metrics; at the particle level against an independent sample of the mesh, whose second sample defines the *sampling floor*, a perfect placement; and through the render terms' own operators, which locate where the render acts but, as we show below, also reward the arrangement of the outer layer. Appendix B.6 defines each measure and the rule, fixed before any metric was computed, for when a metric *sees* the render.

4.6.2 Where the Render Acts

Figure 4.4 follows the render's pull on a deforming 300k bunny. It lands on the ears, the face and the feet: early on mostly in agreement with the physics, later more often against it, where the projection removes the opposing part. Both paths of Section 4.5.2 carry it. With the render weight held (Section 4.5.4) and u switched off, so that the render reaches the body only through $\delta\mathbf{F}_c$, a coin keeps about a third of the lettering the render carves into it with u, while the lettering on a bunny's flank is carved as deeply as with u. The surface offset carries most of the lettering on the coin, and the stress increment alone carries it on the bunny (Appendix B.7.9).

4.6.3 Under the Default Rule: Arriving, Then Polishing

Under the default rule the render's weight follows the physics gradient. Figure 4.5 shows the end: against its twin, the full method ends closer to the exact target on every mesh and seed, by a small margin that sits on the rims of thin parts. We follow the render through a run in three phases: the arrival, the time after it, and the end.

During the arrival, the render redirects material. In the first second the full method's surface parts from its twin's by far more than two runs of the same arm with different seeds part from each other; on the dragon the gap reaches almost a pitch. The render carries only a small share of the surface's motion in that second, but that second holds most of all the motion the morph makes, on the ears, the face and the feet of Figure 4.4.

After the arrival, it only polishes. Once the body has arrived, the physics gradient has fallen by more than an order of magnitude, and with it, by the rule of Section 4.5.3, the render weight and the step. The two arms' surfaces return to a difference no larger than the seed-to-seed noise. What the render term still does is hold a sub-pitch offset on silhouette rims and thin parts against the physics' own refit. A causal test shows this (Figure 4.6): switched off at 6 s,

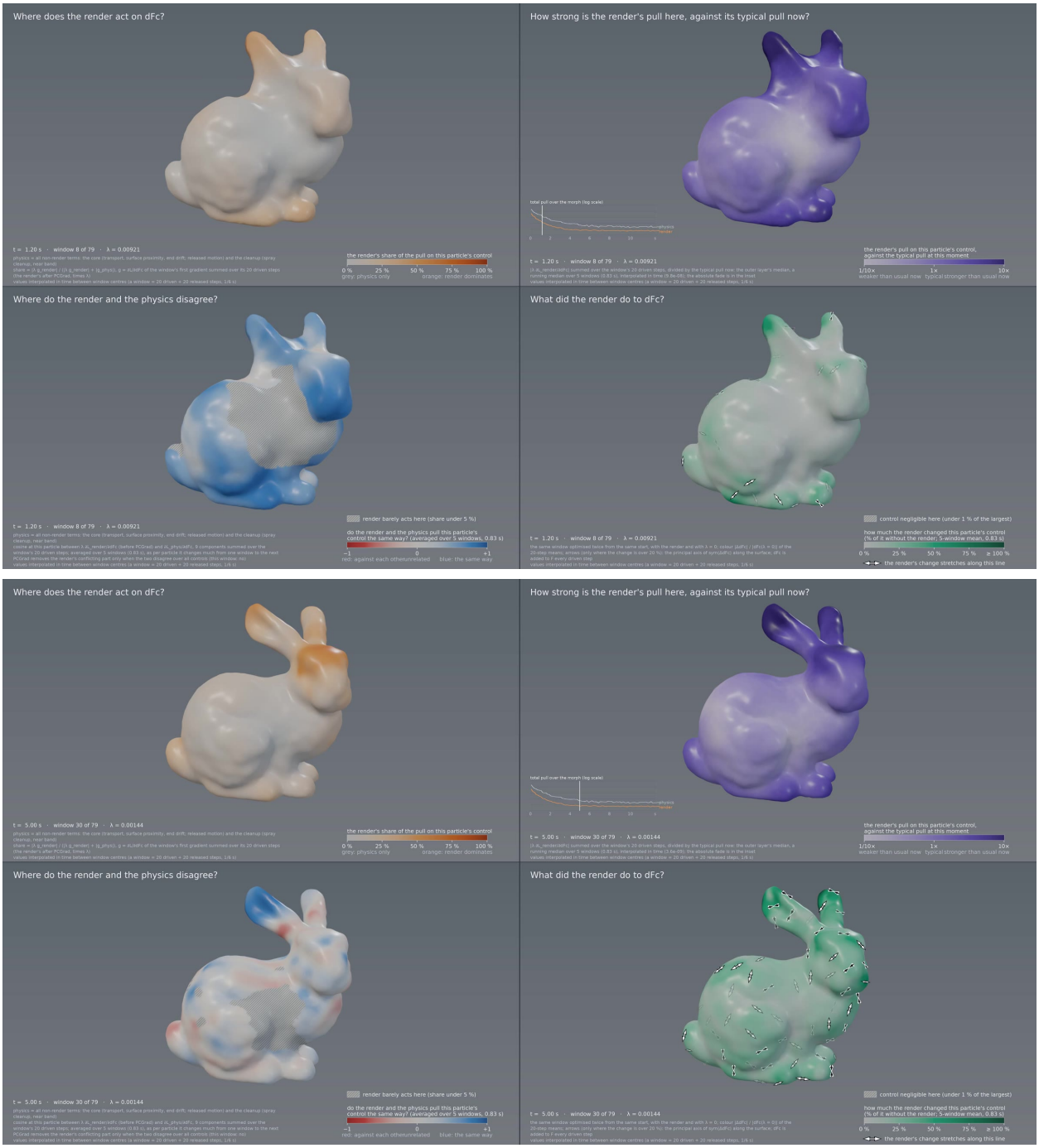


Figure 4.4: **The render gradient on the deforming 300k bunny, early (top) and late (bottom).** Each block shows the render's share of each particle's pull, its strength, its agreement with the physics, and what it changed in $\delta \mathbf{F}_c$.

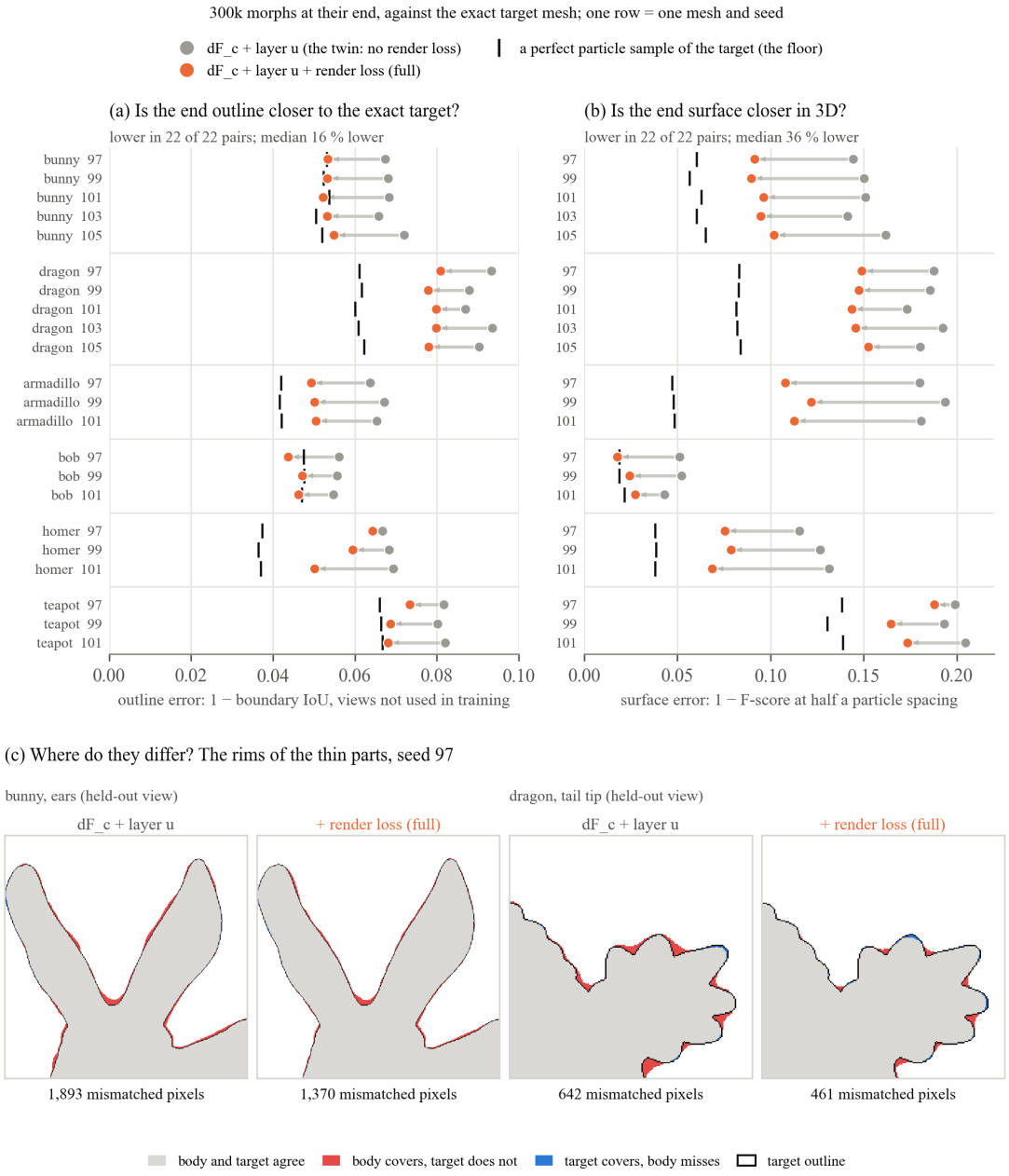


Figure 4.5: **What the render loss changes at the end of a 300k morph.** (a, b) One row per mesh and seed, read against the exact target mesh on views the loss never sees: grey is the $\lambda = 0$ twin ($\delta\mathbf{F}_c$ + layer u), orange the full method, and the black tick a perfect particle sample of the target. The outline error is lower in all 22 pairs, by a median of 16 %, and the surface error at half a pitch in all 22, by a median of 36 %. (c) The difference sits on the rims of thin parts: the end silhouettes of the bunny's ears and the dragon's tail tip (seed 97, held-out views) against the target's outline.

Table 4.1: **The full method against its $\lambda = 0$ twin, read against the exact target mesh.** r is the median share of the twin's error the full method removes (for PSNR, the gain); * marks a metric that sees the render by the rule fixed beforehand.

	300k (6 meshes, 22 pairs)		40k (19 meshes, 57 pairs)	
metric	r	pairs / meshes	r	pairs / meshes
silhouettes, held-out views (1024 px)				
Boundary IoU (2 % of the diagonal)	+16.3 %*	22 / 6	+7.1 %*	52 / 17
boundary F-score (DAVIS, 12 px)	+0.9 %	12 / 3	+12.5 %	31 / 11
contour Chamfer	+20.2 %*	21 / 6	+8.4 %*	54 / 18
contour HD95	+19.6 %*	21 / 6	+8.1 %*	50 / 18
3D, visible exterior				
F-score at 0.5 pitch	+36.4 %*	22 / 6	+8.0 %*	46 / 16
F-score at 1 pitch	+14.4 %*	22 / 6	+3.5 %	39 / 15
normal consistency	+9.2 %*	21 / 6	+2.8 %	42 / 15
Chamfer-L1	+12.9 %*	22 / 6	+4.3 %	46 / 16
images, held-out views (512 px)				
shaded PSNR	+0.73 dB*	22 / 6	+0.30 dB*	53 / 18
shaded SSIM	+10.7 %*	22 / 6	+3.0 %	52 / 18
shaded LPIPS	+7.0 %*	22 / 6	+2.5 %	47 / 17
normal-map PSNR	+0.96 dB*	22 / 6	+0.36 dB*	53 / 18
normal-map SSIM	+9.0 %*	22 / 6	+2.7 %	51 / 17
normal-map LPIPS	+10.0 %*	22 / 6	+3.8 %	51 / 17
for comparison				
particle silhouette IoU (128 px)	+2.3 %	14 / 4	+4.5 %	48 / 19
mask IoU against the raw mesh	+5.2 %	22 / 6	−0.9 %	19 / 5

the full method loses its lead within about a second; switched on at 6 s, the twin builds it within a third of a second and ends level with the run that had the render throughout. Most of what the render asks for lies below the MPM cell, and it is u, not $\delta\mathbf{F}_c$, that reaches it.

At the end, the gains are small but consistent. Table 4.1 reads the end against the exact target meshes, on views the loss never sees. At 300k every standard metric but one improves, in every mesh and in nearly every pair of runs; the one exception is saturated. The improvement is real but small in absolute terms: the Boundary IoU (Cheng et al., 2021) error goes from 0.068 to 0.054, near a perfect sample's 0.052 (Figure 4.5). The gain is the same on views far from every training view, so it is not specific to the pictures the loss sees; Table B.2 gives it per mesh. At 40k the same metrics move in the same direction, by about half as much on the boundary metrics and less elsewhere. The particle-level measures do not see these gains: the particle silhouette, chamfer distance, thin coverage, centre of mass and rest stay within the twin's range (Figure B.1), because the binary particle silhouette resolves almost two pitches per pixel while the render changes the exterior by fractions of a pitch (Figure B.2).

Part of the render's lead on its own term is arrangement, not shape. At the end of the 300k runs the full method's render term lies below that of every uniform sample of the true mesh, which no body of the exact shape, sampled uniformly, can do. What goes below that floor is an outer layer arranged more evenly, and hence an exterior smoother, than any random sample; on the bunny at least two fifths of the full method's lead over its twin on the render term is of this kind, and on the dragon at least a fifth. This is why we read shape only on the exact mesh and use the render

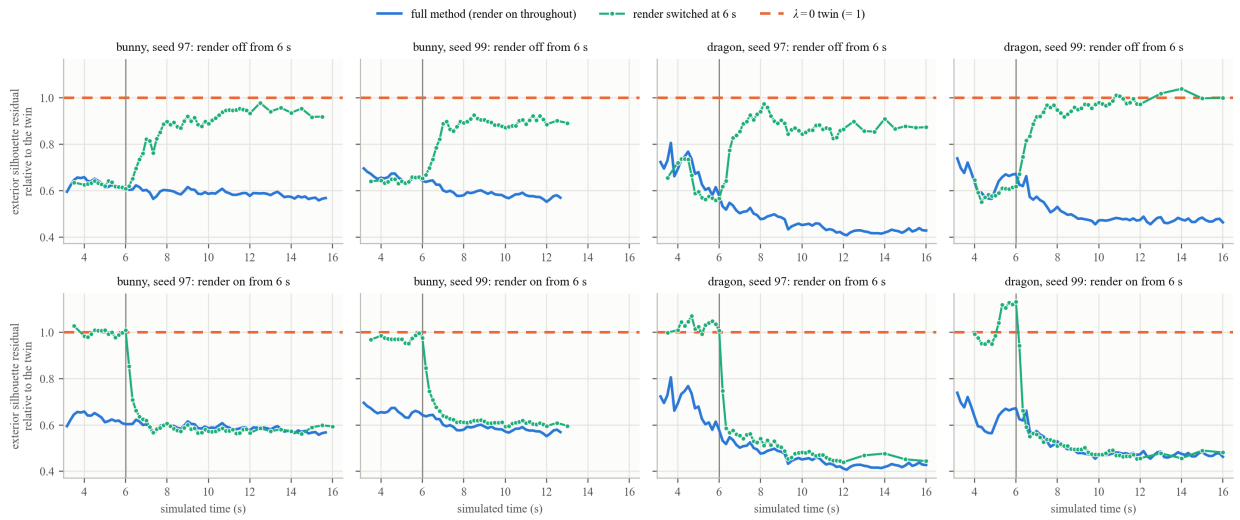


Figure 4.6: **The render term switched at 6 s.** Switched off, the full method loses its lead within about a second; switched on, the twin builds it within a third of a second.

terms' own operators, as in Figures B.1 and 4.6, only to locate the render's action.

In short, under the rule, the render term reshapes the arrival and polishes the end; after arrival it decides nothing that the physics objective leaves open, which is what holding its weight changes (Section 4.6.4). It also has a small cost: it deforms some bodies more, and the ambient drag, which does not respect angular momentum, acts more on a body that moves more. Appendix B.7 gives the measurements behind this section.

4.6.4 What the Render Can Write

Under the default rule of Section 4.5.3 the render only polishes rims once the body has arrived, because its weight follows the physics gradient. That is a property of the weight, not of what the render can reach. With its weight held after arrival (Section 4.5.4), the render does what the physics objective cannot: it writes content that only the pictures carry.

Relief that exists only in the pictures. The 3D target is a smooth plaque; the word SIGGRAPH on its front and VANDERBILT on its back exist only in the render's target pictures (Figure 4.7). The twin, which has the layer offset u but no render loss, ends smooth. With the render weight held, the full method carves both words into the particles, to 58 % of the relief that a 300k sample of the lettered mesh itself carries. This also answers whether the visible relief could come from u alone: u acts in both arms, and only the arm that sees the pictures writes the words. Under the rule, the render's weight has gone by the time the plaque has formed, and the words stay at the twin's level on a 40k plaque. Reading the physics placement terms only at the scale of the loss cell, which leaves the finer scale to the render, lets the held render carve more of the lettering on 40k bodies, up to 0.83 of the ceiling. The cost is the one named in Section 4.5.4: the body is still moving when the run stops, and the plaque's ends are slightly under-filled.

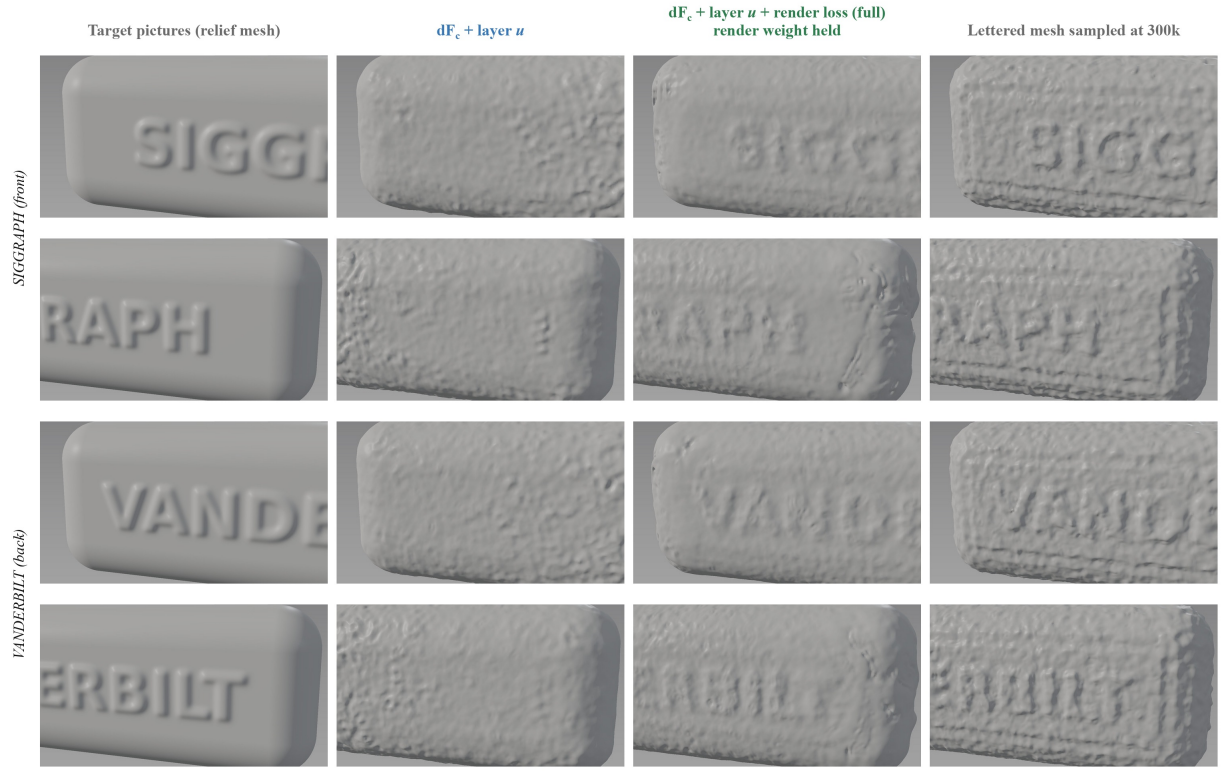


Figure 4.7: **Lettering that exists only in the target pictures.** A 300k plaque whose 3D target is smooth: with the render weight held, the full method carves SIGGRAPH (front) and VANDERBILT (back) into the particles, while its twin, which has the layer offset u, ends smooth.

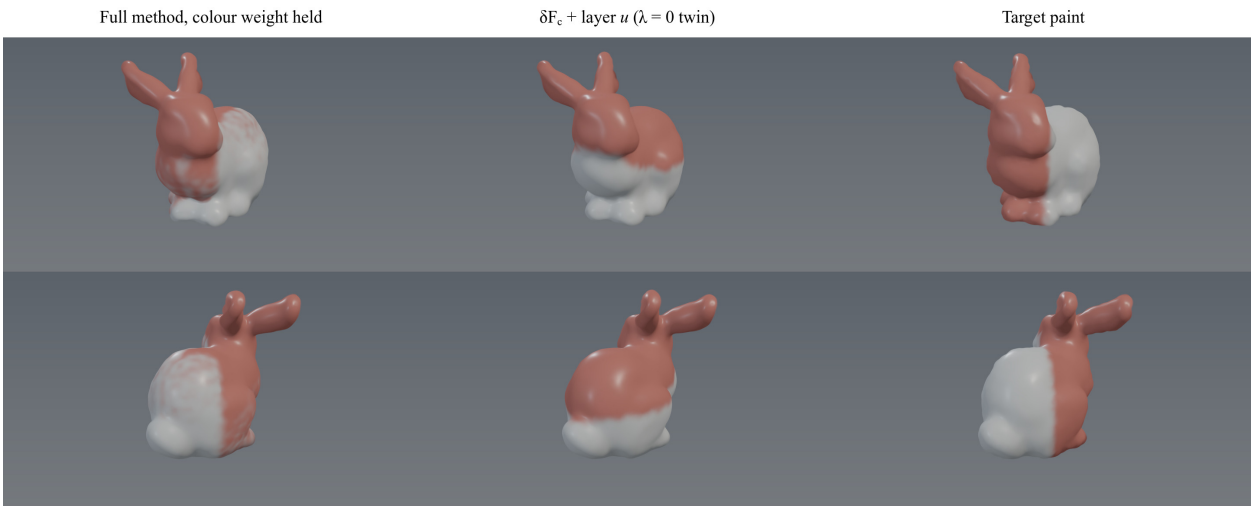


Figure 4.8: **Placing material by its appearance.** The physics objective cannot see which particle carries which colour; with the colour term's weight held, the full method moves the red material to the head half that the target pictures paint, while its twin keeps the transport's top-and-bottom split (bunny, 40k particles, front and back).

Placement by appearance. Each particle carries a colour that is fixed at the source and moves with it, and the target pictures are painted. The physics objective is provably blind to the paint: the twin's trajectory is bit for bit the same under any paint. In the scene of Figure 4.8, the source is red on its upper half and the pictures paint the head half of the bunny red. The twin keeps the transport's top-and-bottom split, and under the rule the full method leaves the layout almost where the twin leaves it. With the colour term's weight held, the full method moves material so that the colours land where the pictures put them, and recovers about 70 % of the layout's error. It too ends while the colour is still moving, and its finest surface metric relaxes slightly. Appendix B.7.9 gives the measurements behind this section.

Where the render acts when it keeps its weight. Figure 4.9 repeats the share map of Figure 4.4 on the 300k bunny, under the rule and with the render weight held. Under the rule, the render's share of the outer layer's pull settles at 0.14–0.23 after arrival; held, it is 0.78–0.90, and the render, not the physics, then decides what the outer layer does for the rest of the run. That is how it writes the content of Section 4.6.4, and it is also its price: the body's kinetic energy at the end is three orders of magnitude above the rule's, and a few more thin target points are left uncovered (Appendix B.7.9).

4.6.5 The Letters Are in the Material

A carving that lived only in the displayed surface would be a decal. To test where the lettering is, we press a finished plaque, letters down, into a soft slab and lift it again, with the controls off (Figure 4.10). The full method's plaque prints SIGGRAPH into the slab. The twin, given the same lettering afterwards as a decal on its displayed surface, prints nothing: in a still picture the decal can even look crisper, but it cannot press. At 300k the print correlates with the letters at 0.46, and at 500k it reads under raking light. This is what separates the render loss from a physics-only

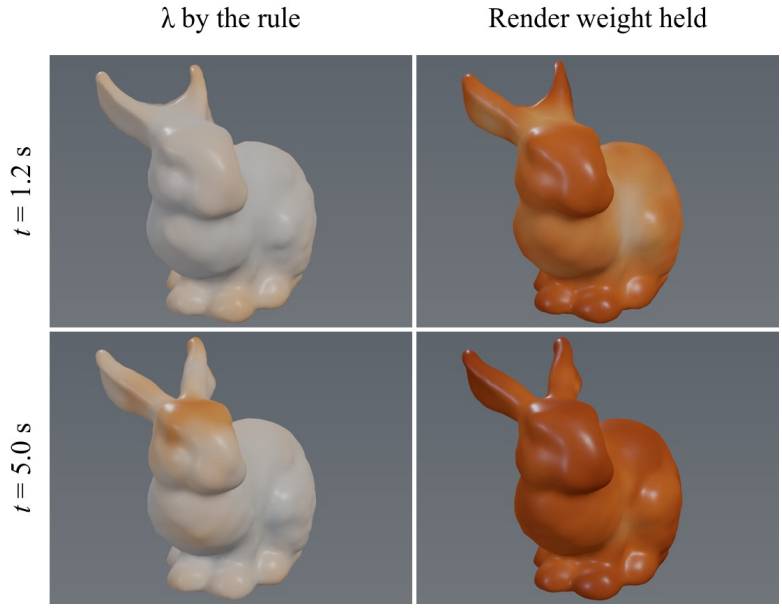


Figure 4.9: **Where the render acts when it keeps its weight.** The render's share of each particle's pull on the 300k bunny, from grey (the physics alone) to orange (the render alone): under the rule it fades after arrival, held it dominates the outer layer.

morph followed by a surface fit to the pictures: the fit changes the picture, while the render loss changes the body. Appendix B.7.9 describes the press.

4.7 Examples and Comparisons

4.7.1 From a Sphere to Five Targets

Figure 4.11 shows five of the targets. The body rises as one piece, and the features grow out of it within the first second or two: the bunny's ears and the armadillo's limbs are in place by 1.5 s, bob's ring opens a little later, and the dragon's horns and claws and the narrow gaps between its coils come last. Most of the particle movement happens in these first seconds; what follows is a long, slow tail that fills most of the delivered frames. The ends are close to what a perfect placement of the particles would give: the particle silhouette lies just below the sampling floor and the chamfer distance below it, because the ends are more even than any random sample. The bodies end at rest, and no run sheds a serious piece. Section 4.6.3 reads these ends against their twins.

4.7.2 The 40k Gallery

On the 19-mesh gallery at $N = 40{,}000$ the render acts in the same direction as at 300k, at about half the size: the particle silhouette is ahead of the twin's on all but one mesh by a small margin, and against the exact meshes the outline improves on all 19. One reading goes the other way: the silhouette area against the raw mesh is slightly worse, because the exterior leans slightly outward. The lean comes from the silhouette term's asymmetric weighting

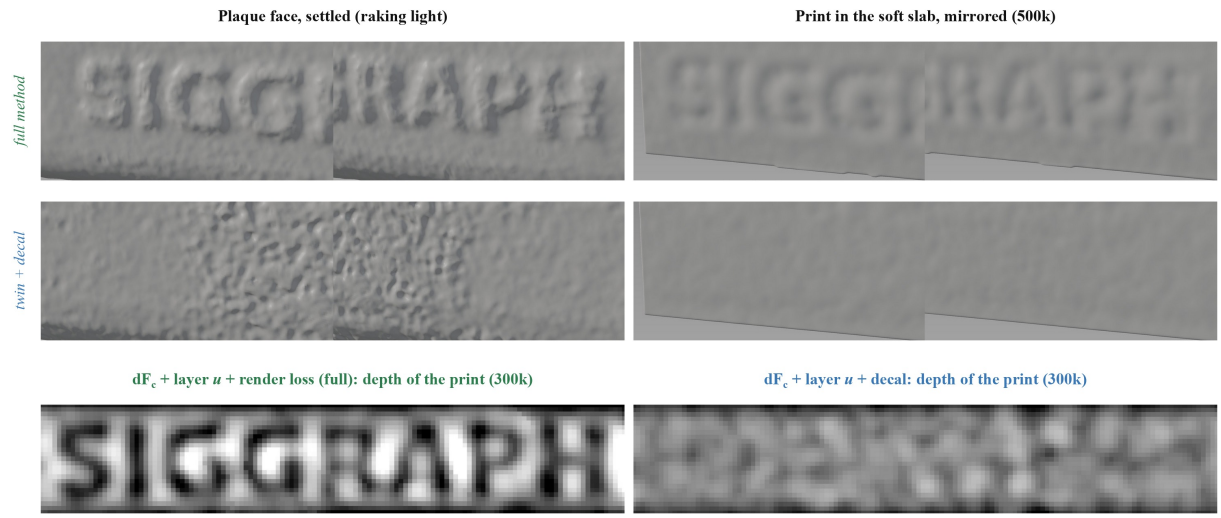


Figure 4.10: **The lettering is in the material.** Plaques lettered as in Figure 4.7, pressed letters down into a soft slab and lifted. Top: at 500k under raking light, the full method's plaque face and the print it leaves, mirrored to read, against its twin with the lettering added afterwards as a decal. Bottom: the depth of the print at 300k, darker where deeper.

of missing against excess coverage, which at 40k, unlike at 300k, is not balanced by an inward pull; Appendix B.7.2 gives the measurement and the next test.

4.7.3 A Gallery of 300k Morphs

To see the method beyond the six main targets, we ran it unchanged from a sphere to the 13 other gallery meshes and to the Happy Buddha, a target of genus six, and between six pairs of shapes; Figure 4.12 shows all of them but the three letters, A, V and C. Every run but one reaches its target nearly as closely as the main six do, comes to rest, and sheds no serious piece. Topology changes as the targets ask. From the teapot to bob, the teapot's handle closes within the first second and bob's ring opens later, so the body ends with the target's single hole; from a sphere, the Happy Buddha's openings form. Where twins were run, the render's lead has the same direction and size as on the main six.

Three examples show where the method still pays. On the letter C the run stops early while the body still flows, because the stop rule reads the objective's progress, not rest. Between two shapes in different orientations, such as spot and the cow, the transport carries the head over the back; yawing the source first makes the same morph direct and twice as fast. And a less compact source such as the armadillo runs on a cell that is coarse relative to the body, and passes through thin, perforated sheets on its way to homer.

4.7.4 Materials and External Forces

Every material we tried reaches its target: a soft solid arrives more slowly, a stiff one faster but less at rest, and the render term's lead holds in each. Under gravity, the single-body window stops early, because a falling body raises the

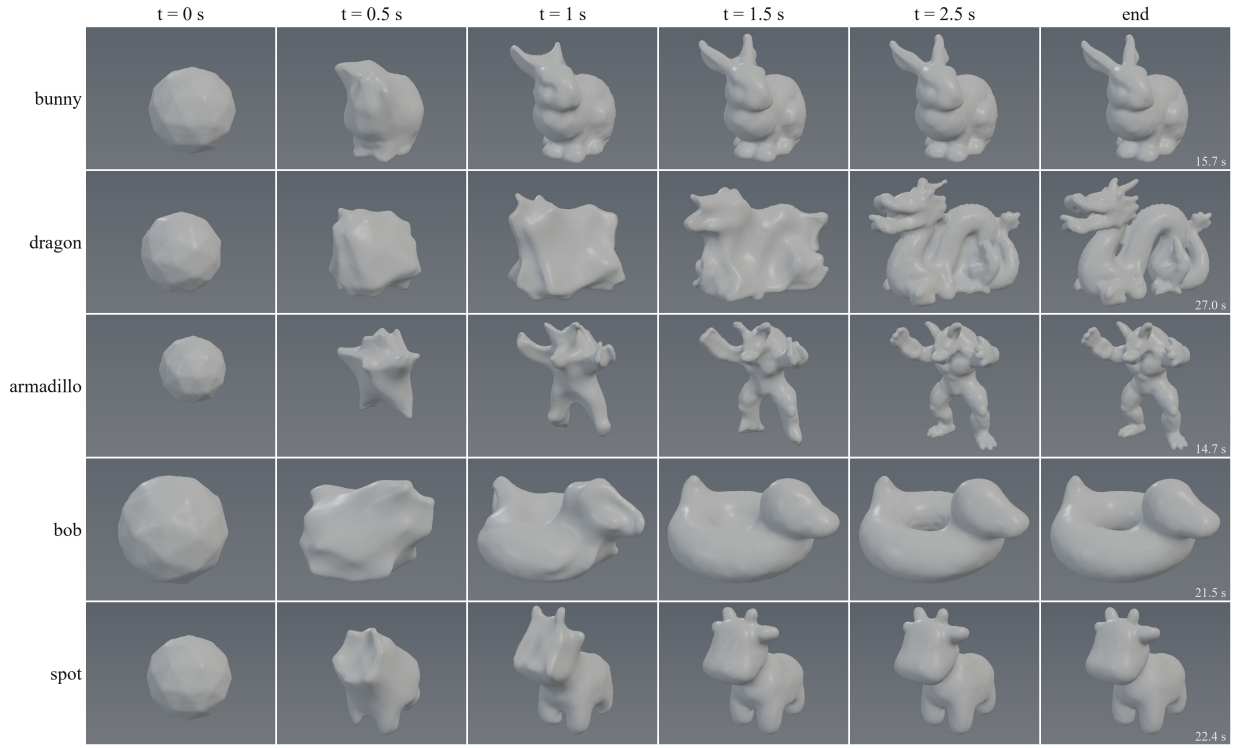


Figure 4.11: **From a sphere to five targets at 300k particles.** Every row is shown at the same times and at its end. The body rises as one piece and the features grow out of it in the first second or two; bob's ring opens a little later and the dragon's narrowest gaps open last. A separate piece of the display surface that holds fewer than ten particles is not drawn (Appendix B.2.2).

world-fixed objective. Through the scene path, with the target standing on the floor, the bunny and the dragon reach their targets on every seed, stand upright at the target's height and keep their horizontal momentum. They do not come to rest: each commit's assimilation makes part of the weight's sag rest shape and the controls raise it again. The next definition assimilates only the controls' share of each window's deformation (Appendix B.7.4).

4.7.5 Ablations

Switching off one part at a time at 300k (Table B.3) gives three readings. First, u is the main shape lever: without it every surface measure is several times worse and the body never comes to rest, although the early floating particles almost vanish, because u is also their source. Second, reading the render on the exterior beats reading it on the particle cloud on every measure of the surface, while the cloud read wins on the particle measures it optimizes; each read wins on the representation it scores, and the exterior is the one a viewer sees. Third, the volume-conserving body, the minimum spacing and the choices made against late motion each earn their place on what they address.

Figure 4.12: **Twenty-four 300k morphs, each shown at its start, halfway and end.** The gallery of Section 4.7.3 without the three letters. The last panel uses an MPM cell set from the source's volume, an option outside the method, for comparison with the panel before it.

4.7.6 Stress Control against Force Control

Physics-based shape control drives matter toward a target with external forces (Treuille et al., 2003; McNamara et al., 2004; Fattal and Lischinski, 2004; Chen et al., 2024; Li et al., 2025). To test our choice, we replaced $\delta\mathbf{F}_c$ by an optimized external force per particle and step and kept everything else. A plain force puts momentum in at every window, which the objective then has to pull back. Projecting the force onto zero net force and torque removes that momentum exactly, but not the shape deficit: both force arms end with several times the stress control's geometry energy and a worse surface, and they do not end more at rest. The stress control gives exact momentum without a projection, and a better end than a momentum-free force reaches with the same objective.

4.7.7 Comparison with Other Methods

We compare with nine methods of five kinds, each run from the same sphere toward the same six targets and read by the same probes: the F-control of Chapter 3 (Xu et al., 2025); the neural implicit morphs of Buonomo et al. (2025), NISE (Novello et al., 2023), ISD (Sang et al., 2025a) and 4Deform (Sang et al., 2025b); as-rigid-as-possible interpolation (Alexa et al., 2000); optimal-transport displacement interpolation (McCann, 1997; Bonneel et al., 2011); and render-only mesh optimization with Large Steps (Nicolet et al., 2021) and FlexiCubes (Shen et al., 2023). Table B.4 and Appendix B.7.7 give the full reading.

No method's silhouette lies above ours on any mesh, and the clearest separation is in the thin parts: ours leaves about as many thin target points uncovered as a second sample of the mesh does, while every other method leaves at least half again as many. What separates the methods most, however, is the way there. ISD fits the end but loses the body on the way; Large Steps keeps one genus-zero surface and can never open a handle; FlexiCubes changes topology without control; transport alone and the F-control of Chapter 3 keep the mass but tear thin parts into clouds. Ours stays one body with exact mass and a near-constant volume throughout and reaches every target, including those of higher genus. Read on their own end meshes, the best mesh methods have sharper surfaces than our particle route, but a perfect particle sample of the target trails them by a similar amount: that part of the gap is the cost of a particle body at this resolution. The render-only mesh methods, ARAP, the F-control and NISE are faster than ours; ISD, the volume-preserving morph and 4Deform are slower.

4.7.8 Runtime

A 300k morph takes ten minutes to half an hour on one GPU (Table B.1). The run length is set by the stop rule, and most of it is the slow tail after arrival. That tail is not wasted: the end measures keep improving through its first half and the body settles, and only its last tenth changes nothing measurable.

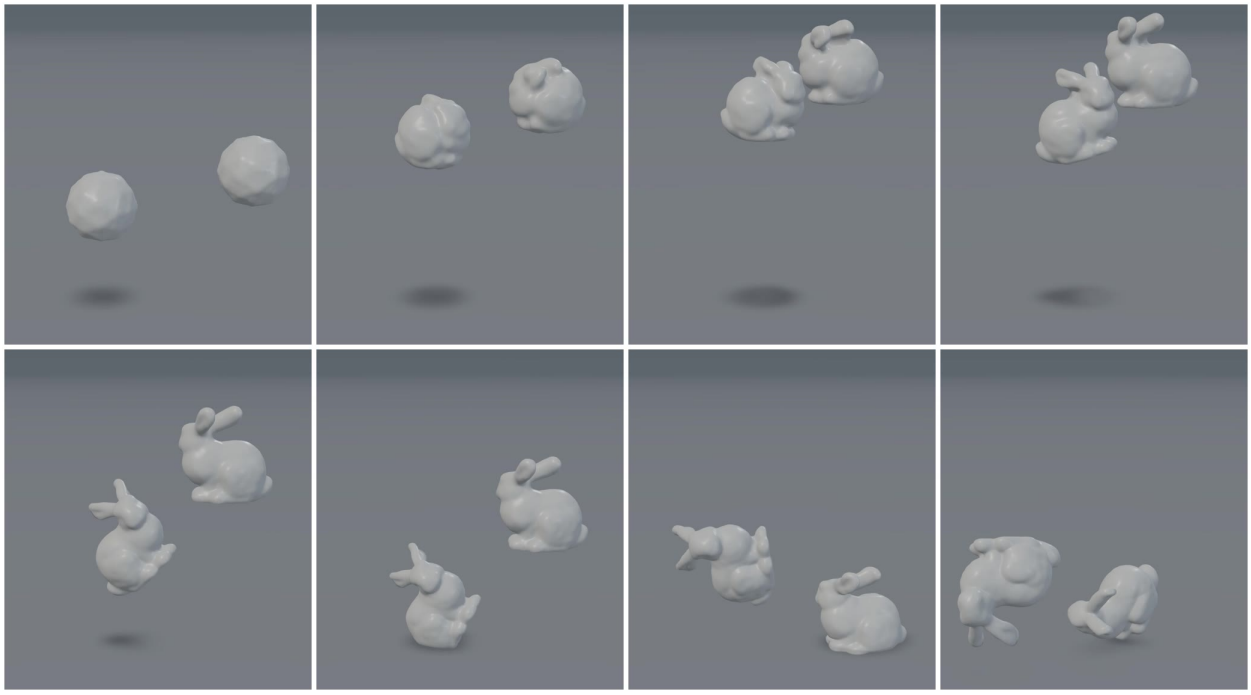


Figure 4.13: **Two thrown bodies, each becoming a bunny.** Each body has its own grid field and its own objective in its own frame, so they part cleanly, fly, land and tumble.

4.7.9 Thrown and Dropped Bodies

Two spheres are thrown toward each other, collide in the air while half-formed, part, and each becomes a bunny that lands and tumbles (Figure 4.13). The scene uses the multi-body definitions of Appendix B.4 and a stiffer material, with the plastic assimilation off so that the controls hold each body's shape against the impacts. In the delivered run the bodies meet when the morph is a little under half done, separate cleanly, land, spring off the floor and tumble, and each ends close to its target in its own frame, with momentum kept through every phase. A single body can fall as well: a sphere released high above the floor becomes a bunny on the way down, lands, bounces, and tips forward onto its head, which the body-frame objective allows.

4.8 Measured Costs and Next Steps

Rest under the render's authority. With its weight held after arrival, the render writes what the physics objective cannot see, but the body is still moving when the run stops, because a held weight does not vanish once the body has arrived. When such a run's controls are released, the body comes to rest and the colour layout stays where the render put it, but part of the carved relief is lost; at 40k about a third of it remains. Two parts of the method can take it back, and they are being separated: the elastic part of the stress the controls built up, and the relaxation of the outer layer toward the 3D target, which keeps acting after the release and pulls the layer toward a target that is smooth. Taking the written end as the rest shape keeps the lettering, as in the 500k press of Figure 4.10; whether a run reaches that rest on

its own is being measured. The other definition under test is a render weight that follows the render term's own excess over a measured floor, so that it fades once the picture-only content is written.

Narrow gaps. The cubic kernel averages the grid velocity over about two cells, so a gap of the target narrower than that cannot open until late and tears into beads when it does. An MPM cell that follows N opens the teapot's handle and sharpens the boundaries, but leaves most of the dragon's loops closed, fills the early surface with holes and more than doubles the dragon's cost. A finer cell is therefore an option of the pipeline, to be chosen after the same comparison on more meshes.

A target given only by pictures. When the 3D target is the visual hull of the training silhouettes and only the render terms see the true mesh, the full method sharpens the held-out outline but does not carve the concavities that only the pictures hold. The information is in the pictures, but only a smooth change of the whole surface lowers the loss; a multiview shape-from-shading descent finds the concave regions in place, and the next step carves them from a hull that contains the object at the render's resolution.

Further measured costs, the transient floaters of the early arrival, the ambient drag's effect on angular momentum and the long, slow tail of the delivered frames, are described in Appendix B.7.10.

4.9 Chapter Summary

This chapter presented a render-guided morphing pipeline built on the differentiable MLS-MPM of Chapter 3. The body is a volume-conserving solid, optimized in settled windows that are scored after the controls are released. The stress increment $\delta\mathbf{F}_c$ is its bulk control: it acts through the grid, keeps the momentum and carries the transport. The layer offset u is its surface control: it moves the outer particles along their normals, below the grid and free of rigid modes. The render loss asks for what a viewer sees, the silhouette and shading of the body's own exterior from 18 views, and it reaches u directly and $\delta\mathbf{F}_c$ through the adjoint of the driven steps; both paths carry it.

Under the default rule of its weight, the render redirects material while the body arrives, then only polishes rims because its weight follows the vanishing physics gradient, and it leaves the end consistently but slightly closer to the exact target, with part of its lead on its own term coming from a smoother arrangement rather than a different shape. With its weight held after arrival, it carves the words SIGGRAPH and VANDERBILT, which exist only in the target pictures, into a plaque whose 3D target is smooth, and the words are in the material: the plaque prints them into a soft slab, where a decal on its twin prints nothing. In the same way, it moves red material to the half of a bunny that the pictures paint red, a layout the physics objective cannot see. Both come at the cost of the body's rest, and a render weight that keeps the authority and the rest is the next step.

The morph keeps exact mass and, up to the ambient drag, momentum; it ends at rest, opens handles a sphere does not have, runs unchanged on a wide range of targets and shape pairs, and takes part in contact and gravity with other bodies. Against nine other methods, no method's silhouette lies above ours on any mesh, and only ours keeps thin

parts at the sampling floor.

CHAPTER 5

CulinaryCut: Physics-Grounded Cutting for Vision–Language–Action Models

This chapter is adapted from Koh et al. (2026), published in the proceedings of the European Conference on Computer Vision (ECCV 2026), with co-authors Hyunseo Koh, Youngjae Choi, Misa Viveiros, David Hyde, and Heewon Kim.

5.1 Overview

Food cutting is a challenging deformable-object manipulation task: positioning the knife correctly does not guarantee separation. Successful execution must localize the target, overcome material-dependent contact resistance, and account for geometry and topology changes during cutting. A kinematically valid path can therefore fail when deformable-material dynamics are included.

This chapter presents *CulinaryCut*, a physics-grounded benchmark generated with *IntSim*, which couples a Moving Least Squares Material Point Method (MLS-MPM) material solver (Hu et al., 2018) to ManiSkill (Gu et al., 2023). ManiSkill supplies robot execution and images; MLS-MPM computes deformation, tool contact, reaction forces, and cutting outcomes. Together, they evaluate physical separation rather than geometric blade motion alone.

Rather than introducing a new force-conditioned policy architecture, our objective is to study whether *physics-grounded training data* improve existing vision–language–action (VLA) models. The benchmark separates two limitations:

- **Geometry gap:** difficulty grounding language-specified ratios, directions, object identities, and sequential cuts in the actual geometry of a deformable object.
- **Physics gap:** difficulty producing sufficient material-dependent interaction to sever an object, even when the predicted knife path satisfies geometric criteria.

These gaps are related but distinct: a policy may select the correct cutting location without inducing a successful cut, or may produce a physically strong action at the wrong location.

CulinaryCut contains **21,000 physics-grounded cutting trajectories** spanning **15 food items**. Fifteen visual backgrounds and at least five instruction variants per trajectory expand the collection to approximately **1.575 million observation–instruction instances**. Contact-force profiles and material dynamics are retained in the generated data and used for simulation analysis and trajectory generation; *force, contact, and damage states are not supplied as policy inputs*. We evaluate the existing RDT (Liu et al., 2025), Octo (Octo Model Team et al., 2024), and OpenVLA (Kim et al., 2025) policies without changing their architectures. This distinction is important: the contribution is a physics-grounded dataset and evaluation protocol, not a closed-loop force-feedback controller or an additional style-transfer

network.

5.1.1 Research Motivation and Contributions

The main contributions are:

1. **Physics-grounded simulation and benchmarking:** coupling robot execution with MLS-MPM deformation, damage, contact forces, and separation.

2. **Geometry- and physics-gap analyses:** measuring ratio, direction, object, and sequential-cut generalization separately from material severing.

3. **Scalable data generation:** constructing 21,000 trajectories for 15 foods and approximately 1.575 million visually and linguistically augmented instances.

4. **Empirical validation:** comparing real and simulated contact forces and evaluating physics-grounded policy training in simulation and on a real robot.

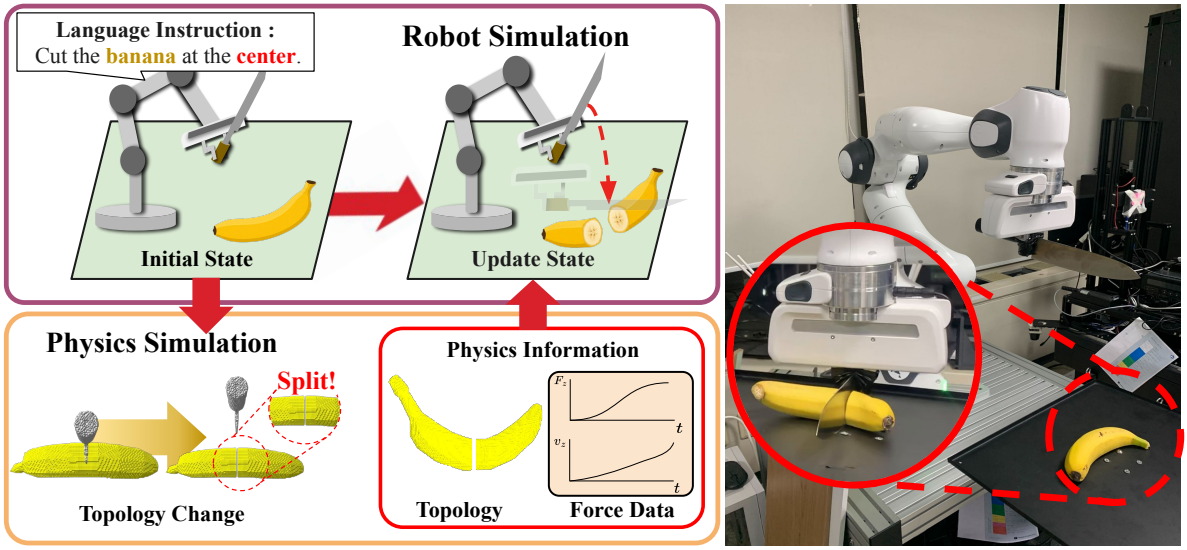


Figure 5.1: **Overview of physics-grounded robotic food cutting.** IntSim combines robot execution with material deformation and topology changes to generate CulinaryCut demonstrations. The pipeline supports evaluation of contact-rich cutting in simulation and transfer to a real robot.

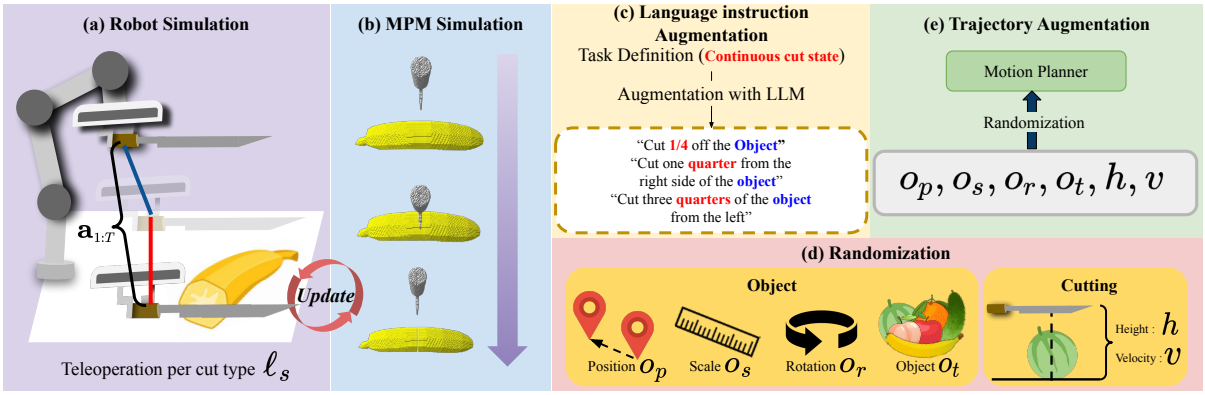


Figure 5.2: **CulinaryCut data generation.** Initial demonstrations and task specifications are converted to cutting motions; IntSim couples ManiSkill robot execution with MLS-MPM material response; language and trajectory augmentation provide diverse geometric configurations and cutting instructions. The benchmark preserves physical contact and topology information while retaining standard VLA action interfaces.

5.2 Background and Related Work

5.2.1 Vision–Language–Action Policies and Geometric Supervision

VLA models combine visual perception, language understanding, and motion generation in a single learned policy. Representative systems include RT-2 (Brohan et al., 2023), OpenVLA (Kim et al., 2025), RDT (Liu et al., 2025), Octo (Octo Model Team et al., 2024), and later generalist-policy architectures such as π_0 (Black et al., 2025). Their shared objective is to connect a natural-language task description to executable robot actions from visual and, when available, robot-state observations. Although the underlying policy architectures and action encodings vary substantially, most large-scale demonstrations are more naturally suited to measuring object placement, grasping, or reachability than the onset of material separation.

Prior work also explores force-conditioned policy inputs, including ForceVLA (Yu et al., 2025). CulinaryCut addresses a complementary problem. Rather than asking how much a new force sensor or force-conditioned architecture improves execution, it asks whether existing models acquire more physically effective behavior when the *demonstrations themselves* have been constructed and selected under deformable-material dynamics. The distinction matters when interpreting the results: improved cutting outcomes do not establish that a VLA has learned explicit online force regulation.

5.2.2 Manipulation Datasets and Cutting Simulators

General robot datasets such as DROID (Khazatsky et al., 2024), Open X-Embodiment (Open X-Embodiment Collaboration et al., 2024), BridgeData V2 (Walke et al., 2023), and LIBERO (Liu et al., 2023a) provide broad task and scene coverage (Open X-Embodiment also spans 22 embodiments). They are important for training visuomotor representations and language-conditioned behaviors. However, they do not jointly emphasize deformable-material fracture,

continuous robot actions, and physical force records in food cutting. Collecting large numbers of real cutting demonstrations is also constrained by safety, tool wear, material variability, and the practical need to repeatedly prepare and replace food objects.

Several simulators and benchmarks have directly addressed cutting and deformable manipulation. DiSECt (Heiden et al., 2021) and SliceIt! (Beltran-Hernandez et al., 2024) employ finite-element approaches to cutting, while TopoCut (Wang et al., 2025) explores MPM-based topology changes. RoboCook (Shi et al., 2023) learns tool–dough interaction with graph-network dynamics trained on real point clouds. Each provides a useful contribution to modeling deformation or robotic interaction, but the comparison motivating CulinaryCut is the lack of a single VLA-oriented benchmark combining robot simulation, continuous actions, language instructions, multi-view images, and contact-aware material evolution. Table 5.1 compares the principal capabilities of these datasets and simulators.

Table 5.1: **Capabilities emphasized by representative cutting and manipulation datasets.** The comparison concerns benchmark capabilities rather than the numerical accuracy of each simulator. "–" denotes that the corresponding capability was not identified as part of the referenced benchmark.

Dataset or system	Physics sim.	Robot sim.	Data origin	Multi-cam.	Language	Cont. action	Cutting
DROID (Khazatsky et al., 2024)	–	–	Real	Yes	Yes	Yes	–
ARNOLD (Gong et al., 2023)	–	Isaac Sim	Synthetic	Yes	Yes	Yes	–
RoboChop (Dikshit et al., 2023)	–	–	Real	–	–	–	Yes
RoboCook (Shi et al., 2023)	–	–	Hybrid	Yes	–	–	Yes
Jamdagni and Jia (2021)	FEM	–	Hybrid	–	–	–	Yes
SliceIt! (Beltran-Hernandez et al., 2024)	FEM	Gazebo	Synthetic	–	–	Yes	Yes
DiSECt (Heiden et al., 2021)	FEM	–	Synthetic	–	–	–	Yes
TopoCut (Wang et al., 2025)	MPM	–	Synthetic	–	–	–	Yes
CulinaryCut	**MPM**	**ManiSkill**	**Synthetic**	**Yes**	**Yes**	**Yes**	**Yes**

5.2.3 Material Simulation and the Visual/Physical Domain Gap

The Material Point Method represents a continuum with moving particles whose momentum equations are solved on a temporary background grid (Sulsky et al., 1994; Stomakhin et al., 2013). MLS-MPM (Hu et al., 2018) and APIC (Jiang et al., 2015, 2017) improve particle–grid transfer, while continuum damage models permit progressive weakening (Wolper et al., 2019; Wang et al., 2019). These are established ingredients of IntSim rather than newly proposed numerical methods.

The *visual domain gap* concerns appearance, lighting, and background differences between simulation and reality; domain randomization and world models can mitigate these differences (Tobin et al., 2017; NVIDIA et al., 2025). The separate *physics gap* concerns material resistance, force, and actual separation. Photorealistic images cannot validate

the latter, motivating a simulator that explicitly evaluates deformable contact.

5.3 IntSim: Physics-Grounded Cutting Simulation

5.3.1 Robot–Material Simulation Coupling

IntSim couples MLS-MPM to a ManiSkill Franka Emika Panda robot. ManiSkill handles poses, commanded motions, and visual observations; MLS-MPM advances food particles under constitutive forces, gravity, and contact with the knife and board. Momentum exchange yields contact-force estimates, while updated geometry exposes whether the motion physically severs the food.

The paired robotic and material representations are shown in Figure 5.1, while the complete data-generation process is summarized in Figure 5.2.

The MPM solver uses a Y-up coordinate system, with gravity

$$\mathbf{g} = (0, -9.81, 0)\ \mathrm{m\,s^{-2}}. \tag{5.1}$$

Robot observations and exported end-effector trajectories are transformed to ManiSkill's Z-up convention by exchanging the vertical axes. Unless stated otherwise, task instructions and reported robot trajectories use the Z-up convention.

5.3.2 MLS-MPM Formulation

The material is discretized into Lagrangian particles coupled through an Eulerian background grid. Each particle p stores its position $\mathbf{x}_p$, velocity $\mathbf{v}_p$, mass m_p, volume V_p, deformation gradient $\mathbf{F}_p$, affine velocity matrix $\mathbf{C}_p$, accumulated plasticity α_p, and scalar damage $D_p \in [0,1]$. The affine term improves the fidelity of angular and shear motion during particle–grid transfers (Jiang et al., 2017), while the damage field provides a mechanism for softening material near the cutting interface. Particle–grid–particle transfers use a quadratic B-spline kernel over a $3 \times 3 \times 3$ node stencil.

5.3.2.1 Conservation Laws and Constitutive Response

At the continuum level, the material obeys the mass and linear momentum balances and the kinematic evolution of its deformation gradient:

$$\dot{\rho} + \rho\, \nabla \cdot \mathbf{v} = 0, \tag{5.2a}$$

$$\rho\, \dot{\mathbf{v}} = \nabla \cdot \sigma + \rho\, \mathbf{g}, \tag{5.2b}$$

$$\dot{\mathbf{F}} = (\nabla \mathbf{v})\, \mathbf{F}. \tag{5.2c}$$

Here ρ is density, $\mathbf{v}$ is material velocity, σ is the Cauchy stress, and $\mathbf{g}$ is gravitational acceleration.

We employ fixed corotated elasticity (Stomakhin et al., 2012) as the elastic backbone, with optional J_2 plasticity treated through a radial-return update. Writing the polar decomposition as $\mathbf{F} = \mathbf{RS}$, the first Piola–Kirchhoff and Cauchy stresses are given by

$$\mathbf{P} = 2\mu(\mathbf{F} - \mathbf{R}) + \lambda(J-1)J\mathbf{F}^{-\mathsf{T}}, \tag{5.3a}$$

$$\sigma = J^{-1}\mathbf{P}\mathbf{F}^{\mathsf{T}}, \qquad J = \det(\mathbf{F}). \tag{5.3b}$$

The Lamé parameters are determined by Young's modulus E and Poisson's ratio ν:

$$\mu = \frac{E}{2(1+\nu)}, \qquad \lambda = \frac{E\nu}{(1+\nu)(1-2\nu)}. \tag{5.4}$$

The blade-localized damage variable reduces the effective elastic response by scaling the Lamé parameters as damage accumulates. This provides progressive material weakening near the knife, while the particle and surface reconstruction steps expose the resulting opening and separation. These choices are a tractable approximation to food cutting rather than a complete microscopic fracture or fiber-tearing model.

5.3.2.2 Particle-to-Grid Transfer

At the beginning of each MLS-MPM time step, the current material configuration is stored on Lagrangian particles. The background grid is then populated by transferring each particle's mass and momentum to its neighboring nodes through quadratic B-spline weights. In three dimensions, each particle influences a $3 \times 3 \times 3$ stencil. Reconstructing grid quantities in this fashion allows the solver to evaluate continuum dynamics without maintaining a mesh whose elements deform permanently with the food.

The affine velocity matrix $\mathbf{C}_p$ carries a local first-order description of particle motion. Including it in the particle-to-grid transfer helps preserve rotational and shear behavior that would otherwise be lost through repeated interpolation. Internal forces are generated from the particle deformation state through the fixed corotated constitutive law, and the material response is modified when yielding or local damage is active. In cutting, this stage links the evolving internal state of the food to the forces that influence the subsequent grid velocity update.

5.3.2.3 Grid Evolution and Tool Contact

After the particle-to-grid transfer, grid velocities are formed from the accumulated nodal momentum and mass. External acceleration, including gravity, is applied at the grid level. Boundary handling and mild numerical stabilization further constrain the velocity field before the updated grid state is transferred back to the particles.

The knife and cutting board are represented by signed distance fields (SDFs). For a position in the active region, the sign and magnitude of the SDF determine whether the corresponding location lies inside or outside a tool and its

distance from the surface. Contact queries are restricted to each tool's axis-aligned bounding box to avoid evaluating blade or board geometry throughout the entire computational domain. At candidate contact nodes, the relative tool–material velocity is decomposed into normal and tangential components. Non-penetration, restitution, and Coulomb-friction responses are then applied to determine the post-contact motion.

The velocity impulse generated by contact is accumulated and divided by the corresponding measurement interval Δt_{acc} to estimate mean contact force:

$$\mathbf{F}_{\text{avg}} = \frac{1}{\Delta t_{\text{acc}}} \sum_{i \in \mathscr{C}} m_i \big(\mathbf{v}_i^{\text{after}} - \mathbf{v}_i^{\text{before}}\big), \tag{5.5}$$

where $\mathscr{C}$ denotes the material degrees of freedom contributing to the contact impulse over the accumulation interval. The equal-and-opposite reaction acts on the tool; reported force magnitudes refer to the contact interaction rather than the static robot-arm sensor load. This impulse-based measurement is recorded alongside generated trajectories but is not a VLA observation at inference time.

The implementation uses a normalized contact-strength indicator $\hat{c} \in [0,1]$ to gate blade-localized damage and to modulate a material-dependent knife-resistance update. The update depends on the approach speed and material properties, so geometrically similar cutting trajectories may produce different responses across materials. The contact normalization and resistance-update equation are provided in Appendix C, Eq. (C.1).

5.3.2.4 Grid-to-Particle Transfer and Deformation Update

Updated grid velocities are interpolated back to particles to compute particle velocities and affine velocity matrices. A small blade-tip separation term is applied in a narrow edge band to mitigate numerical sticking and promote crack opening. The deformation gradient is then advanced by

$$\mathbf{F}_p^{n+1} = \big(\mathbf{I} + \Delta t\, \mathbf{C}_p^{n+1}\big)\mathbf{F}_p^n. \tag{5.6}$$

The implementation additionally applies a volume-normalization safeguard to deformation gradients whose Jacobian leaves a specified safe interval, using the rescaling $\mathbf{F} \leftarrow J^{-1/3}\mathbf{F}$. This operation restores unit determinant; it should not be interpreted as a constitutive plasticity update.

5.3.3 Damage-Gated Separation and Numerical Stability

Damage is updated only for particles in a narrow blade-SDF band $|\phi| < b$, and only when the following conditions hold simultaneously: (i) contact strength exceeds a threshold $\hat{c} \geq \hat{c}_{\min}$, (ii) the material approaches the blade sufficiently quickly, $v_n \leq -v_{\text{th}}$, and (iii) the tool is executing a downward cutting stroke. As damage accumulates, local stiffness decreases; particle-connectivity or color-based surface updates stabilize the visual representation of separated regions.

To account for changes in numerical resolution, the blade-SDF band and approach-speed threshold are adjusted using grid-spacing and time-step scales. The solver uses a CFL-limited time step together with speed caps, damping, bounded deformation determinants, and local SDF-query culling. The precise scaling rules and time-step bound are given in Appendix C (Eqs. (C.2) and (C.3)). The GPU implementation uses Taichi (Hu et al., 2019a) kernels and atomic reductions for contact impulses. In addition to the generated image/action data, diagnostic records include force–time curves, deformation-Jacobian statistics, contact-strength traces, impulse checks, and simulation parameters.

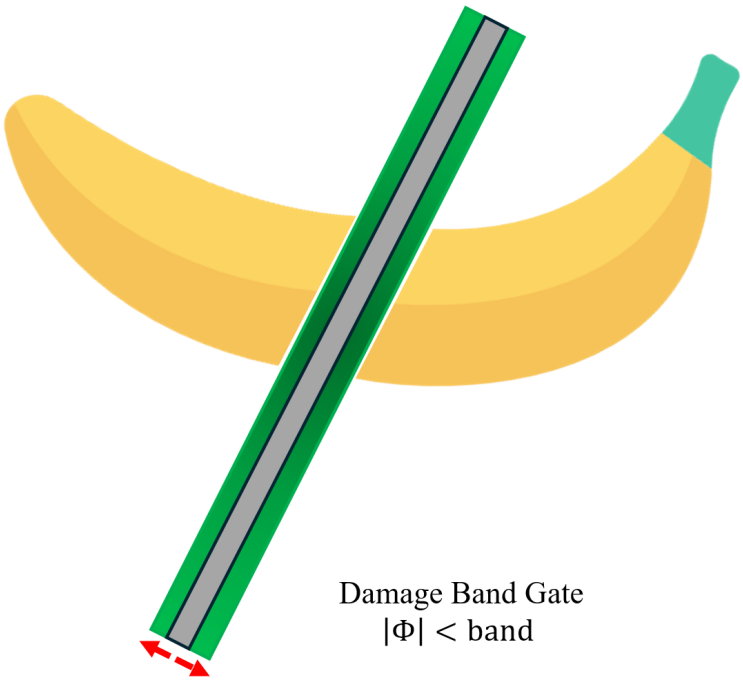


Figure 5.3: **Blade-localized damage gate.** Damage is accumulated in a signed-distance band surrounding the cutting tool when contact-strength, approach-speed, and stroke-direction criteria are satisfied.

5.3.4 Implementation Diagnostics and Resolution Treatment

The implementation also records contact-strength traces, blade/board impulses, and relevant metadata such as Δx, Δt, material properties, and end-effector states. When the grid resolution or time increment changes, the SDF-band width, speed threshold, and contact normalization scale are adjusted according to the rules stated above. These choices are intended to maintain interpretable cutting behavior across numerical settings, but they should not be confused with a formal proof that all cutting outcomes are fully resolution independent.

5.3.5 Material Parameters and Simulation Settings

Where published food-engineering measurements report a material parameter, the simulation takes it from them (Kheiralipour et al., 2008; Mattos et al., 2005; Nourain et al., 2005; Jahanbakhshi et al., 2020; Kurniawati et al.,

2023; Kabas and Vladut, 2015; An et al., 2025); the remaining parameters are set for the simulation, and Table 5.3 marks which values come from each source. The material solver operates over a $0.5 \times 0.5 \times 0.5$ m domain, spanning $[-0.25, 0.25] \times [0, 0.5] \times [-0.25, 0.25]$ m in its Y-up frame. A 120^3 Cartesian grid yields $\Delta x \approx 4.17 \times 10^{-3}$ m, and food particles are initialized at approximately 2.5×10^{-3} m spacing. The reported base integration time step is $\Delta t = 2 \times 10^{-5}$ s and is reduced automatically when required by the stability criterion. The robot simulation provides the Franka arm, its knife, table, and visual observations.

The robot environment renders multi-view RGB observations and records end-effector position and velocity alongside simulated contact-force logs. Its representative camera is located at eye $= (0.5, 0.3, 0.5)$ m, looking toward $(0.0, 0.1, 0.0)$ m with a 35° field of view. The observation stream is reported at 60 fps, with six substeps per output record; the output cadence is decoupled from the CFL-controlled material integration step. For VLA fine-tuning, the particular baseline may use a restricted view and a model-specific control frequency, as described below.

Table 5.2 summarizes the main simulation and observation parameters. The fact that the food particles are spaced more densely than the MPM grid is not inconsistent: particles encode material samples and histories, while the computational grid carries temporary masses, momentum, and stress forces. Both scales must be specified to interpret the spatial detail visible in the particle galleries and the numerical resolution of the solver.

Table 5.2: **Representative IntSim numerical and observational settings** , used for the representative IntSim configuration. Spatial and temporal quantities refer to different parts of the coupled pipeline.

Quantity	Reported setting
Material grid	120^3 nodes
Domain side lengths	$0.5 \times 0.5 \times 0.5$ m
Grid spacing Δx	4.17×10^{-3} m (approx.)
Particle spacing Δx_p	2.5×10^{-3} m
Base physics step Δt	2×10^{-5} s (CFL-adaptive)
Output rate	60 fps (reported observation stream)
Reported substeps per output	6
Representative camera FOV	35°
Board–food friction	$\mu = 0.5$

CulinaryCut uses a single robot arm for cutting, without a second hand to stabilize the food. Passive board–object friction therefore matters: the friction ablation shows that removing board friction (changing $\mu = 0.5$ to $\mu = 0$) causes approximately 0.034 m of center-of-mass drift under an otherwise matched cutting motion. This sensitivity motivates explicit contact and boundary modeling rather than treating the food as a fixed geometric target.

Table 5.3: **Representative material parameters in CulinaryCut.** The benchmark contains 15 food categories; only the seven representative items for which numerical properties are reported in the paper are shown. The values are those used in the simulation. Unmarked values are reported by the source in the last column; ‡ marks a value chosen within the range that source reports, and † a value set for the simulation because the source reports no value or a different one.

Food item	Density ρ (kg/m^3)	Young's modulus E (MPa)	Poisson's ratio ν	Source
Apple	850†	3.0‡	0.33†	Kheiralipour et al. (2008)
Cucumber	980	1.0†	0.35†	Mattos et al. (2005)
Melon	1100‡	1.2‡	0.45†	Nourain et al. (2005)
Banana	960	0.007†	0.35	Jahanbakhshi et al. (2020)
Orange	1020	0.584†	0.24†	Kurniawati et al. (2023)
Peach	968	0.89	0.29	Kabas and Vladut (2015)
Strawberry (berry)	1010	0.20‡	0.40	An et al. (2025)

5.3.6 Execution-Time Velocity Guardrail

To avoid overly aggressive commands during policy evaluation, an external velocity guardrail limits the magnitude of predicted end-effector motion. The guardrail does *not* alter the VLA architecture, supply force feedback to the policy, or replan the target cutting location. Instead, a lightweight regression $\mathscr{R}$ is fit on simulator-generated data spanning knife speeds and material parameters $\mathbf{m} = (E, \sigma_y)$. The predicted peak interaction force determines an allowable motion speed:

$$\hat{F} = \mathscr{R}(v, \mathbf{m}), \qquad v_{\text{safe}} = \max\{v \mid \mathscr{R}(v, \mathbf{m}) \le F_{\max}\}. \tag{5.7}$$

Commands exceeding v_{safe} are clipped at execution time. The same filter is applied to all evaluated policies, separating this deployment safeguard from any learned force-aware behavior. Its measured effects are reported in Section 5.5.5.

5.4 CulinaryCut Benchmark and Training Protocol

5.4.1 Task Design and Geometric Grounding

The benchmark combines target-object identity, cutting location, directional reference, and cutting style. A ratio-cut instruction specifies a location relative to the object's perceived extent (e.g., "cut 0.3 of the length from the right"). A middle-cut instruction selects the center. A split-cut instruction requires dividing an object into multiple approximately equal pieces; after the first cut, the remaining geometry changes, and subsequent cut locations must be estimated from the updated object state.

For a one-dimensional object extent of length L measured between its left and right reference points, a left-referenced ratio r and a right-referenced ratio r correspond conceptually to distances rL and $(1-r)L$ from the left endpoint, respectively. Thus the instructions "0.25 from the left" and "0.75 from the right" describe the same target location. This geometric identity is straightforward in an explicit planner but is not guaranteed to emerge from a policy trained on images and language. In CulinaryCut, left and right references are defined with respect to the camera

viewpoint; the planner converts them into a target within the object's estimated extent.

Four cutting styles provide distinct motion profiles: *normal cutting* uses a predominantly downward stroke; *saw cutting* adds periodic lateral oscillation; *bias cutting* adds a sustained lateral displacement during descent; and *guillotine cutting* uses a comparatively rapid vertical stroke. These styles are represented as task and demonstration variations. They are *not* generated by a separately trained Knife Style Transfer Module.

Normal Cut

Normal cutting is characterized by an approximately vertical downward movement. In a middle-cut task, the end effector descends once toward the center of the object. In a split-cut task, the motion contains several descending segments, one for each required boundary. Consequently, the trajectories for cutting into three, four, or five pieces differ primarily in the number and location of the individual descents, while retaining the underlying non-oscillatory cutting style.

Saw Cut

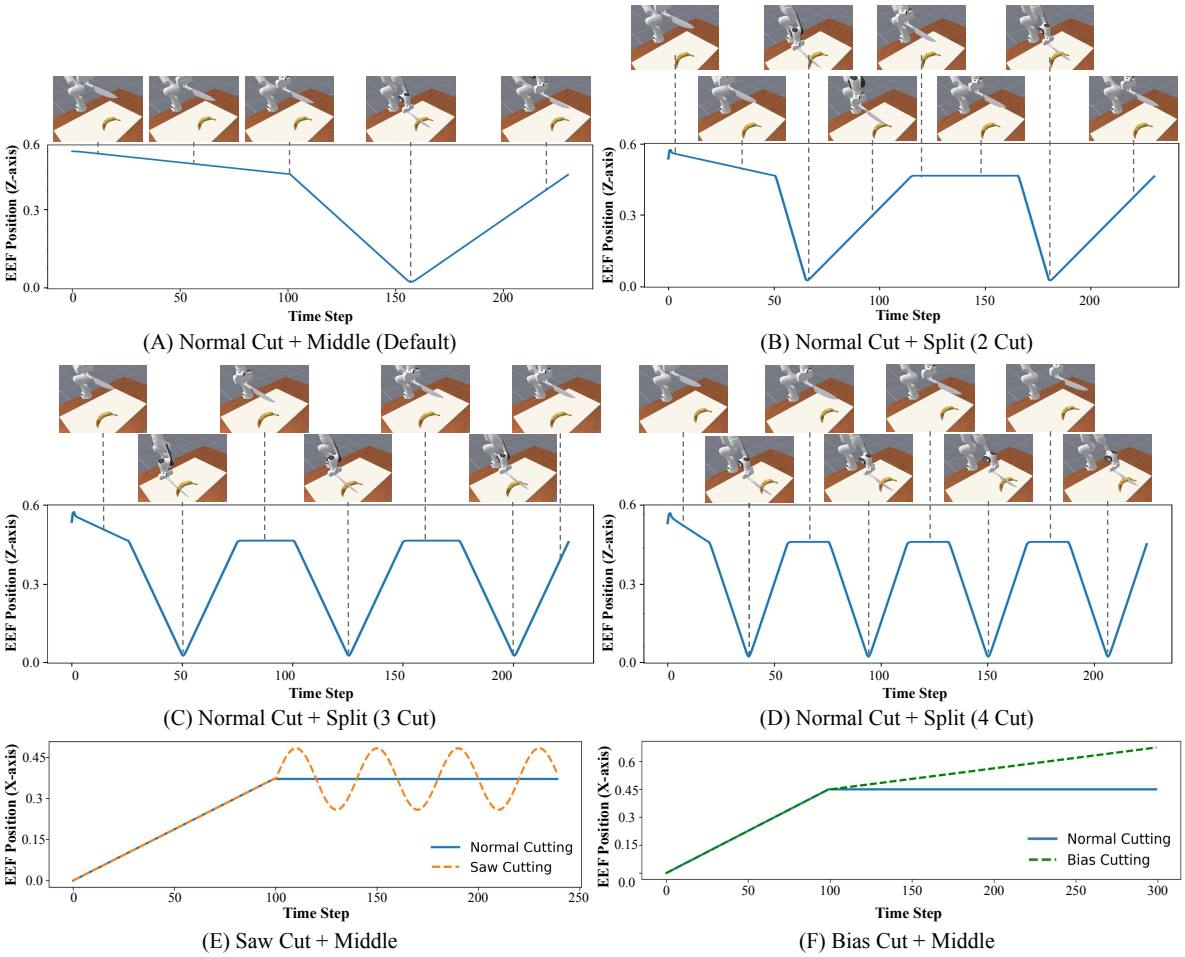
Saw cutting overlays periodic lateral motion on the downward stroke. We introduce forward–backward horizontal oscillations and randomly sample their frequency for each demonstration. This style changes the local temporal structure of contact and should not be treated as equivalent to a simple straight-line path. It is included as a data-generation variation, not as the output of a separately learned motion-style adapter.

Bias Cut

Bias cutting applies a sustained lateral shift while the knife descends. The resulting end-effector path is oblique rather than vertical, thereby modifying the geometry of the knife–material interaction. As illustrated by the trajectory samples, this style can be distinguished from normal cutting directly from the position trace.

Guillotine Cut

Guillotine cutting uses a comparatively rapid vertical stroke. Its position-only trajectory can look nearly identical to that of a normal cut; the distinguishing information is largely the time parameterization and associated contact transient. Accordingly, the trajectory figure omits a separate guillotine position trace and explains that the contact-force history carries information that a position-only plot would hide.



(A) Normal Cut + Middle (Default)

(B) Normal Cut + Split (2 Cut)

(C) Normal Cut + Split (3 Cut)

(D) Normal Cut + Split (4 Cut)

(E) Saw Cut + Middle

(F) Bias Cut + Middle

Figure 5.4: **Representative cutting motions.** Normal cutting follows one or several vertical descents, depending on the cut state; saw cutting adds oscillatory motion, and bias cutting produces a slanted trajectory. Guillotine cutting is primarily distinguished from normal cutting by speed, not its position-only trajectory.

5.4.2 Instruction Representation and Ambiguity Resolution

A cutting instruction is constructed from a *Cutting Specification*: a target food object O_t, a motion style ℓ_s, a cut state, and a directional reference. The cut state may specify a ratio, the middle of an object, or a split into approximately equal pieces. The directional reference describes the perspective from which the target location is measured, such as from the left end, from the right end, or along the principal extent of the object. The full instruction therefore contains more information than the object name and the verb "cut": it jointly specifies *what* to cut, *where* to cut it, and *how* the knife should move.

Instructions are assembled using a slot-filling template of the form

```
[CUT_STYLE] [VERB] the [OBJECT] at [POSITION] [DIRECTION]
```

where the slots are instantiated using predefined lexical alternatives. Numerical ratios may be expressed with decimals, percentages, or words, and LLM-generated paraphrases provide additional linguistic diversity while preserving the intended cutting specification. Table 5.4 gives representative instructions for the twelve style–state combinations. Here O_t denotes the selected target object rather than a literal string shown to a policy.

Table 5.4: **Representative language instructions in CulinaryCut.** The four cut styles are combined with the Ratio, Middle, and Split cut states; the specified reference frame changes the physical interpretation of otherwise similar ratio expressions.

Style	State	Example instruction
Normal	Ratio	Normal cut O_t at a 0.3 ratio from the right.
Normal	Middle	Normal cut O_t at the midpoint along its length.
Normal	Split	Normal cut O_t at the first split boundary from the right.
Bias	Ratio	Bias cut O_t at a 0.4 ratio toward the right end.
Bias	Middle	Bias cut O_t at the midpoint from the left.
Bias	Split	Bias cut O_t at the first split boundary.
Guillotine	Ratio	Guillotine cut O_t at a 0.5 ratio from the left.
Guillotine	Middle	Guillotine cut O_t at the center from above.
Guillotine	Split	Guillotine cut O_t at the first split boundary.
Saw	Ratio	Saw cut O_t at a 0.6 ratio along its length.
Saw	Middle	Saw cut O_t at the midpoint from the right.
Saw	Split	Saw cut O_t at the third boundary toward the right.

The dataset also defines default conventions to keep instructions complete. If only a cut state is specified, the style defaults to *Normal Cut*. If a style is given without a cut state, the state defaults to *Middle Cut*. These defaults are a practical property of the dataset construction process. They prevent an omitted attribute from creating multiple conflicting interpretations during trajectory generation and benchmark evaluation.

5.4.3 Trajectory Generation and Dataset Composition

The generation process begins with cutting demonstrations and task-specific trajectories rather than with a policy that is already capable of successful contact-rich cutting. The generation pipeline combines initial teleoperation examples for each cutting style, an AABB-based motion planner, and material simulation. The planner estimates the target object's spatial extent and maps the language-specified cut state and direction onto an executable cutting position. Object placement and cutting parameters are subsequently varied so that the same command family is not tied to a single initial pose or scene.

Data generation consists of five stages:

1. **Specify the task:** choose object, cut state, style, and directional reference.
2. **Plan the knife trajectory:** map the instruction to a target using object geometry and style demonstrations.
3. **Simulate contact:** advance ManiSkill and MLS-MPM together, including material resistance and separation.
4. **Record the rollout:** store synchronized actions, observations, material states, and force profiles.
5. **Augment the data:** vary object states, visual scenes, and language while preserving task semantics.

An important feature of the third stage is that the physical result is not inferred from the requested instruction. A candidate can reach the intended position yet fail to produce material separation. This allows the generation process to distinguish an *intended cut* from an *executed cut*. The comparison between ManiSkill-only and IntSim-grounded training data later uses this distinction as the central experimental variable.

The 15 food items are banana, cucumber, apple, peach, melon, orange, strawberry, pear, kiwi, tomato, lemon, grape, shine muscat, plum, and cherry. The dataset contains 21,000 base trajectories. Each is combined with 15 rendered indoor backgrounds, drawn from ReplicaCAD (Szot et al., 2021), AI2-THOR (Kolve et al., 2017), and ProcTHOR (Deitke et al., 2022), and at least five language realizations. Under the reported expansion protocol, the resulting instance count is

$$N_{\text{instances}} = 21{,}000 \times 15 \times 5 = 1{,}575{,}000. \tag{5.8}$$

The expanded instances need not represent independent physical rollouts: visual scenes and language variants may reuse the underlying action/material trajectory. Keeping this distinction explicit avoids interpreting augmentation as additional physics simulations.

5.4.4 Dataset Preview: Robotic Observations and Particle-Level Cutting

The gallery figures serve an important methodological purpose in addition to visual presentation. The upper row for each example is the robot-scene RGB observation generated through ManiSkill; the lower row shows the corresponding MLS-MPM particle configuration. Because the two representations are temporally aligned, they reveal both what

a VLA observes and what the material solver computes. The progression of frames makes the distinction between blade motion, local deformation, and visible separation easier to identify than a single final-state image would.

Figure 5.5 contains apple, banana, peach, grape, and cucumber examples. Figure 5.6 adds berry, lemon, orange, and melon. These figures illustrate nine representative foods rather than every one of the fifteen benchmark categories. The purpose is therefore qualitative breadth, not a claim that each material's force response has been calibrated to the same degree. In particular, the images show how the simulator handles different geometry and separation patterns, whereas the real-versus-simulation force experiment is quantitatively reported for banana cutting.

Two observations from the galleries are relevant to the benchmark's design. First, the material's evolving geometry can be a source of supervision or an evaluation criterion even when the VLA itself receives only rendered RGB images. Second, the location that should be cut next may depend on the separation that has already occurred. This becomes particularly important for the topology-update and sequential split-cut experiments, where a policy must interpret the changed scene after its previous action. For a dissertation centered on deformable simulation, these examples also show the scope of the numerical implementation more directly than a table of VLA success rates can.

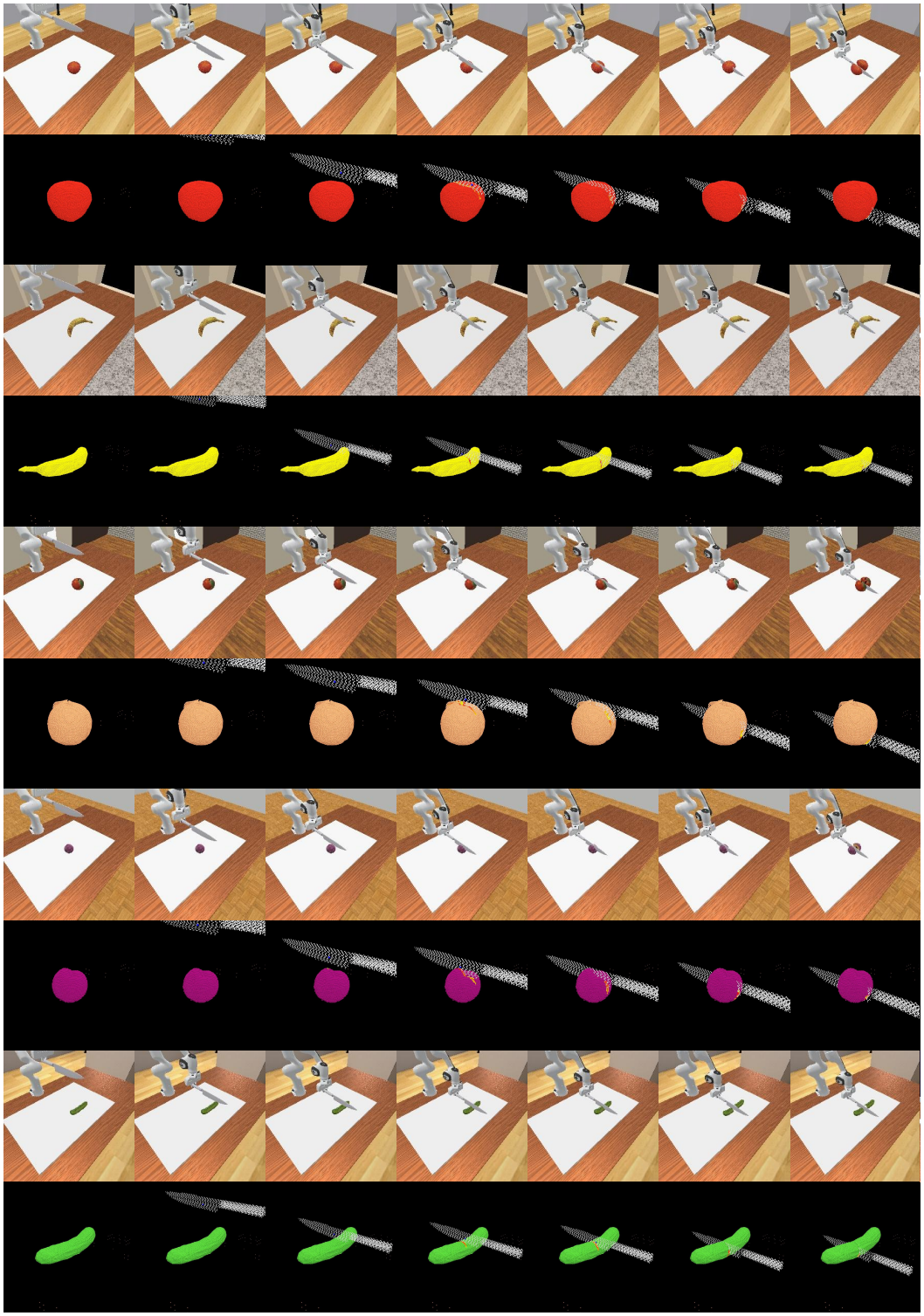


Figure 5.5: **CulinaryCut dataset gallery (1/2).** Representative cutting sequences for apple, banana, peach, grape, and cucumber. For each food object, the upper row shows temporally ordered RGB observations rendered in ManiSkill with the Franka Panda, blade, and cutting board, while the lower row shows the corresponding MLS-MPM particle configurations. Columns progress from blade approach through deformation to the material-separation stage. The paired views expose particle-level deformation and topology changes that cannot be inferred from robot motion alone.

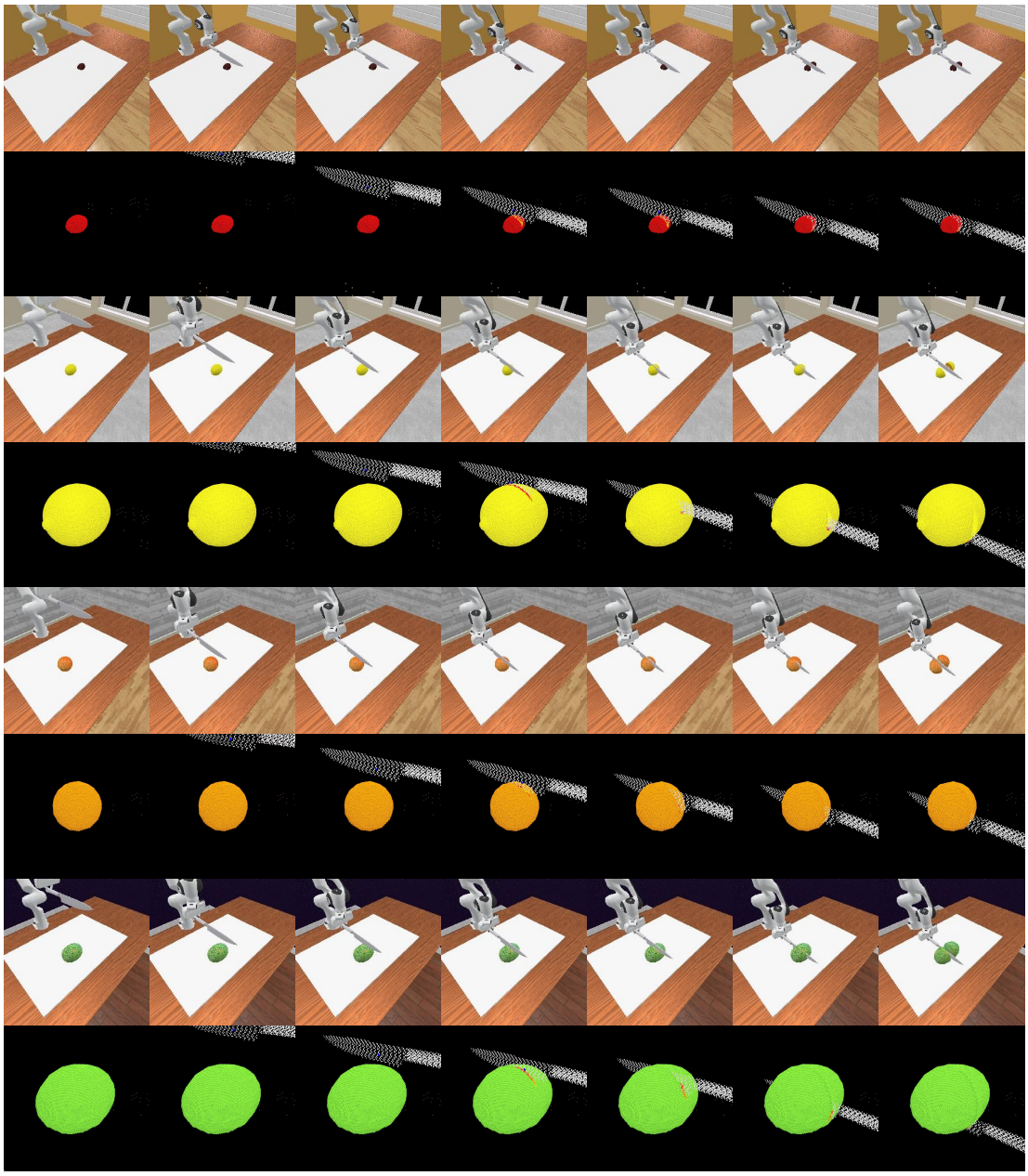


Figure 5.6: **CulinaryCut dataset gallery (2/2).** Additional cutting examples for berry, lemon, orange, and melon, using the same temporally aligned ManiSkill RGB/MLS-MPM particle-view arrangement as Figure 5.5. The displayed sequences illustrate the material-response pipeline for food objects of different sizes and shapes. These images document simulated deformation and separation; force realism is assessed separately through the real-robot measurements.

5.4.5 VLA Baselines and Optimization Interface

We fine-tune RDT-1B, Octo, and OpenVLA, abstracting their interfaces as

$$\hat{\mathbf{a}}_{t:t+H_\pi-1} = \pi_\theta(o_{\le t}, \ell), \tag{5.9}$$

where ℓ is the instruction and H_π the model-specific action horizon. The observation contains supported vision/proprioception, *not* the simulator's force, contact, or damage states. Baseline architectures and fine-tuning settings are retained.

Table 5.5: **Fine-tuning settings for the three VLA baselines.** All three baselines use the front camera as the policy image input.

Setting	RDT	OpenVLA	Octo
Training scheme	Full fine-tuning	LoRA ($r = 32$)	Full fine-tuning
Batch size	32	16	128
Learning rate	1×10^{-4}	5×10^{-4}	3×10^{-5}
Action representation	Joint position	Delta end-effector	Joint position
Action chunk	64 steps	1 step	4 steps
Control frequency	20 Hz	5 Hz	5 Hz
Training updates	200K	200K	1M

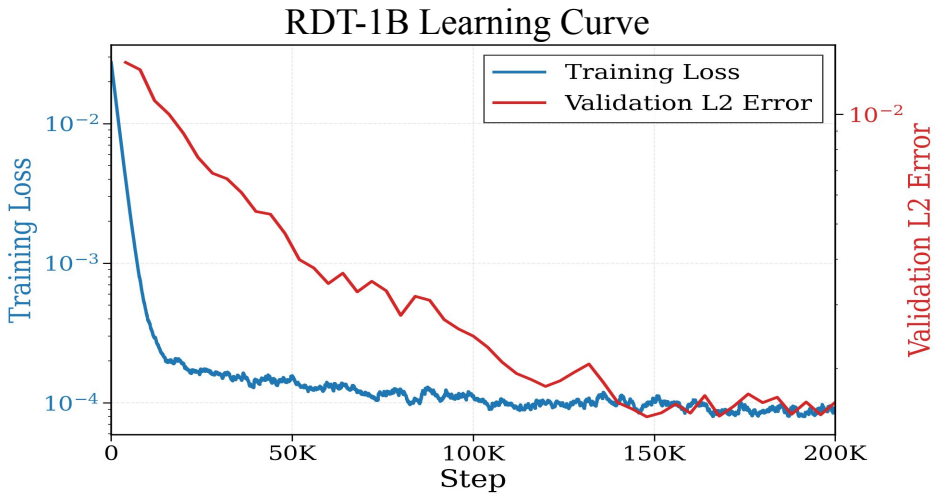


Figure 5.7: **RDT fine-tuning learning curve.** The learning curve provides a qualitative check of RDT fine-tuning convergence.

To isolate the value of physics-grounded demonstrations, we also compare *the same RDT architecture* trained using two different data sources. The kinematic baseline uses ManiSkill-generated trajectories that satisfy geometric cutting constraints. The physics-grounded variant uses IntSim-generated rollouts selected under MLS-MPM material dynamics. Neither receives forces as policy inputs. This experiment specifically tests whether material-aware behavior can be learned from *physically consistent action trajectories*, rather than from an additional online force-sensing

modality.

5.4.6 Evaluation Metrics

Evaluation uses randomized object positions, orientations, and scales, with 20 held-out randomized trials per object/task configuration in the reported simulation protocol. Under the geometry-based criterion, a cut succeeds when the following conditions are satisfied: (i) the knife tip reaches the table within 0.05 m positional error; (ii) the blade intersects the intended region; (iii) the blade–object contact angle is within $90^\circ \pm 10^\circ$; and (iv) the cutting location is within 10% of the object's length from the instructed position. Separate physical-severing evaluation within IntSim additionally checks whether the executed trajectory produces an actual cut under deformable-material dynamics.

For real-robot experiments, success is evaluated using both functional penetration and geometric accuracy: the blade must penetrate at least two-thirds of the object's cross-section, and the resulting piece lengths must satisfy the specified cut-location tolerance. These criteria should not be conflated with geometry-only accuracy reported from ManiSkill; the two environments measure different aspects of the same task.

5.5 Experiments and Results

5.5.1 Visual and Physical Validation of IntSim

Figure 5.8 shows a representative orange-cutting sequence. ManiSkill supplies the arm and visual scene, while MLS-MPM advances the material state and exposes progressive separation. This example demonstrates why a visual path-following criterion is insufficient: the material dynamics determine whether a knife trajectory actually severs the food.

We also compare simulated knife contact with the vertical reaction force measured during real Franka Panda banana cutting. In a representative experiment, the wrist force/torque sensor exhibits a pre-contact baseline of approximately 25 N, attributed to tool load and robot-side sensor offset. After baseline subtraction, the real cutting-force peak is approximately 12.5 N, compared with approximately 12.0 N from IntSim (less than 5% relative difference). Normalizing both profiles by their peaks gives similar rise and peak timing, although the release phases differ.

We also compare force profiles across multiple trials: 20 real banana trials are compared with 30 simulated trials after time rescaling. Their mean profiles share an overall rise–peak–decay pattern, but the simulated mean is smoother and somewhat lower around the cutting transition. This residual discrepancy is consistent with the simplified damage model and numerical smoothing intrinsic to particle–grid transfers; it does not establish fidelity for all materials or cutting styles.

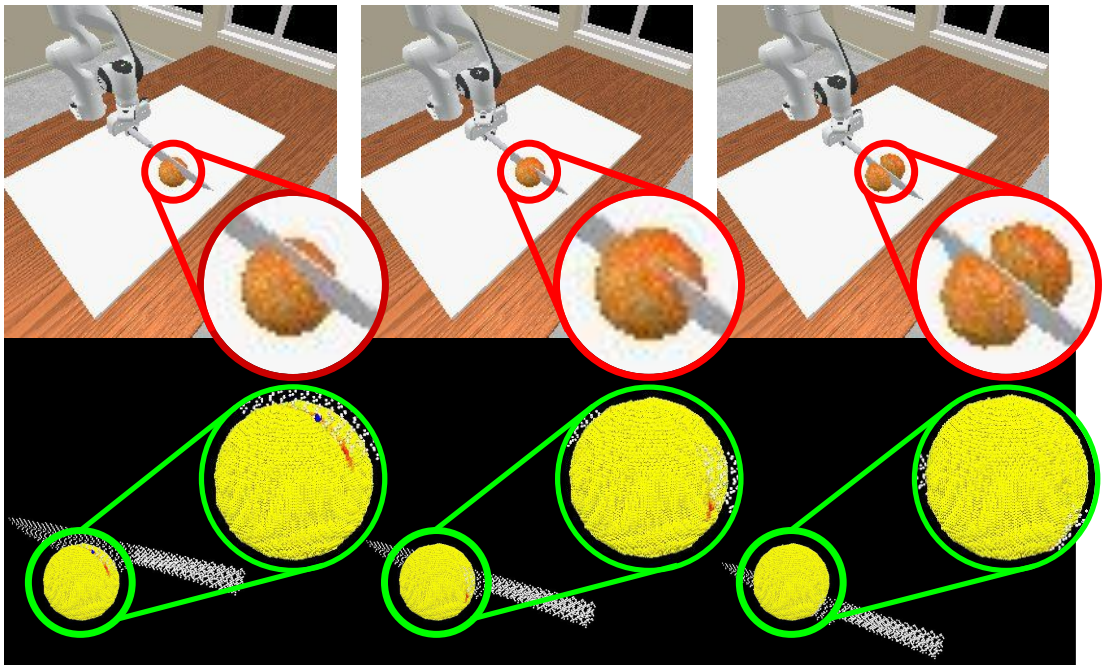


Figure 5.8: **IntSim material response during orange cutting.** The coupled system updates robot motion and particle-level deformation, allowing geometric blade intersection to be distinguished from actual material separation.

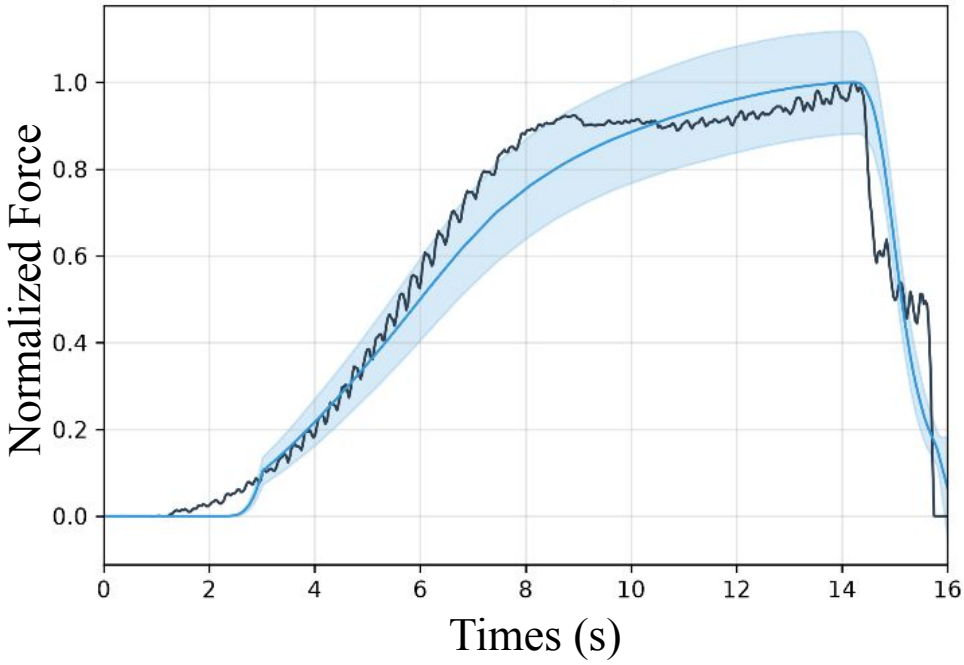


Figure 5.9: **Representative force validation against a real Franka robot.** Real banana-cutting force measurements are compared with IntSim contact-force estimates after accounting for the real sensor's pre-contact baseline. Peak-normalized profiles support temporal comparison.

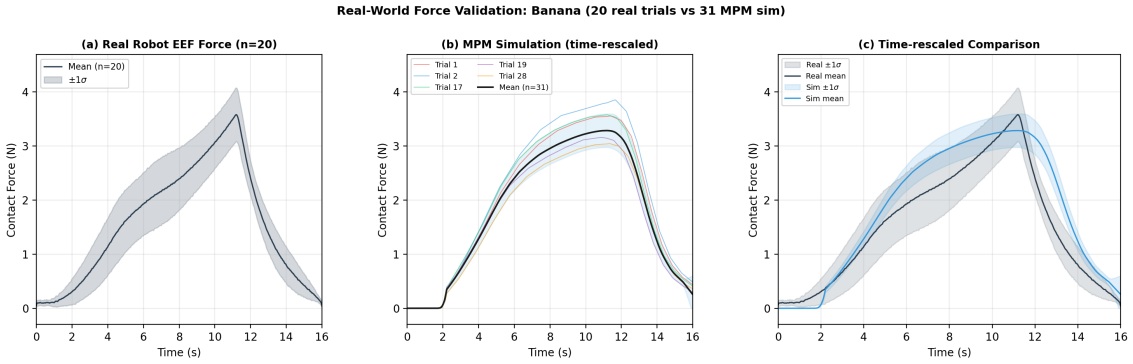


Figure 5.10: **Real–simulation force comparison.** Mean banana-cutting force curves from 20 real and 30 simulated trials, with variation bands, show broadly aligned rise–peak–decay behavior after time rescaling; the simulated cutting transition is smoother. This multi-trial analysis complements the representative peak-force comparison.

5.5.2 Geometry-Gap Evaluation

5.5.2.1 Object-Scale Sensitivity and Multi-Object Scenes

Single-object evaluation shows that cutting-point localization varies markedly with object geometry and scale, with smaller objects presenting a particular challenge. Adding distractor foods further degrades target grounding, even when the instruction names the intended target. Table 5.6 summarizes geometric generalization performance.

Table 5.6: **Geometric generalization across evaluation settings.** Success rates under single-object, multi-object, and held-out-object evaluation conditions.

Evaluation setting	RDT	Octo	OpenVLA
Single object	69%	42%	51%
Multi-object scene	31%	17%	28%
Held-out object category	43%	34%	26%

In the multi-object setting, RDT decreases from 69% to 31%, Octo from 42% to 17%, and OpenVLA from 51% to 28%. In leave-one-object-out transfer, performance also drops: RDT reaches 43%, Octo 34%, and OpenVLA 26%. Thus, policies that can execute familiar cutting instructions on known objects remain sensitive to clutter and unseen object appearance or geometry.

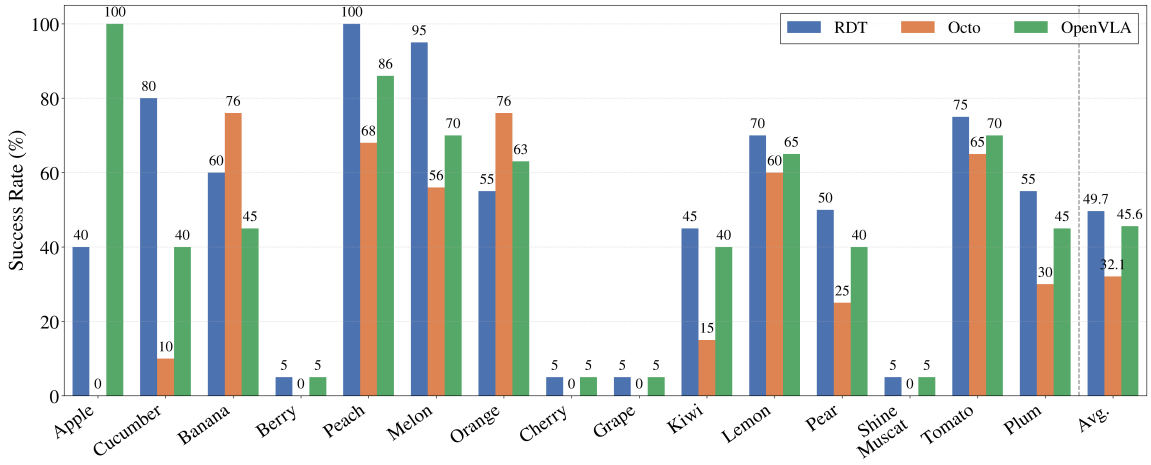


Figure 5.11: **Single-object cutting across food categories.** The experiment compares success rates under variations in food geometry and scale.

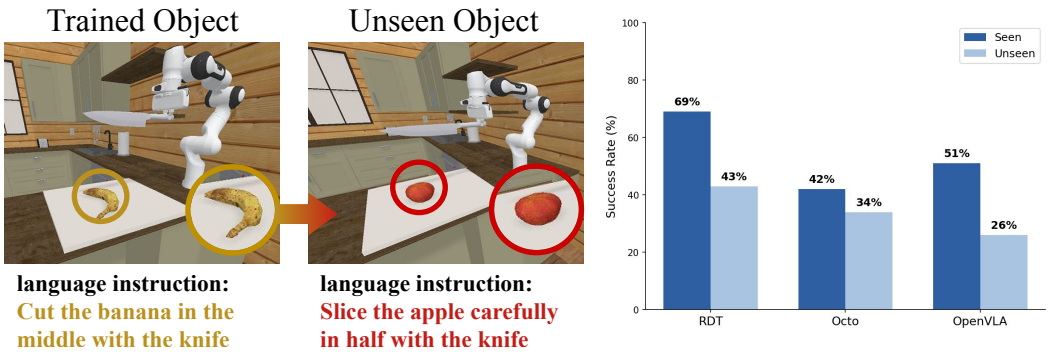


Figure 5.12: **Transfer to an unseen food object category.** Held-out-category evaluation measures the ability of trained VLA policies to execute cutting motions on objects with previously unseen visual appearance, shape, and material properties.

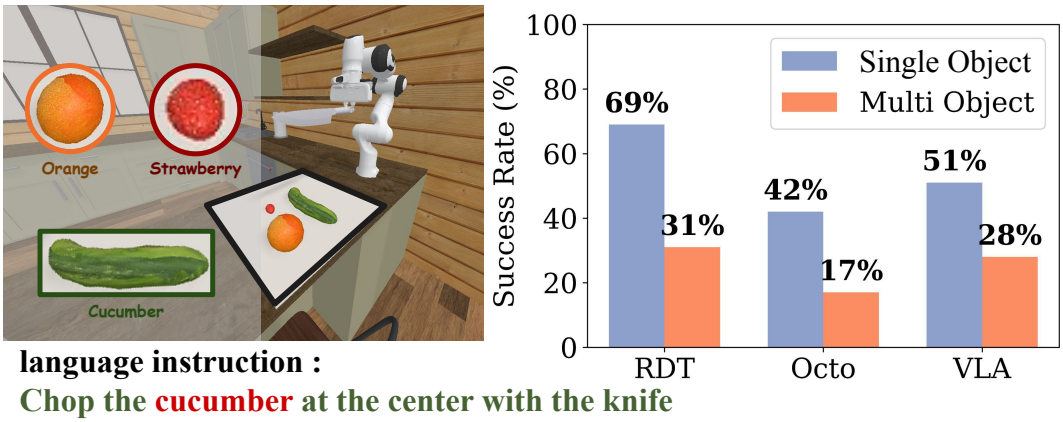


Figure 5.13: **Multi-object geometric grounding.** The presence of additional food items decreases the probability of selecting and cutting the requested target across all three VLA baselines.

5.5.2.2 Continuous Ratios and Directional Reference

Continuous-ratio evaluation probes whether a policy maps instructions such as "0.4 from the right" to an object-centric coordinate. Across ratios between 0.1 and 0.9, performance is unstable even for nearby numerical values, particularly for Octo and OpenVLA. A further RDT transfer experiment shows that accuracy decreases from 55% to 20% when the training condition at 0.25 from the right is changed to the tested left-side condition; in the reverse-direction configuration, success decreases from 35% to 0%. Transferring from a 0.5-ratio condition to a 0.25-ratio condition reduces success from 60% to 25%. These experiments identify shortcomings in jointly representing numerical ratios, directional references, and perceived object extent.

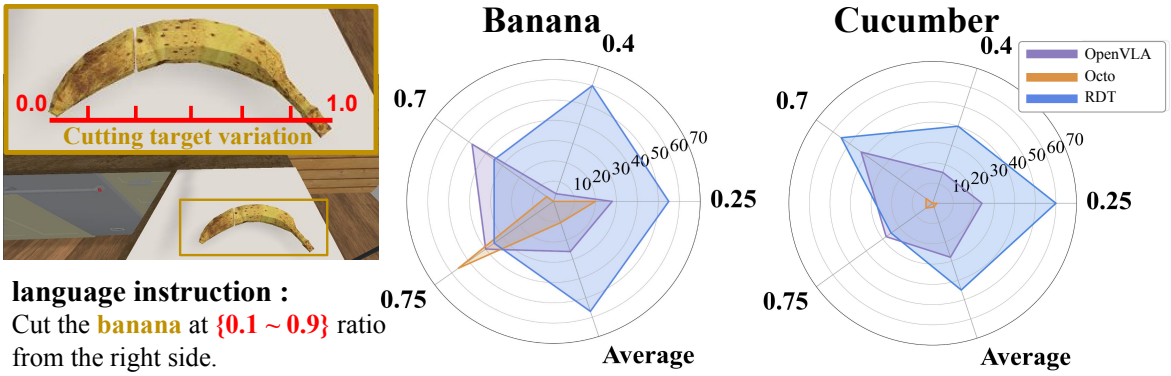


Figure 5.14: **Continuous-ratio cutting.** Performance across ratio-conditioned banana and cucumber tasks varies strongly as the requested proportion changes, revealing a geometry-grounding gap despite successful execution at some trained configurations.

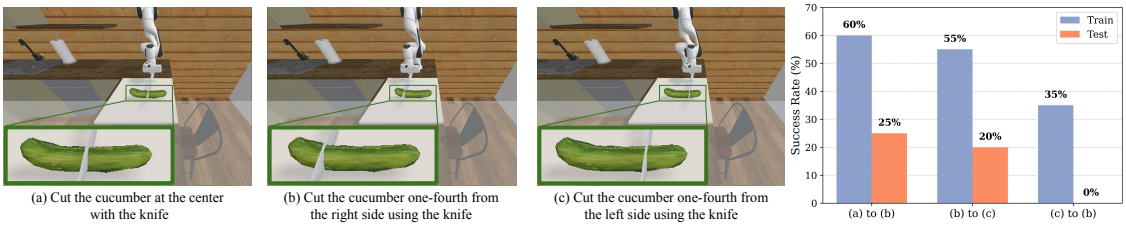


Figure 5.15: **Directional and ratio transfer.** Transfer experiments compare training and test conditions with different directional references or cut ratios. Accurate execution of a familiar ratio does not guarantee generalization to a changed instruction.

5.5.2.3 Sequential Split Cuts

When an instruction requests equal-size pieces, the robot must repeatedly localize a new cutting position after every separation. Table 5.7 reports the step-wise cut-placement success for RDT on banana and cucumber. Accuracy is already low on the first placement and approaches zero on subsequent cuts. These failures indicate that short-horizon cut localization does not automatically provide sequential geometric reasoning over the changing material topology.

The table records the accuracy of *individual numbered cuts*, not the fraction of episodes completing an entire

multi-piece division. Correctly placing one of the two boundaries required for three pieces is only partial success; later boundaries must be recomputed after material separation.

Table 5.7: **RDT step-wise split-cut evaluation.** Each entry is the percentage of episodes in which the corresponding numbered cut is placed within the prescribed $\pm 10\%$ tolerance; a dash indicates that the specified division does not require that cut.

Food	Division	Cut 1	Cut 2	Cut 3	Cut 4
Banana	Into 3 pieces	20%	0%	–	–
Banana	Into 4 pieces	10%	5%	0%	–
Banana	Into 5 pieces	5%	0%	0%	0%
Cucumber	Into 3 pieces	30%	0%	–	–
Cucumber	Into 4 pieces	10%	0%	0%	–
Cucumber	Into 5 pieces	0%	0%	0%	0%

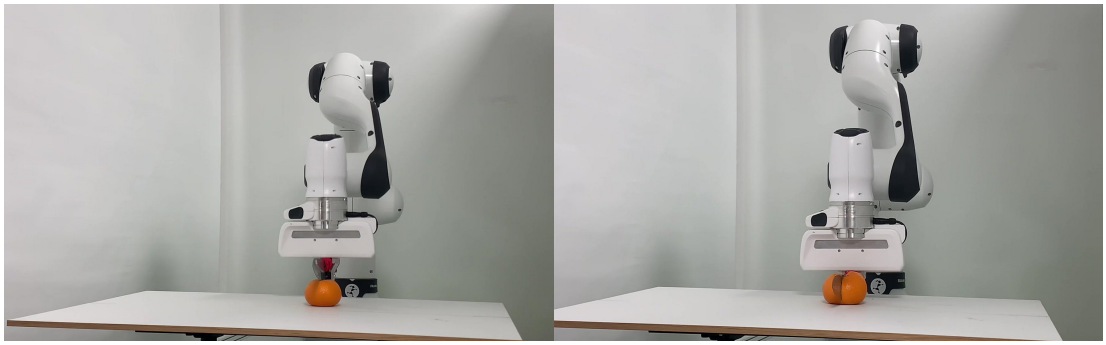
5.5.3 Physics Gap and Real-Robot Transfer

Geometry-based success does not establish physical success. To quantify this distinction, we train RDT either on ManiSkill trajectory-only demonstrations or on IntSim demonstrations selected under material dynamics, without changing the architecture or its input modalities. Table 5.8 compares these training sources when the resulting policy is tested in a geometry-only robot simulator, in the integrated material simulator, and on a real Franka Panda robot cutting an orange.

Table 5.8: **Effect of physics-grounded demonstrations for RDT orange cutting.** Both policies use the same architecture and do not receive force as policy input. The large difference under IntSim is hidden by the geometry-only ManiSkill evaluation. Real-robot results are counts out of 10 trials.

Training data source	ManiSkill	IntSim	Real robot
ManiSkill trajectories	62%	23%	1/10
IntSim trajectories	**68%**	**59%**	**3/10**

The two policies perform similarly when evaluation checks geometric execution alone (62% versus 68%). Once material resistance and severing are enforced, however, the policy trained on ManiSkill data achieves only 23% success, compared with 59% for the policy trained on IntSim data, an improvement of **36 percentage points**. Real-robot tests show the same direction, increasing from 1/10 to 3/10 successful cuts, although the sample is too small for strong statistical claims. This contrast isolates the benefit of physically grounded training trajectories without attributing the improvement to an online force-conditioned policy.



(a) ManiSkill-trained policy (b) IntSim-trained policy (Ours)

Figure 5.16: **Real-world comparison of trajectory sources.** Representative Franka Panda orange-cutting attempts contrast a policy fine-tuned on ManiSkill-generated trajectories and one fine-tuned on IntSim-generated physically grounded trajectories. These qualitative examples complement the reported trial-level success rates.

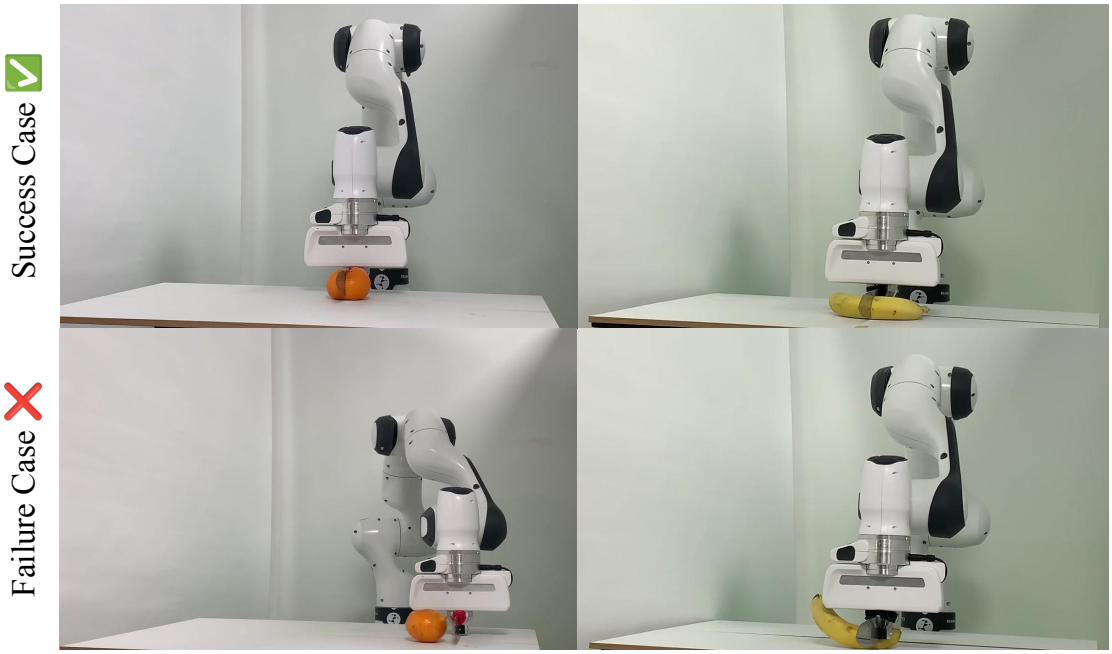


Figure 5.17: **Real-world orange cutting.** Qualitative examples compare RDT fine-tuned using kinematic ManiSkill demonstrations and RDT fine-tuned using IntSim demonstrations. The latter yields more physically effective cutting in the illustrated trials; quantitative results are reported in Table 5.8.

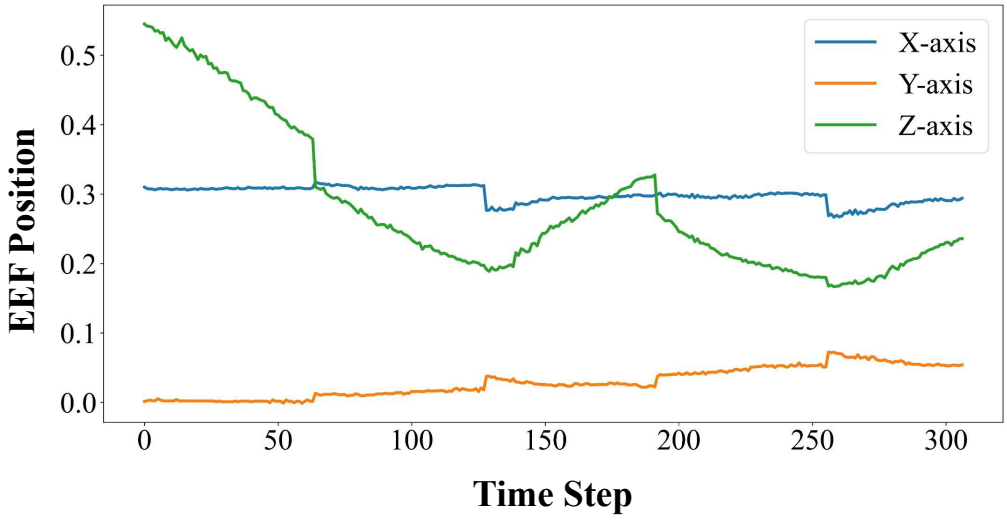
5.5.4 Ablation: Importance of Topology Updates

To examine whether the representation of cutting-state changes affects learned behavior, we independently enable or disable post-cut mesh-connectivity updates in training and evaluation. Table 5.9 reports the resulting RDT success rates. The aligned topology-aware setting achieves 60%, compared with 55% when neither training nor evaluation updates topology; mismatched settings decrease to 50% and 45%. Beyond these moderate numerical differences, qualitative rollouts reveal that topology-aware training combined with topology-unaware evaluation can cause the policy to repeat a cut, because the observation does not clearly indicate that the previous cut has completed.

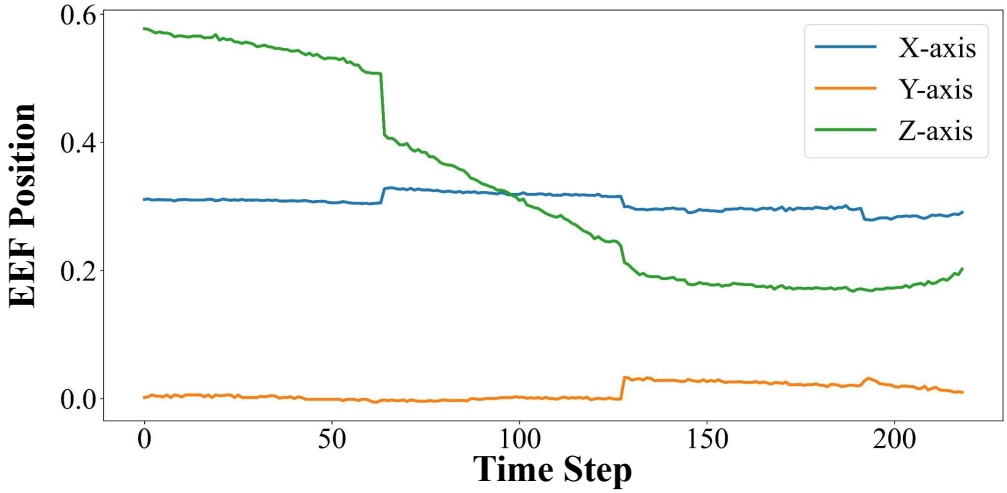
Table 5.9: **RDT topology-update ablation.** "Yes" indicates that the object's rendered connectivity is updated after successful cutting.

Case	Training update	Evaluation update	Success
1	No	No	55%
2	No	Yes	50%
3	Yes	No	45%
4 (ours)	Yes	Yes	**60%**

This finding is consistent with the split-cut results: sequential manipulation requires a representation of *what has already changed*, not simply a memorized knife motion. The ablation does not imply that mesh-connectivity updates alone solve long-horizon planning, since split-cut performance remains poor.



(A) w/o Topology Update. (Case 3)



(B) w/ Topology Update. (Case 4)

Figure 5.18: **Qualitative impact of topology updates.** RDT trajectories can revisit an already cut location when post-cut connectivity is not reflected in visual observations; topology-aware evaluation more clearly signals that the cut has completed.

5.5.5 Ablation: Execution-Time Velocity Guardrail

The velocity-guardrail ablation uses 20 trials to compare unconstrained execution to filtered end-effector velocities. Without filtering, the reported maximum cutting speed is 3.63 m/s and maximum contact force reaches 129.31 N. With the filter enabled, these quantities decrease to 0.89 m/s and 37.78 N, respectively. Under the 100 N threshold used in the study, the unconstrained run exhibits excessive interaction forces, whereas the filtered run suppresses them. These measurements characterize an *external safety mechanism*, not an ablation of force inputs to the learned VLA.

Table 5.10: **Velocity-guardrail ablation.** Reported maximum cutting speed and contact force with and without the execution-time filter.

Execution setting	Max. knife speed	Max. contact force
No guardrail	3.63 m/s	129.31 N
With guardrail	0.89 m/s	37.78 N

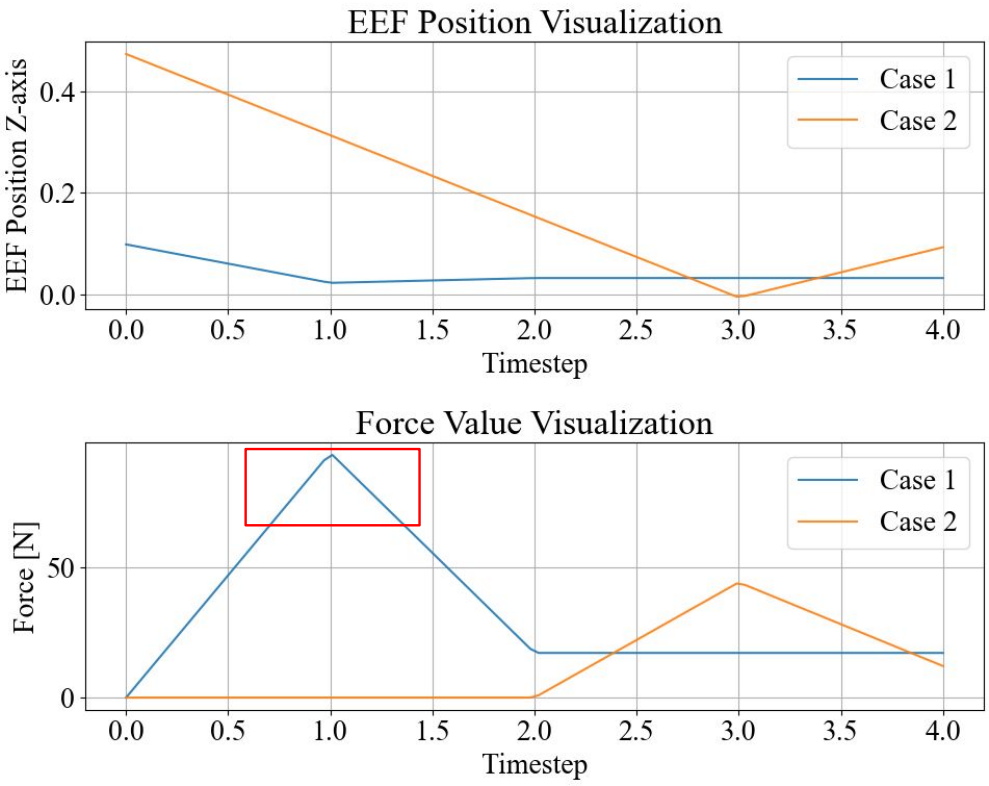


Figure 5.19: **Cutting motion and force with the velocity guardrail.** The evaluation compares vertical end-effector motion and contact-force traces with and without the external velocity clip, illustrating the suppression of sharp force spikes.

5.5.6 Ablation: Food–Board Friction

A single-arm cutting arrangement relies on the interaction between the food and cutting board to keep the food from moving with the blade. We compare board–food friction under otherwise matched cutting motions: with board–food friction coefficient $\mu = 0.5$, the object is more stable, whereas setting $\mu = 0$ yields approximately $0.034\,\mathrm{m}$ of center-of-mass drift. This effect is important because an object translation can change both the realized ratio location and the contact forces, even if the robot's commanded knife path is unchanged.

5.6 Discussion and Limitations

5.6.1 Interpreting the Geometry and Physics Gaps

Object variation, clutter, ratios, and split cuts evaluate how a VLA grounds instructions in geometry. In contrast, the ManiSkill/IntSim comparison assesses whether a geometrically plausible motion actually severs deformable material. The two success criteria are complementary and should not be conflated.

5.6.2 What the Benchmark Demonstrates about Simulation

MLS-MPM provides evolving material states, while signed-distance contact, friction, and damage connect tool motion to deformation and separation. The galleries demonstrate qualitative breadth, whereas measured banana force curves provide narrower quantitative evidence. Improved IntSim-trained RDT performance supports *data-level physical grounding*; it does not establish that the policy learned online force control or an explicit constitutive model.

5.6.3 Limitations and Future Directions

Material Modeling and Cutting Fidelity

The material solver approximates cutting using fixed corotated elasticity, optional plasticity, damage-dependent stiffness reduction, and contact rules. It does not explicitly reproduce every relevant tissue fiber, moisture effect, adhesion response, viscoelastic relaxation mechanism, or microscopic fracture process. Its particle–grid discretization can also smooth sharp force transitions. Consequently, the demonstrated resemblance between real and simulated banana force profiles should not be interpreted as complete constitutive validation of fifteen food categories. The reported material parameters are literature-informed starting values with numerical adjustments where needed.

Scope and Size of Physical Validation

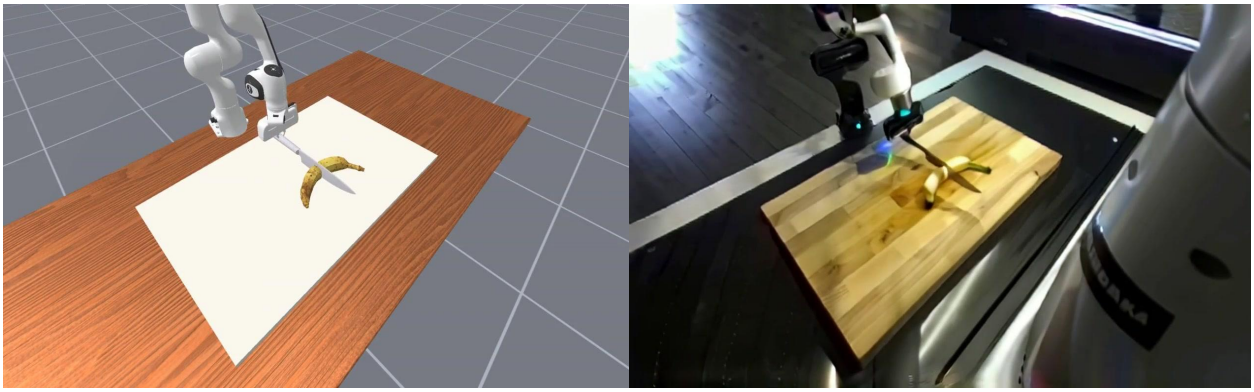
Force validation focuses on banana cutting, and the real orange-cutting comparison uses ten trials per policy. Broader food categories, cutting styles, and trial counts are required to characterize physical fidelity and deployment uncertainty.

Object Stabilization and Execution

Single-arm cutting relies on passive friction against the board. Its removal produces centimeter-scale drift in the reported ablation; active holding and its additional contact mechanics remain future work.

Visual Realism and World-Model Augmentation

The simulator's images retain a real-world appearance gap. We also examine Cosmos-based appearance enhancement (NVIDIA et al., 2025) while preserving the underlying cutting sequence. Such enhancement complements physical modeling but does not improve force or fracture accuracy by itself (Figure 5.20).



(a) Simulation dataset (Ours) (b) Real-world transfer with a world model

Figure 5.20: **Exploratory visual enhancement with Cosmos.** The figure compares simulator-rendered cutting frames with appearance-enhanced outputs. Visual enhancement is complementary to, not a replacement for, the physical simulation and force validation in IntSim.

5.7 Chapter Summary

This chapter introduced CulinaryCut and its IntSim environment, which couples MLS-MPM deformable dynamics to ManiSkill robot execution. The solver uses fixed corotated elasticity, optional plasticity, damage, tool contact, and numerical safeguards to generate and evaluate cutting trajectories beyond geometric knife placement.

CulinaryCut comprises 21,000 base trajectories over 15 foods, augmented to approximately 1.575 million instances. VLA experiments reveal both geometry and physics gaps. With unchanged RDT architecture and force-free inference, IntSim-grounded training improves physical orange-cutting success from 23% to 59% in integrated simulation, with an initial real-robot improvement from 1/10 to 3/10. These findings support physics-grounded simulation as a source of richer data and more meaningful evaluation for deformable manipulation.

CHAPTER 6

Limitations and Future Directions

The preceding chapters demonstrate three complementary uses of physics-based particle simulation: controlling deformable motion through optimized internal stresses (Chapter 3), incorporating image-space feedback into that control (Chapter 4), and generating and evaluating physically grounded robotic cutting demonstrations (Chapter 5). Although these studies share the Material Point Method (MPM) as a modeling foundation, they address different objectives and do not constitute a single end-to-end differentiable system. This chapter summarizes the limitations of each study and identifies directions in which their methods and evaluations can be strengthened.

6.1 Limitations of Physics-Based Shape Morphing

6.1.1 Time Integration and Optimization Cost

The differentiable MLS-MPM method in Chapter 3 uses explicit time stepping and relatively small simulation time steps; the reported morphing examples typically use $\Delta t = 1/120\,\mathrm{s}$. This numerical time step should not be confused with a measured real-time execution rate. Gradient-based control requires repeated forward simulations and backward passes, so the wall-clock cost grows with the number of simulation steps, control intervals, and optimization iterations. Damping and numerical stabilization improve robustness but can also change the temporal character of the resulting motion. More efficient integration and control optimization would be needed for applications requiring rapid interactive updates.

6.1.2 Limited Optimization Horizons

Differentiating through many MPM time steps requires retaining or reconstructing intermediate simulation states. Long unrolled trajectories can therefore be costly in memory and susceptible to poorly conditioned or attenuated gradients. Chapter 3 addresses this problem by optimizing shorter simulation networks and chaining their final states. This permits longer morphing sequences, but it does not optimize all control variables over an arbitrarily long sequence simultaneously. Consequently, local improvements need not yield a globally optimal long-horizon trajectory.

6.1.3 Dependence on the Objective and Smoothing Parameters

The correspondence-free, log-based nodal-mass objective reduces the mass ejection observed with a direct nodal-mass discrepancy. Nevertheless, the need for a mass penalty and carefully selected optimizer settings shows that optimization behavior is sensitive to the objective's scale and shape. Matching grid-mass distributions also does not guarantee an equally accurate rendered boundary or recovery of every thin feature. In the reported examples, deformation-

gradient smoothing factors are selected within approximately $\gamma = 0.95$–0.98 to suppress oscillatory artifacts. Such smoothing is useful for visual stability but can reduce sensitivity to rapid local deformation, and its preferred strength may change with geometry, material stiffness, and discretization.

6.2 Limitations of Render-Guided Volumetric Morphing

6.2.1 The Render Term's Authority after Arrival

In Chapter 4 the render weight is set in every window so that the render term holds a fixed share of the first gradient. While the body travels to the target this gives the render term real influence: it redirects material well beyond the run-to-run noise. Once the body has arrived, however, the physics gradient becomes small, the render weight follows it, and the render term only polishes the outline. The end is consistently but only slightly closer to the target than without the render term. The physics objective reads only the positions of the particles, so it cannot tell which material ends where, nor relief finer than the three-dimensional target resolves, and these are exactly the things a picture could decide. Holding the render weight fixed lets the render term act on them, but the body then keeps moving at the end of the run. A weight that keeps the render term's authority where it has something to decide and still lets the body come to rest has yet to be found.

6.2.2 Surface Detail below the Grid

Much of what a picture asks for lies below one MPM cell. The rigid-free offset of the outer particle layer reaches this scale, but it is also the main source of the few particles that briefly separate from the body while it arrives. The interpolation kernel averages the grid velocity over about two cells, so gaps of the target that are narrower than this open late and can tear into beads when they do. A finer cell opens some of these gaps but costs time and leaves holes in the early surface, so the cell size remains a trade-off to be settled by further comparison.

6.2.3 Rendering Representation versus Physical State

The render terms read a display surface built from the particles, the exterior of their own field covered by surface discs; this surface carries no mass of its own and is not a second physical discretization. An improved silhouette therefore cannot certify the accuracy of internal stress, contact response or material parameters, and part of the render term's lead on its own measure comes from a smoother arrangement of the outer layer rather than a different shape, which is why Chapter 4 reads shape against the exact target mesh. The weak ambient drag does not conserve angular momentum, and most of the delivered frames come from a long, slow tail after the body has arrived. Joint evaluation of rendering quality and physical behavior remains important.

6.3 Limitations of Physics-Grounded Robotic Cutting

6.3.1 Geometric Generalization and Sequential Actions

The CulinaryCut experiments in Chapter 5 expose a *geometry gap*: successful cutting on familiar objects or instructions does not ensure accurate placement under changes in object scale, target identity, directional reference, or requested cut ratio. Multi-object scenes and held-out food categories reduce performance, and sequential split cuts remain particularly difficult because the next cutting location must be recomputed after the geometry changes. These observations indicate that short-horizon action prediction does not by itself provide reliable object-centric or long-horizon reasoning.

6.3.2 Approximate Cutting Mechanics and Material Calibration

IntSim combines MLS-MPM, signed-distance representations of the knife and board, contact and friction handling, and a localized damage mechanism. Its damage thresholds, blade-proximity bands, and resolution-dependent scalings are practical modeling choices rather than a general guarantee of resolution-independent fracture. Real food materials may exhibit heterogeneity, rate dependence, adhesion, and fracture behavior beyond the chosen constitutive and damage approximations. Material parameters are initialized from available information and adjusted where necessary for numerical robustness; the work does not establish independent constitutive calibration for all 15 food categories. More extensive measurements would be necessary before claiming material-independent predictive force fidelity.

6.3.3 Limits of Physical Validation and Sim-to-Real Transfer

The reported force validation compares real and simulated banana cutting, including an aggregate comparison of 20 real and 30 simulated trials. These measurements provide useful evidence for a representative contact interaction, but cannot establish force accuracy across every food, knife motion, or material condition. The real-robot orange-cutting comparison is also preliminary: the two RDT training conditions yield 1/10 and 3/10 successful trials, respectively. This sample is insufficient to infer robust transfer across objects, embodiments, or deployment settings. Furthermore, CulinaryCut's approximately 1.575 million augmented observation–instruction instances are derived from 21,000 underlying physical trajectories; visual and language augmentation should not be counted as independent physics rollouts.

The evaluated VLA architectures do not receive simulated or measured contact forces as policy inputs. Consequently, the observed improvement from physics-grounded training data supports *data-level physical grounding*, not the acquisition of explicit online force feedback. Likewise, the execution-time velocity guardrail is an external safety mechanism rather than a learned force-conditioned controller. Incorporating force sensing and feedback into learned policies remains a separate research direction.

6.4 Shared Challenges in Efficiency and Reproducibility

The three studies place different demands on computation. The shape-morphing experiments in Chapter 3 report a CPU-parallel MPM implementation, Chapter 4 runs on one GPU and adds Sinkhorn transport solves, surface searches, rendering, and an adjoint through every optimization window, and the IntSim implementation in Chapter 5 uses GPU-based Taichi kernels for material and contact computations. A limitation of one implementation should therefore not be generalized to all three systems. Reproducible comparisons require separate reporting of simulation resolution, physical simulation duration, wall-clock runtime, optimization budget, and hardware.

Results can also depend on material parameters, contact thresholds, particle and grid resolution, sampling seeds, and optimizer schedules. Future evaluations should disclose these settings, distinguish numerical robustness from physically measured agreement, and report variation across multiple trials. In particular, visual quality, geometric task success, material separation, and measured interaction forces should remain distinct evaluation criteria rather than being combined into an unsupported claim of universal physical fidelity.

6.5 Future Research Directions

6.5.1 More Efficient and Stable Simulation-Based Optimization

A promising direction is to investigate implicit or mixed time integration, adaptive time stepping, and more effective control parameterizations for large-deformation morphing. Checkpointing, recomputation, and memory-aware adjoint methods could reduce the cost of long differentiable simulations. These improvements should be evaluated against both convergence quality and actual computational expense, rather than assuming that larger numerical time steps necessarily make inverse optimization faster.

6.5.2 Lasting Rendering Feedback and Finer Surface Control

For render-guided morphing, the next step is a render weight that keeps its authority where the picture has something to decide, such as a color layout, lettering or relief finer than the three-dimensional target, and still lets the body come to rest. Further directions are a symmetric link between neighboring steps of the layer offset that removes transient floating particles without moving the center of mass, a choice of the MPM cell that opens narrow gaps at an acceptable cost, a drag that damps the whole APIC momentum, and targets given only by pictures, where the concavities that only the images contain must be carved from a visual hull.

6.5.3 Measured Material Properties and Broader Robotic Validation

Future work should identify constitutive, contact, and damage parameters from controlled measurements across additional foods, tools, and cutting styles. Repeated real-robot trials should quantify force–time uncertainty and evaluate cluttered scenes, sequential cuts, and active object stabilization. Force-conditioned policies should be investigated as

a separate extension.

6.5.4 Bridging Simulation Objectives and Downstream Task Success

Across the three studies, an open question is how to connect numerical simulation quality to what ultimately matters for a particular application. In animation, this includes deformation control and rendered temporal coherence; in robotic manipulation, it includes correct geometry grounding, physical separation, and safe interaction. Benchmarks that report these outcomes alongside discretization error, material sensitivity, and computational cost would help determine when increased simulation fidelity produces meaningful improvements downstream.

6.6 Concluding Remarks

The dissertation demonstrates that MPM-based simulation can support physically directed shape transformations, differentiable rendering-guided control, and physics-grounded data generation for robotic cutting. Each application benefits from a common representation of deformable material motion, while requiring different optimization, rendering, or evaluation components. The remaining challenges concern both numerical fidelity and the connection between simulation outputs and real application outcomes. Addressing them will require more efficient optimization, more reliable surface and contact models, and broader validation against observed physical behavior.

Appendix A

MLS-MPM Details and Notation

Table A.1: **A table of variables used in our MPM simulation formulation.**

Symbol	Type	Meaning
ρ	scalar	density (kg/m^3)
λ	scalar	material parameter
μ	scalar	material parameter
ζ	scalar	damping coefficient
γ	scalar	smoothing factor
Δt	scalar	time step size
Δx	scalar	grid node spacing
$\mathbf{x}_p$	vector	position
m_p	scalar	mass
V_0	scalar	initial volume
$\mathbf{v}_p$	vector	velocity
$\mathbf{C}_p$	matrix	affine velocity field
$\mathbf{P}_p$	matrix	PK1 stress tensor
$\mathbf{R}$	matrix	rotation matrix
$\mathbf{F}_p$	matrix	deformation gradient
$\tilde{\mathbf{F}}_p$	matrix	control deformation gradient
$\mathbf{x}_i$	vector	position
m_i	scalar	mass
$\mathbf{v}_i$	vector	velocity
$\mathbf{p}_i$	vector	momentum
$\mathbf{f}_{\text{ext}}$	vector	external force
N	scalar	cubic B-spline function

[1]The subscripts index into the domain of the symbol, where p indexes into material points and i indexes into background grid nodes.

Here we describe our implementation of the MLS-MPM algorithm, as well as the minor modifications we made to the algorithm and the ensuing gradients. For an in-depth introduction to MPM, we refer the reader to the MPM course notes by Jiang et al. (2016). We also refer to Hu et al. (2019b), which derived the analytical gradients for the unmodified MLS-MPM algorithm.

The first modification we make to the standard algorithm is to introduce a damping coefficient ζ, which is helpful for running explicit MPM simulations with large timesteps. We used $\zeta = 0.5$ for all our examples. We incorporate this term when interpolating particle momenta to momentum on grid nodes, see Equation A6.

The second change we make is the introduction of a control deformation gradient variable $\tilde{\mathbf{F}}_p$. Section 3.2 discusses the details of using this term in the morphing algorithm.

The third change was implemented to create a smoother transition in the deformation gradient changes. This

was achieved by using a smoothing factor γ to apply a linear combination of the previous and next gradients, i.e. $\mathbf{F}^{n+1} = (1-\gamma)\mathbf{F}^{n+1} + \gamma\mathbf{F}^{n}$. This approach helped to eliminate oscillations in the simulation (see Section 3.4).

With these three changes, the modified MLS-MPM algorithm we use proceeds as follows:

P1. The first step of MPM is to compute each particle's Piola–Kirchhoff stress tensor, which is a function of the deformation gradient $\mathbf{F}$ and depends on the constitutive model. We use the fixed-corotated hyperelastic model (Stomakhin et al., 2012; Jiang et al., 2016), which makes use of the determinant J of $\mathbf{F}$ and the polar decomposition (Gast et al., 2016) of $\mathbf{F}$ into its rotation matrix $\mathbf{R}$ and its symmetric scaling matrix $\mathbf{S}$:

$$\mathbf{P}(\mathbf{F}) = 2\mu(\mathbf{F}-\mathbf{R}) + \lambda(J-1)J\mathbf{F}^{-T}. \tag{A1}$$

The total deformation gradient on a material point at a given timestep is the sum of the time-evolved deformation gradient $\mathbf{F}_p^n$ and the control parameter $\tilde{\mathbf{F}}_p^n$. In all instances where the time-evolved deformation gradient is used in the normal MPM algorithm, we instead use the sum of it and the control parameter $\tilde{\mathbf{F}}_p^n$. Using this sum allows the loss function to have identical gradients with respect to $\mathbf{F}_p^n$ and $\tilde{\mathbf{F}}_p^n$. It also continues to allow the deformation gradient $\mathbf{F}_p^n$ to time evolve rather than be overwritten by a control deformation gradient, which can be seen in the P2 step. Accordingly, we compute the per-particle stress by evaluating the prior equation at $\mathbf{F}_p^n + \tilde{\mathbf{F}}_p^n$, i.e., $\mathbf{P}_p^n = \mathbf{P}(\mathbf{F}_p^n + \tilde{\mathbf{F}}_p^n)$.

P2G. The second step of MPM is to interpolate particle mass and momentum onto the grid nodes, which is the precursor step to calculating updated grid node velocities. We use APIC (Jiang et al., 2015, 2017) for this transfer, which requires an affine velocity field $\mathbf{C}_p$ stored as a matrix on each of our material points. For ease of exposition, we define helper variables $\mathbf{d}_{ip}^n$, w_{ip}^n, and $\mathbf{G}_p^n$ used in the transfers:

$$\mathbf{d}_{ip}^n = \mathbf{x}_i - \mathbf{x}_p^n, \tag{A2}$$

$$w_{ip}^n = N(\mathbf{d}_{ip}^n), \tag{A3}$$

$$\mathbf{G}_p^n = \frac{-3}{\Delta x^2}\Delta t V_p^0 \mathbf{P}\left(\mathbf{F}_p^n + \tilde{\mathbf{F}}_p^n\right)^T + m_p \mathbf{C}_p^n, \tag{A4}$$

where Equation A4 uses the moving least squares (MLS) force discretization (Hu et al., 2018, 2019b). With these variables, we may express the interpolated grid node masses as

$$m_i^n = \sum_p w_{ip} m_p, \tag{A5}$$

and the interpolated grid node momenta as

$$\mathbf{p}_i^n = \sum_p w_{ip}^n (m_p \mathbf{v}_p^n \cdot (1-\zeta\Delta t) + \mathbf{G}_p^n \mathbf{d}_{ip}^n). \tag{A6}$$

G. The third step of MPM is to calculate nodal velocities by dividing the nodal momentum by the nodal mass. Accelerations from external forces such as gravity are also applied here:

$$\mathbf{v}_i^n = \frac{1}{m_i^n}(\mathbf{p}_i^n + \mathbf{f}_{\text{ext}}\Delta t). \tag{A7}$$

G2P. In the previous step, the velocity of the grid nodes were calculated by interpolating nearby material point properties and then updated by external forces such as gravity. Updated velocities are then interpolated back to nearby material points:

$$\mathbf{v}_p^{n+1} = \sum_i w_{ip}^n \mathbf{v}_i^n. \tag{A8}$$

Updated material point affine velocity matrices (Jiang et al., 2015, 2017) must also be computed from the grid nodes. The equation we provide is specific to using cubic B-spline interpolation functions:

$$\mathbf{C}_p^{n+1} = \frac{3}{\Delta x^2}\sum_i w_{ip}^n \mathbf{v}_i^n (\mathbf{d}_{ip}^n)^T. \tag{A9}$$

P2. The final step of an MPM timestep is to update the material point positions and deformation gradients, where we use a smoothing factor γ to control the deformation update:

$$\mathbf{x}_p^{n+1} = \mathbf{x}_p^n + \Delta t \mathbf{v}_p^{n+1} \tag{A10}$$

$$\mathbf{F}_p^{n+\frac{1}{2}} = (\mathbf{I} + \Delta t \mathbf{C}_p^{n+1})(\mathbf{F}_p^n + \tilde{\mathbf{F}}_p^{n+1}) \tag{A11}$$

$$\beta = u(||\mathbf{F}_p^{n+\frac{1}{2}} - \mathbf{F}_p^n|| - \gamma) \tag{A12}$$

$$\mathbf{F}_p^{n+1} = (1-\beta)\mathbf{F}_p^{n+\frac{1}{2}} + \beta \mathbf{F}_p^n \tag{A13}$$

where u is the unit step function. When $\beta = 1$, i.e. when the deformation change exceeds our allowed tolerance γ, the deformation update is equivalent to overwriting the control deformation gradient $\tilde{\mathbf{F}}_p^{n+1}$ such that $\tilde{\mathbf{F}}_p^{n+1} = (\mathbf{I} + \Delta t \mathbf{C}_p^{n+1})^{-1}(\mathbf{F}_p^n - (\mathbf{I} + \Delta t \mathbf{C}_p^{n+1})\mathbf{F}_p^n)$.

Appendix B

Render-Guided Morphing Details

This appendix collects the constants, per-step updates, term formulas, selection rules and longer measurements behind Chapter 4.

B.1 The Simulated Body

Discretization. The source and the target mesh are each sampled into N particles by stratified volume sampling, one jittered particle per voxel of the sampler's fill, and the source sample is rescaled so that its fill volume equals the target sample's. Every particle has the same dynamic mass $m_p = \bar{m}/N$. The MPM cell Δx is the source's bounding-box diagonal divided by 26, which puts about 23 particles in a cell at $N = 40{,}000$ and about 183 at $N = 300{,}000$; the time step is $\Delta t = 1/240\,\mathrm{s}$. The elastic response is fixed corotated (Stomakhin et al., 2012, 2013),

$$\mathbf{P}(\mathbf{F}) = 2\mu\,(\mathbf{F}-\mathbf{R}) + \lambda_L\,(J-1)\,J\,\mathbf{F}^{-\top}, \qquad J = \det\mathbf{F}, \tag{B.1}$$

with $\mathbf{R}$ the rotation of the polar decomposition, Lamé parameters μ, λ_L from $E = 1.4\times10^5$ and $\nu = 0.2$, and J floored at 10^{-6}.

One timestep. Each particle carries its position $\mathbf{x}_p$, velocity $\mathbf{v}_p$, APIC affine matrix $\mathbf{C}_p$, deformation gradient $\mathbf{F}_p$, plastic deformation gradient $\mathbf{F}_p^{\mathrm{P}}$, carried volume ratio J_p and rest volume V_p^0. With the Kirchhoff stress of Eq. (4.1), the particle-to-grid transfer of MLS-MPM (Hu et al., 2018) with APIC (Jiang et al., 2015) reads

$$m_i\mathbf{v}_i = \sum_p w_{ip}\Big[m_p(1-\kappa\Delta t)\,\mathbf{v}_p + \Big(m_p\mathbf{C}_p - \tfrac{3}{\Delta x^2}\Delta t\,V_p^0\,\tau_p\Big)(\mathbf{x}_i-\mathbf{x}_p)\Big], \qquad m_i = \sum_p w_{ip}m_p, \tag{B.2}$$

where w_{ip} is the cubic B-spline weight and $\kappa = 0.9\,\mathrm{s}^{-1}$ a weak ambient drag. After the grid update (external forces and the walls of the domain, when present), the grid-to-particle transfer gathers $\mathbf{v}_p = \sum_i w_{ip}\mathbf{v}_i$ and $\mathbf{C}_p = \frac{3}{\Delta x^2}\sum_i w_{ip}\mathbf{v}_i(\mathbf{x}_i - \mathbf{x}_p)^{\top}$.

Deformation and volume. The deformation gradient is advected with the control included and then relaxed toward its previous value with the smoothing factor of Chapter 3, $\hat{\mathbf{F}}_p = (\mathbf{I}+\Delta t\,\mathbf{C}_p)(\mathbf{F}_p + \delta\mathbf{F}_{c,p})$ and $\bar{\mathbf{F}}_p = (1-\gamma)\,\hat{\mathbf{F}}_p + \gamma\mathbf{F}_p$ with $\gamma = 0.955$. This keeps 4.5 % of each step's increment; applied to the volume as well, it lets thin parts in transit stream out as a dilated sheet whose stress reads almost no volume change. The motion's own volume ratio is therefore carried separately, and the smoothed gradient is rescaled to it after every step,

$$J_p \leftarrow J_p\det(\mathbf{I}+\Delta t\,\mathbf{C}_p), \qquad \mathbf{F}_p \leftarrow \big(J_p/\det\bar{\mathbf{F}}_p\big)^{1/3}\bar{\mathbf{F}}_p, \tag{B.3}$$

so that $\det \mathbf{F}_p = J_p$ while the isochoric part keeps the smoothing's shape.

Plastic assimilation. When a window is committed, with $\mathbf{F}^{\mathrm{E}} = \mathbf{R}_e \mathbf{S}_e$, $\mathbf{F}^{\mathrm{P}} \leftarrow \mathbf{S}_e^{\eta} \mathbf{F}^{\mathrm{P}}$ with $\eta = 0.5$, and every principal stretch of $\mathbf{F}^{\mathrm{P}}$ is kept in $[0.2, 5]$. The assimilation takes the volumetric part of the stretch as well.

B.1.1 Position Updates below the Grid

Each update is spread over the steps of a window like a constraint projection.

- **Minimum spacing.** Each particle is moved away from its neighbours nearer than $r_{\min} = 0.9\,a$ by half the overlap over one window: $\mathbf{x}_p \mathrel{+}= \frac{1}{2T} \sum_{q \in \mathcal{N}_p} \max(0, r_{\min} - \ell_{pq})\,(\mathbf{x}_p - \mathbf{x}_q)/\ell_{pq}$, $\ell_{pq} = \|\mathbf{x}_p - \mathbf{x}_q\|$. The neighbour lists are the symmetric closure of each particle's 16 nearest neighbours at the window's start, so every push is pairwise, equal and opposite, and central.

- **Outer-layer relaxation.** The outer layer (particles whose offset from the centroid of their 32 nearest neighbours exceeds half a spacing) is relaxed toward the plane of its same-side neighbours. With $d_p = \mathbf{n}_p \cdot (\mathbf{x}_p - \sum_q w_{pq} \mathbf{x}_q)$ the plane residual over 24 same-side neighbours and $\bar{d}_p = \sum_q w_{pq} d_q$, each driven step moves $s_p = -\frac{1}{T}\,(d_p - \bar{d}_p - r_p)$ along $\mathbf{n}_p$. The reference r_p is the same operator applied to the feet of the layer particles on the target mesh (for particles within one spacing of it). The step's components along the six rigid modes of the layer are removed, giving the step s'_p of Eq. (4.2).

- **Fragments.** A particle with no other particle in the 3^3 cells around it takes the mean velocity of its eight source neighbours and is pulled back toward their rest distances over one window.

The relaxation acts only in the driven half of a window; in the released half the only position updates are the minimum spacing and the fragments' bonds.

B.2 Objective Terms

Transport. The body's mass and the target sample's are rasterized on a loss grid, and

$$\begin{aligned} \mathscr{L}_{\mathrm{OT}} = S_\varepsilon(\alpha, \beta) = \mathrm{OT}_\varepsilon(\alpha, \beta) - \tfrac{1}{2}\mathrm{OT}_\varepsilon(\alpha, \alpha) - \tfrac{1}{2}\mathrm{OT}_\varepsilon(\beta, \beta), \\ \mathrm{OT}_\varepsilon(\alpha, \beta) = \min_{\pi \in \Pi(\alpha, \beta)} \langle \mathbf{c}, \pi \rangle + \varepsilon\, \mathrm{KL}(\pi \,\|\, \alpha \otimes \beta), \end{aligned} \tag{B.4}$$

the debiased Sinkhorn divergence (Feydy et al., 2019) of entropic optimal transport (Cuturi, 2013), with the squared Euclidean cost and blur $\varepsilon = \Delta x_\ell^2$, one loss cell. The loss cell equals the MPM cell up to $N = 40{,}000$ and follows the particle spacing above it, $\Delta x_\ell = \Delta x\,(40{,}000/N)^{1/3}$. The problem is solved in the log domain with ε-scaling from the squared grid diameter down to the blur; every sweep is three one-dimensional log-sum-exp passes on the GPU, and the gradient is the converged potentials differentiated through the rasterization weights.

Surface proximity.

$$\mathscr{L}_{\text{prox}} = \rho^2 \operatorname*{mean}_{\mathbf{y}} \max\Big(0,\, 1 - \frac{K(d_{\min}(\mathbf{y}))}{\frac{1}{2}K(\varsigma)}\Big)^2, \qquad K(d) = e^{-d^2/2h^2}, \tag{B.5}$$

over the outer points $\mathbf{y}$ of the target sample, where $d_{\min}(\mathbf{y})$ is the distance to the nearest body particle, ρ the target sample's median distance to its 8th neighbour, $h = \rho/2$, and ς its median nearest-neighbour spacing. The term is zero where a particle sits within about 1.5 spacings of $\mathbf{y}$; it has no weight and no bound.

Settling.

$$\mathscr{L}_{\text{drift}} = (T\Delta t)^2 \operatorname*{mean}_{p} \|\mathbf{v}_p(t_{2T})\|^2, \qquad \mathscr{L}_{\text{rel}} = (T\Delta t)^2 \operatorname*{mean}_{t\in\text{released},\,p} \|\mathbf{v}_p(t)\|^2. \tag{B.6}$$

Without the released motion, several gallery meshes stopped after 8–11 windows while still moving; without the end drift, runs were sound but their released end moved 1.4–2.9 times faster. Regularizers of the control's magnitude and smoothness were 10^{-10} and 10^{-8} of the objective, and removing them together with a volume prior changed none of the measured quantities ($\det \mathbf{F}$, anisotropy, control and surface roughness).

B.2.1 Cleanup Terms

The *spray pull* is a one-sided Wasserstein-1 pull down the target's distance field $d_{\mathscr{T}}$,

$$\mathscr{L}_{\text{spray}} = \sum_p g_p\, m_p\, d_{\mathscr{T}}(\mathbf{x}_p), \tag{B.7}$$

on the particles an isolation gate g_p marks (a ramp of the distance to the 8th neighbour from 1.2 to 1.8 times its median, frozen per window). The *near band* pulls each particle that lies between the sampling berth b (about two target spacings) and one loss cell from the target toward its nearest target point,

$$\mathscr{L}_{\text{near}} = \sum_p e_p\, m_p \max\big(0, \|\mathbf{x}_p - \mathbf{y}_{a(p)}\| - b\big). \tag{B.8}$$

Both carry the weight $w_c = 0.2$ in the unit set by σ (Eq. (4.3)), and $w_c = 0$ removes them. Their effect was measured by leaving each out of the objective while it was being fixed, on the 40k gallery (two runs per arm). Without the spray pull, the silhouette and the uncovered thin points were unchanged (median differences 0.0000 and $+0.11$ points), but the median share of isolated particles rose from 0.05–0.07 % to 0.10 % and the runs took 48–52 windows instead of 38–44. Without the near band, the uncovered thin target points rose from 7.1 to 10.4 % on the heart and from 7.5 to 10.5 % on spot, and C stopped after 19–20 windows, still moving. At 300k particles the band's far part pulled the whole body toward its nearest target points against the transport's assignment, which is why the band ends one loss cell from the target. These tests were made before the final volume handling and the exterior render terms.

B.2.2 Render Terms

The exterior is the zero set of Zhu and Bridson's field (Zhu and Bridson, 2005),

$$\phi(\mathbf{q}) = \|\mathbf{q} - \bar{\mathbf{x}}(\mathbf{q})\| - r_o, \qquad \bar{\mathbf{x}}(\mathbf{q}) = \frac{\sum_p k_p(\mathbf{q})\,\mathbf{x}_p}{\sum_p k_p(\mathbf{q})}, \quad k_p(\mathbf{q}) = \max\Big(0, 1 - \tfrac{\|\mathbf{q}-\mathbf{x}_p\|^2}{R^2}\Big)^3, \tag{B.9}$$

with kernel radius $R = 3a$ and offset $r_o = 0.8a$. Surface discs $(\mathbf{p}_j, \mathbf{n}_j)$ sit one to each cell of a lattice fixed in space that the zero set crosses (the cell centre projected onto the zero set), at a spacing of $\min(\text{half a picture pixel}, 0.92a)$. They follow the particles between searches by one Newton step of the field and are found again when a tenth of them has moved more than half a lattice spacing. The pictures are 18 orthographic views (six azimuths at elevations 0 and ± 0.5 rad) at $96\,(N/40{,}000)^{1/3}$ px (188 px at 300k). The silhouette splats the discs with opacity $\alpha = 1 - e^{-kw}$ ($k = 1.5$, w the bilinear coverage),

$$\mathscr{L}_{\text{sil}} = \operatorname*{mean}_{\text{views, pixels}} \Big[w_h \max(0, \alpha^* - \alpha)^2 + w_s \max(0, \alpha - \alpha^*)^2 \Big], \qquad w_h = 2,\ w_s = 1, \tag{B.10}$$

and the shading term compares headlight-Lambertian images, $b = 0.25 + 0.75\max(0, \mathbf{n}_j \cdot \mathbf{l})$, each disc weighted by its facing cosine and a soft front bias and masked by the coverage: $\mathscr{L}_{\text{shade}} = \text{mean}(s - s^*)^2$. The display uses the field with Solenthaler et al.'s correction (Solenthaler et al., 2007), which moves 0.4–5.2 % of the discs, by a median 10^{-4} pitches, and removes blobs in the empty space between near surfaces; every metric reads the uncorrected field or the particles. For the figures, the display's discs are split into connected pieces (discs within two lattice spacings are joined), each particle is counted with the piece of its nearest disc, and a piece that holds fewer than ten particles, the size this chapter calls a serious piece, is not drawn. The rule changes the display only. At the times shown in Figure 4.11 it removes only specks of a few particles, and no separate piece of ten or more particles lies off the body there.

B.3 Window, Step and Selection

A window has $T = 20$ driven and $T = 20$ released steps ($1/6$ s), and the optimizer runs eight iterations per window; a 300k bunny takes 76–106 committed windows, a dragon 139–186. The physics, cleanup and render gradients are taken through the same tape, captured as CUDA graphs; finite differences agree with the adjoint within 3 % for every term at windows 1 and 20 of a 300k run. The control $\delta\mathbf{F}_c$ is capped at $\|\delta\mathbf{F}^n_{c,p}\|_F \le 0.02$ per particle and step, and a window starts from half of the previous window's control, kept only when it gives a lower objective than starting from zero. A trial step is accepted only when the full objective decreases by the Armijo amount (or by more than the rollout's replay noise, measured at each window's start) and the trajectory stays finite, without inverted $\mathbf{F}$ and inside the domain. The Adam moments of $\delta\mathbf{F}_c$ start afresh in every window; those of u carry from one committed window

to the next, each coordinate's bias correction counting its own age, because restarted every window u's first step is full-sized whatever its gradient and reverses from window to window.

A window's result is committed by replaying the accepted candidate on the exterior discs it was scored with, the plastic assimilation is applied, and the committed state is scored by one selection merit, the objective at that state. A window that raises the merit by more than 5 % is rejected and the previous state kept; because the merit is linear in λ, the references are rescored whenever λ changes. The run ends after three consecutive rejections or five windows without a relative improvement of 0.3 %, and delivers the trajectory up to its best window.

Over whole 300k morphs the centre of mass moves by 1.6–6.9 $\times 10^{-5}$ pitches at most, the float32 floor of the grid transfers, and the body's physical volume $\sum_p J_p V_p^0$ stays within -0.8 % and $+3.1$ % of the target's over the run and ends 0.1–0.4 % above it (bunny and dragon). The drag damps $m_p \mathbf{v}_p$ only, not APIC's affine momentum, so it can trade angular momentum between the orbital and affine parts.

B.4 Multi-Body Scenes

The scene path extends the single-body window with four definitions and keeps every per-body term unchanged.

- **One grid field per body.** Each body deposits into its own grid field. At a node where both fields have mass, the bodies are in contact when their node volumes fill the cell, $V_0 + V_1 \geq \Delta x^3$, and approach, $(\mathbf{v}_0 - \mathbf{v}_{cm}) \cdot \mathbf{n} > 0$, with $\mathbf{n}$ the normalized difference of the fields' mass gradients. Body 0 then loses its normal velocity relative to $\mathbf{v}_{cm}$ with Coulomb friction ($\mu = 0.5$), and body 1 takes the opposite change weighted by the masses, following Bardenhagen et al. (2000). The changes are equal and opposite at each node, so linear and angular momentum are conserved; the floor is the same rule with a body of infinite mass.

- **A body frame.** Each body's objective is evaluated in $\mathbf{y} = \mathbf{R}^\top(\mathbf{x} - \mathbf{c})$ with $\mathbf{c}$ the centroid and $\mathbf{R} = \text{polar}\left(\sum_p (\mathbf{x}_p - \mathbf{c})(\mathbf{X}_p - \bar{\mathbf{X}})^\top\right)$, $\mathbf{X}$ the body's source sample. The polar factor is differentiated exactly, so the position gradient carries no net force or torque. The cameras ride with the body, and the velocity terms read the deformation velocity $\mathbf{v} - (\mathbf{v}_c + \boldsymbol{\omega} \times \mathbf{r})$.

- **Time always advances.** Every window commits its line-searched control, or the uncontrolled continuation when no step was accepted. A window whose replay does not reproduce its accepted candidate is solved again; each body's loss grid grows to cover its reach under a landing; a body whose own terms are not finite at a window's start is left out of that window. The ambient drag is replaced by a damping of each body's deformation velocity alone.

- **A moving target.** Each body's target at time t is $\mathbf{Z}_s = \mathbf{Y} + (1 - s)\mathbf{D}_Y(\mathbf{Y})$ with $s = 3\tau^2 - 2\tau^3$, $\tau = t/t_m$, the displacement interpolation of its transport read from the target's side (McCann, 1997).

For gravity with one body (Section 4.7), the same path is used with the target standing on the floor, fixed in the world.

Table B.1: **Wall-clock time of a 300k morph on one RTX 6000 Ada.** The full method, ranges over seeds, with the median number of committed windows.

mesh	minutes	windows
bunny	8.8–11.3	87
dragon	23.3–28.7	165
teapot	12.6–13.7	123
bob	15.5–16.1	130
armadillo	12.3–14.8	90
homer	23.4–33.8	197

B.5 Implementation and Runtime

The simulation and its adjoint are written in NVIDIA Warp (Macklin, 2022). The run stage is GPU-only: particle states stay on the device between windows, and neighbour queries and grid morphology run on the GPU through a device KD-tree; only the preparation (mesh loading and sampling) runs on the CPU and is cached. The P2G transfer visits particles in cell order so that the 32 lanes of a warp share a stencil and add each node once, which removed most of the same-address atomic contention at 183 particles per cell. Two further exact speed-ups skip the layer kernels in steps where neither the relaxation nor u acts and reuse the proximity term's neighbour lists within an iteration; together they save 4.4–6.1 % a window and leave every result bit for bit identical. The deliverable videos are rendered at 3840×2160, 20 frames per second, from every 12th simulated step.

The card's peak memory is 19.3–23.9 GB. An optimizer attempt takes 6.5 s on the bunny and 8.8 s on the dragon; early in a dragon run the Sinkhorn solves take 32 % of an attempt, the surface proximity 17 % and the three adjoint sweeps 18 %, late the sweeps 28 % and the Sinkhorn solves 21 %. The render's silhouette term reaches the value that an independent sample of the mesh scores against the run's pictures at 2.2–12.0 s in 18 of 18 runs, and 54–90 % of the windows come after it. Over that tail the end measures keep improving: a stop at that point would give up 11–48 % of the exterior silhouette residual and 20–42 % of its shading on the bunny and the dragon. The last half of the wall time improves the end measures by a median 5–14 %, the last quarter by 1–4 %, and the last tenth (0.9–3.4 min) by nothing measurable (median 0 %, at most ± 4 %).

B.6 Measures

Every morph of the main evaluation runs at seeds 97, 99 and 101, and the bunny and dragon also at 103 and 105. The dragon has genus 15, the teapot and bob genus 1.

- *Against the exact mesh.* The referee is the exact target mesh in the run's own frame, grown by c, the mean offset of the own target sample's exterior from the mesh ($c = 0.67$–0.95 pitch by mesh); the raw mesh is read beside it. Silhouettes are ray-cast at 1024 px on 46 held-out views (a 64-point Fibonacci sphere without the directions within 12° of a training view or its opposite), 13 of them farther than 25° from every training line. Silhouette metrics: Boundary IoU (Cheng et al., 2021) at 2 % of the image diagonal, the boundary F-score at

the DAVIS tolerance (Perazzi et al., 2016), the 2D contour Chamfer distance and HD95 (pitches). 3D metrics on the visible exterior, 200k area-uniform points on each surface: F-score at $\tau = 0.5$ and 1 pitch (Knapitsch et al., 2017; Tatarchenko et al., 2019), Chamfer-L1 and normal consistency (Mescheder et al., 2019). Images: the 46 held-out views as studio cameras at 512 px with the deliverable's shading model, shaded images and camera-space normal maps, by PSNR, SSIM (Wang et al., 2004) and LPIPS (Zhang et al., 2018).

- *The rule.* Per mesh-seed pair, $r = (e_{\text{twin}} - e_{\text{full}})/e_{\text{twin}}$ (the error is $1-$score for IoU, F-score, normal consistency and SSIM, the distance or LPIPS otherwise). A metric sees the render when a Wilcoxon signed-rank test over the pairs gives $p < 0.01$, the per-mesh median r has one sign on at least 5 of 6 (300k) or 16 of 19 (40k) meshes, and $|r| \geq 5\,\%$.

- *Particle level.* The particle silhouette IoU over 24 views at 128 px (a binary 3×3-pixel point splat), the symmetric chamfer distance, and the thin coverage: the share of the target's thin outer points (local thickness below two MPM cells) with no particle within 1.5 spacings, against the run's own sample. Everywhere else, the reference is an independent stratified sample drawn with a seed no run uses.

- *Render operators.* The exterior silhouette and shading residuals of the end state against the independent sample's exterior (18 views at 96 px), and the displayed surface's $1 -$ IoU at 4K, front and back.

- *Physical.* At rest: the mean squared particle speed at the last committed window's released end. Momentum: the largest displacement of the centre of mass and the net rotation over all kept frames. Floating pieces: sets of particles not linked to the body within 1.5 pitches; *serious* if at least ten particles, at least five pitches from the body, and persisting.

Elsewhere, two arms differ on a measure only when their seed ranges do not overlap, and changes are median against median.

B.7 Additional Results

B.7.1 The Render's Effect

During the arrival. On the outer layer of the 300k bunny and dragon, the full method's surface departs from its twin's at 0.5–1 s by 0.36 pitch on the bunny (seed-to-seed floor 0.11–0.26) and 0.82–0.85 pitch on the dragon (floor 0.17–0.29); after arrival both arms are at the floor. At 40k the render carries 5–10 % of the outer layer's motion in the first second, which holds 66–99 % of all the motion. After arrival the physics gradient has fallen 20–40-fold and the step 20–70-fold. Late in a run the render's displacement of the end state is 75–77 % on the outer layer, and a window without the render term gives back 67–81 % of the previous window's render-driven displacement; only 5–9 % of the render's demand is at the cell scale, which $\delta\mathbf{F}_c$ reaches, while u reaches its wanted move at a cosine of 0.79–0.81.

Table B.2: **The six 300k meshes at the end, full method / twin.** Left, against the exact mesh on held-out views (seed ranges); right, particle-level readings (medians).

mesh	Boundary IoU error (10^{-2})	F-score at 0.5 pitch	held-out PSNR (dB)	silhouette IoU	thin uncovered (%)	at rest (10^{-6})
bunny	5.22–5.49 / 6.58–7.20	0.898–0.910 / 0.838–0.859	36.64–36.89 / 35.51–35.94	0.9873 / 0.9863	3.97 / 3.85	1.31 / 1.51
dragon	7.79–8.09 / 8.70–9.36	0.847–0.856 / 0.808–0.827	34.06–34.15 / 33.40–33.60	0.9874 / 0.9874	3.26 / 3.23	7.67 / 7.64
teapot	6.81–7.34 / 8.01–8.20	0.812–0.835 / 0.795–0.807	34.79–35.25 / 34.50–34.58	0.9899 / 0.9892	2.42 / 3.28	0.35 / 0.82
bob	4.36–4.71 / 5.47–5.61	0.973–0.982 / 0.948–0.957	37.89–38.19 / 36.74–36.92	0.9895 / 0.9886	3.83 / 4.29	0.04 / 0.10
armadillo	4.93–5.05 / 6.37–6.71	0.878–0.892 / 0.806–0.820	35.87–35.96 / 34.98–35.29	0.9864 / 0.9861	4.09 / 3.79	13.2 / 10.3
homer	5.02–6.43 / 6.67–6.94	0.921–0.931 / 0.868–0.884	38.43–38.67 / 37.53–37.70	0.9882 / 0.9880	4.04 / 4.90	1.95 / 2.82

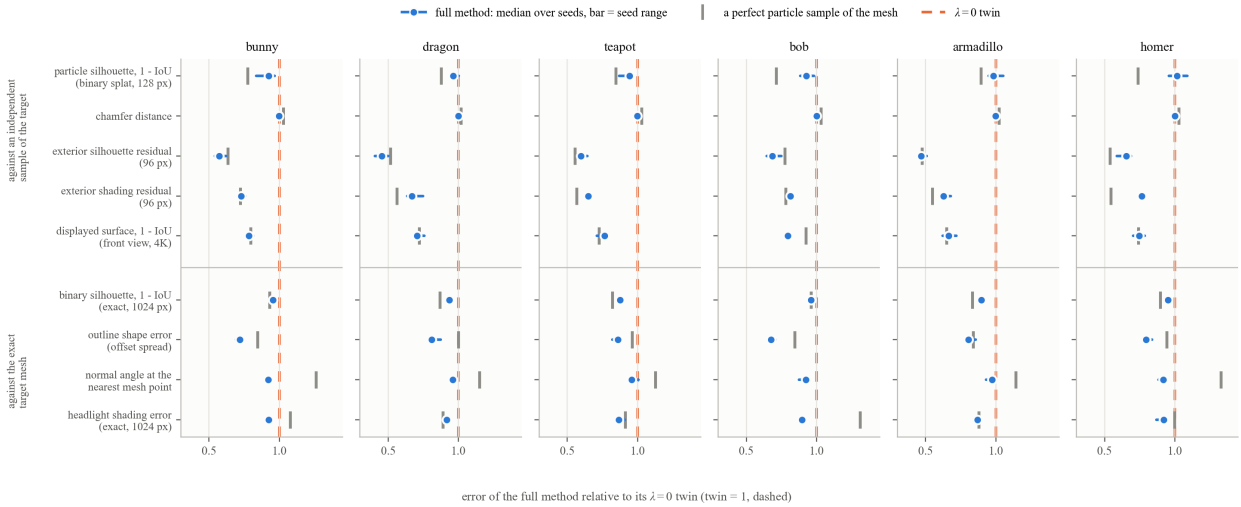


Figure B.1: **The end of the run, full method against its $\lambda = 0$ twin on six meshes.** The render terms' own residuals move the most, the exact-mesh readings less, and the particle IoU and chamfer hardly at all.

The causal switch. Switched off at 6 s, the full method loses half of its lead on the exterior silhouette within 4–6 windows and 63–92 % of its lead within ten; switched on, the twin builds half the lead within 1–2 windows and ends at 1.00–1.07 times the exterior silhouette residual of the run that had the render throughout. The switch moves the centre of mass by 2.6×10^{-5}–1.5×10^{-4} pitches.

Per mesh. Table B.2 gives the end of the six 300k meshes. In absolute terms at 300k, the Boundary IoU error is 0.054 (full method), 0.068 (twin) and 0.052 (a perfect sample); the F-score at half a pitch 0.895, 0.832 and 0.940; and the full method closes 53–90 % of the twin's gap to a perfect sample on the contour metrics and the F-score at half a pitch. On the 18 training views, the 46 held-out views and the 13 far views the Boundary IoU error falls by 22, 16 and 15 %. Against the mesh, the recall of thin parts within one pitch improves by 9.9 %. Bob's net rotation rises from 0.0009° to 0.0044°, and homer's runs are 34 % longer with the render term.

The particle silhouette. The binary particle silhouette resolves about 1.8 pitches per pixel at 128 px, and 72–89 % of the twin's error there is the sampling floor itself. Moving the full method's outer layer uniformly by 0.1 pitch along its normal changes $1 - \text{IoU}$ by 0.0008–0.0025, as much as the render term's whole effect, 0.0005–0.0021 (Figure B.2). Against the raw mesh, 78–80 % of the exact mask-IoU error at 300k is the exterior's shared offset.

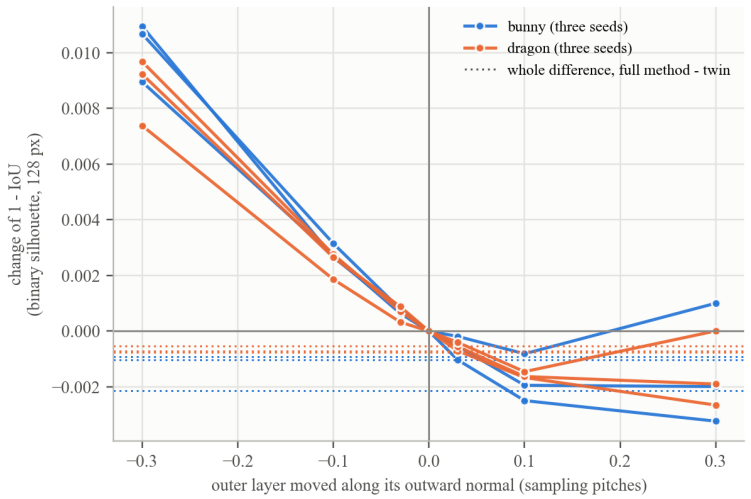


Figure B.2: **Sensitivity of the particle silhouette IoU.** A uniform shift of a tenth of a pitch moves it as much as the render term's entire effect (dotted).

Arrangement. Read on the run's own target pictures, the full method's render term at the end of the 300k runs is lower than that of each of eight uniform samples of the true mesh on all five seeds of the bunny and the dragon: 0.46–0.58 (bunny) and 0.69–0.79 (dragon) times their mean, against 1.4–1.9 times for the twin. The part of the full method's lead below that floor is 43–56 % on the bunny and 21–26 % on the dragon, mostly in the silhouette term.

B.7.2 The 40k Gallery

On 19 meshes at three seeds, the full method's particle silhouette IoU is above its twin's at the three-seed mean on 18 of 19 meshes (bob is level), with a paired Wilcoxon test over the meshes giving $p = 1.1 \times 10^{-5}$ and a median gain of $+0.0007$. Against the exact meshes the outline deviates 3–15 % less than the twin's on all 19 meshes (56 of 57 pairs; median -10 %, Figure B.3), and the normals come about 2 % closer; the binary silhouette error against the raw mesh is about 1 % higher, because the exterior sits 0.006 pitch farther out.

The lean comes from the silhouette term's 2:1 weighting. The target pictures' edge is 0.44–0.50 pitch wide for the eight-draw mean against 0.27–0.33 for one draw; the asymmetry shifts the term's optimum outward on every state read (81 of 81 at 40k, 48 of 48 at 300k) by an amount set by the pitch, not the pixel. At 300k the twin's exterior lies outside the target pictures and the term's net demand is inward, -0.037 pitch, which the full method follows (22 of 22 pairs); at 40k the twin sits near the symmetric optimum ($+0.005$), and the full method's outline moves out by $+0.008$ pitch within a tenth of a render pixel of a training outline (43 of 57 pairs, $p = 2.8 \times 10^{-5}$), about a fifth of the term's uniform optimum, following the twin's demand across mesh-seeds (Spearman 0.55). The pre-registered rule leaves the explanation undecided because the rim-against-head-on contrast reached $p = 0.020$; the next measurement is the gallery with equal weights.

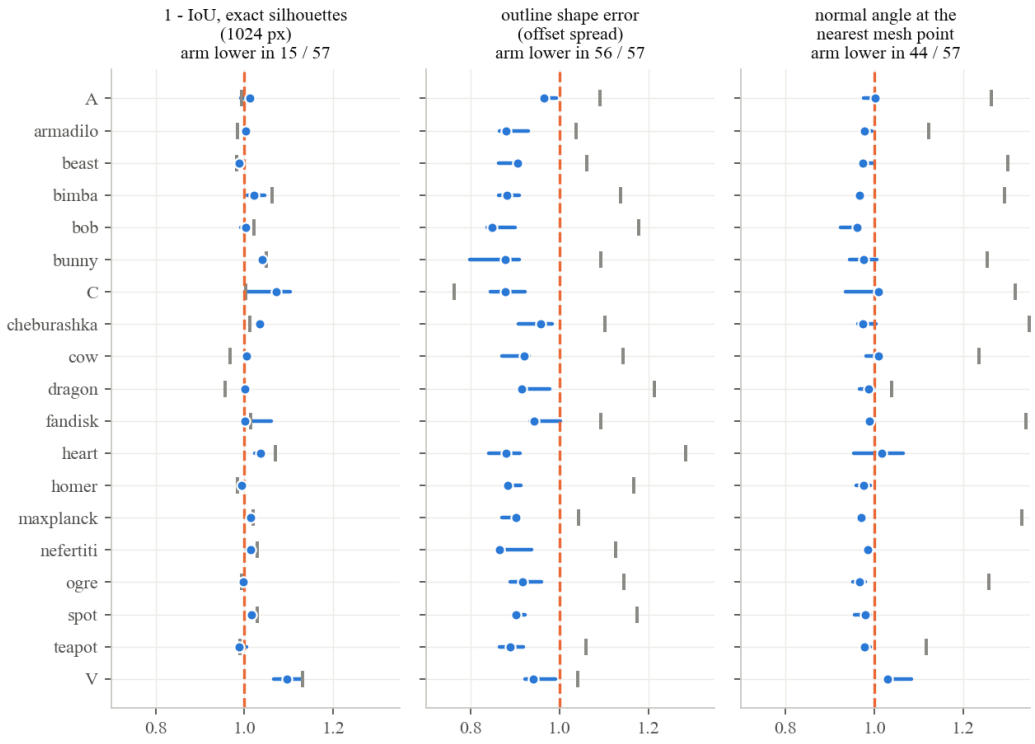


Figure B.3: **The 40k gallery against the exact meshes, full method relative to its twin ("arm" in the panels).** The outline improves on all 19 meshes; the silhouette's area does not.

B.7.3 The 300k Gallery

Against an independent sample, the 13 new targets reached from a sphere other than the letter C end at a particle silhouette IoU of 0.9838–0.9901 and the six shape pairs at 0.9832–0.9888; no run has a piece of ten or more particles five pitches from the body at its end, and every end but the letter C's has an end kinetic measure of at most 1.3×10^{-5}. A run to a new target takes 7.8–27.8 min and a shape pair 12.3–32.4 min. Where twins were run, the full method's particle silhouette error is 2–8 % lower and its exterior silhouette residual 30–58 % lower. A voxel reading of the Happy Buddha's end counts seven tunnels against the target's six, and bunny $\rightarrow$ Happy Buddha opens four of the six.

B.7.4 Materials and Gravity

Every material reaches its target at 300k (particle IoU 0.9863–0.9876): a soft solid ($E \times 0.1$) with a transport energy at 2 s 55–178 times the default's, a stiff one ($E \times 7$) with 2.2–3.5 times the default's end kinetic measure, and $\nu = 0.45$ at a cost of 9–21 % on some surface and thin measures; the render's lead on the exterior silhouette is -29 to -57 %. Under gravity (0.65 world units s^{-2}, $G = \rho g H / E \approx 0.02$) the single-body path stops two of three bunny seeds and the twin after seven windows, because the landing raises the selection merit by 7–9 %. Through the scene path with a standing target, both meshes reach on three of three seeds (particle IoU 0.9866–0.9875 bunny, 0.9871–0.9879 dragon,

Table B.3: **Ablations at 300k (bunny / dragon, three seeds).** Changes against the full method; all with seed ranges apart unless marked "within".

part switched off	what changes
layer offset u (render loss kept)	exterior silhouette $\times 3.0$ / $\times 7.7$, shading $\times 2.2$ / $\times 4.0$, displayed surface $\times 2.0$ / $\times 3.4$, thin $+26\,\%$ / $\times 3.0$; never at rest (end kinetic $\times 54$ / $\times 53$); early floating particles -99 / $-98\,\%$
relief reference of the relaxation	dragon: exterior silhouette $+16\,\%$, shading $+20\,\%$; bunny thin $-23\,\%$
volume-conserving body	early floating particles $\times 5.0$ / $\times 3.4$, end kinetic $+79$ / $+69\,\%$, dragon rotation $\times 2.0$; distant pieces of up to 42 particles; thin -25 / $-21\,\%$
carried volume alone	early floating particles $\times 5.1$ / $\times 3.5$
volumetric assimilation alone	not at rest: end kinetic $\times 36$ / $\times 23$
density match alone	within the full method's range
render on the exterior (read on the cloud instead)	exterior silhouette $+28$ / $+44\,\%$, shading $+21$ / $+19\,\%$, displayed surface $+27$ / $+20\,\%$; particle IoU and thin within
render weight set every window (a moving average instead)	render share 0.42–0.80 in early windows; end within the range but for small shading and thin changes
minimum spacing	surface roughness $\times 2.1$, thin $\times 2.9$ / $\times 3.8$, exterior silhouette $+37$ / $+41\,\%$, visible pieces $\times 6.9$ / $\times 7.6$
relaxation in the driven half only (every step instead)	late wobble returns ($\times 2.1$ / $\times 1.8$); thin $+14$ / $+30\,\%$; runs end sooner
carried moments of u (fresh every window instead)	window-to-window reversal returns; visible pieces $\times 2.0$ / $\times 2.3$
symmetric minimum-spacing pairs	bunny centre-of-mass drift returns ($\times 3$); shape within a few per cent
MPM cell from the shape (following N instead)	mixed: gaps open earlier and thin parts improve on some meshes; bob's exterior silhouette $+34\,\%$; rest mixed

against 0.9860–0.9891 and 0.9866–0.9887 without gravity), end within 0.08 and 0.19 pitches of the standing height, stand within 0.41° of upright, and show no horizontal drift. The render's lead holds (particle IoU $+0.0010$ / $+0.0025$, exterior silhouette residual -49 / $-56\,\%$ at seed 97). The sag of the first 1.5 s (0.14 wu bunny, 0.27 wu dragon) is partly made rest shape at each commit, because a window is far shorter than the elastic period, so material keeps circulating (a mean particle move of 0.026–0.050 pitch per kept-frame pair at the end, against 0.0012–0.0029 without gravity).

B.7.5 Ablations

Table B.3 lists the ablations. At 40k the cloud read's particle IoU is ahead of the exterior read on 17 of 19 meshes, and the driven-only relaxation also improves the particle silhouette on 13 of 19, while the carried moments and the symmetric spacing are neutral there. The choices against late motion lengthen the dragon's runs.

B.7.6 Stress against Force Control

The plain force moves the centre of mass by 0.1–2.9 pitches and turns the body by up to 3.9°, 10^3–10^4 and 10^2–10^3 times the stress control, with 77–158 times its angular momentum. Both force arms end with 3.1–5.7 times (plain) and 4.5–5.1 times (projected; bob 207 times) the stress control's geometry energy and 17–97 % more exterior silhouette error, at the same render share, and the projected force morphs 1.6–2.4 times slower.

Table B.4: **Every method on the six 300k targets.** Per mesh the median over seeds, then the range over the meshes the method ran on; "ours ahead" counts the meshes on which our seed range lies above the method's.

method	meshes	silhouette IoU (indep.)	1 − IoU / floor	chamfer / floor	thin uncovered (%)	ours ahead (meshes)	end genus = target	volume over the frames	frames with a piece ≥ 5 pitches off	wall time
ours	6	0.9864–0.9899	1.1–1.3	0.97–0.98[e]	4.1–6.1	–	3 of 6[f]	0.96–1.09	0–8 %	8.8–33.7 min
F-control (Ch. 3)	6	0.9396–0.9754	2.2–7.4	1.03–2.20	30.2–48.4	6 of 6	(particles)	0.72–1.35	7–98 %	2.3–6.8 min
vol.-preserving neural morph	6	0.9163–0.9688	3.8–9.2	1.05–1.66	29.8–45.1	6 of 6	0 of 4[a]	0.73–1.47	0–71 %	45–47 min[b]
NISE	6	0.9660–0.9830	1.9–3.0	1.01–1.02	20.1–35.2	6 of 6	3 of 3[a]	0.67–1.01	0–24 %	13 min[b]
ISD	6	0.9822–0.9901	1.2–1.6	1.00–1.01	12.2–18.1	3 of 6	5 of 6	0.00–5.09	5–38 %	27–39 min
ISD, radial corr.	2	0.9816–0.9862	1.3–1.7	1.01	14.3–19.0	1 of 2	1 of 2	0.36–3.99	–	27–34 min
4Deform	6	0.4955–0.7628	26–57	2.8–10.3	67.5–81.5	6 of 6	3 of 6[c]	0.24–2.10	0–10 %	68–138 min
4Deform, radial corr.	2	0.9329–0.9784	2.0–6.0	1.04–3.94	22.7–32.2	2 of 2	0 of 2	0.86–1.48	–	69–75 min
ARAP[d]	3	0.9849–0.9884	1.2–1.3	1.01	9.7–16.8	1 of 3	3 of 3	0.86–1.02	0 %	0.2–12 min
OT displacement	2	0.9306–0.9440	5.0–6.5	1.04–1.11	24.0–24.7	2 of 2	(particles)	0.96–1.28	76–81 %	–
Large Steps (a)	6	0.9777–0.9881	1.2–2.0	1.01–1.46	9.7–17.7	3 of 6	3 of 6	0.85–1.74	0 %	18–33 s
Large Steps (b)	6	0.9807–0.9897	1.2–2.1	1.01–1.17	9.6–16.3	3 of 6	3 of 6	0.84–1.62	0 %	27–53 s
FlexiCubes (a)	6	0.8307–0.9315	7.5–20.7	1.21–6.64	53.5–70.8	6 of 6	0 of 6	0.76–1.10	67–86 %	10–21 s
FlexiCubes (b)	6	0.9055–0.9803	2.2–11.6	1.02–3.61	31.2–60.8	6 of 6	0 of 6	0.73–1.20	57–90 %	17–28 s

[a]Over the closed ends. [b]Plus signed-distance fitting per shape. [c]4Deform stays near the sphere. [d]Genus-zero targets only. [e]Below the floor because our particles are more even than a random sample. [f]Our display surface, meshed: one watertight component without self-intersection in every frame read.

B.7.7 Comparison with Other Methods

The F-control of Chapter 3 is reimplemented in our simulator and verified at the published scale ($N = 11{,}160$: a loss reduction of $-97.9\,\%$ against the published $-98.3\,\%$). The volume-preserving neural morph and NISE run from the former's code with the authors' defaults, and ISD and 4Deform from the authors' code. Both methods are driven by a correspondence term: ISD takes given sparse correspondences and 4Deform estimates its own with a matching module. A sphere and a target have no meaningful correspondences, so the main arm sets the matching weight to zero and does not use 4Deform's matching module, and a second arm supplies radial correspondences (each sphere sample paired with the ray hit from the pair's centre) on the bunny and the dragon. ARAP interpolation (Alexa et al., 2000) runs in gradient-domain form (Sumner and Popović, 2004; Xu et al., 2005) with the sphere correspondence from conformalized mean curvature flow (Kazhdan et al., 2012), on genus-zero targets. Large Steps and FlexiCubes run with our 18 views at 188 px (a) and with 25 views at 512 px (b). Mesh and SDF frames are sampled by our volume sampler at 300k; thin coverage is read on one shared set of thin points per mesh.

The end. Ours lies above every other method, with the seed ranges apart, on the dragon, the teapot and the armadillo, and is level with ISD, Large Steps and ARAP on the bunny, bob and homer. On the shared thin points, ours leaves 4.1–6.1 % uncovered, as many as a second sample of the mesh (2.9–8.1 %); every other method leaves at least 9.2 %. On the exterior silhouette residual ours is ahead of every external method on five meshes (bunny 7.0–7.7×10^{-4} against Large Steps (b) at 10.0–12.0 and ARAP at 10.7) and level on the sixth. The F-control of Chapter 3 ends with a kinetic measure two to seven orders of magnitude above ours.

Against the exact mesh. Read through our particle route, our held-out PSNR is the highest of every method on all six meshes, and our Boundary IoU error is level with or below that of ISD and of Large Steps (b) on five of six. A perfect particle sample read the same way has a lower Boundary IoU error than ours on four meshes and a higher F-

score at half a pitch on five, but a lower held-out PSNR on all six. Read on their own end meshes against the raw mesh, ISD and Large Steps (b) have a lower Boundary IoU error than our route on three and four meshes; a perfect 300k particle sample trails them by a similar amount (on the bunny, 0.051–0.054 against Large Steps (b)'s 0.006–0.007).

Topology. Our display surface, meshed by marching cubes (Lorensen and Cline, 1987) and read by the mesh methods' readers, is one closed watertight 2-manifold without self-intersection at every frame read (22 ends, 18 mid-progress frames and 83 frames of the first seconds). Its end genus matches the target on the bunny, bob and armadillo. The teapot's handle eye (about 5.5 pitches across, against an MPM cell of 6.4 pitches) is filled, 13 of the dragon's 15 loops close, and the twin loses the same openings. With an MPM cell that follows N (3.3 pitches), the teapot's handle opens in both arms at 6.5–7 s and the held-out Boundary IoU error falls by 17–36 % on the teapot, homer and the dragon, but the dragon opens only two or three of its loops, the early display is full of holes (genus up to 92 on homer and 102 on the dragon at 1–2 s), up to 155 particles leave the body beyond five pitches around 1.2–1.4 s before rejoining by 2.6 s, and the dragon's attempts take 2.3 times as long.

The way there. In every ISD run the volume collapses in mid-run to 0.1–65 % of the target's, and 6–21 of its 21 frames have more than one component. Large Steps never opens the dragon's handles, the teapot's handle or bob's ring and imitates them with self-intersecting surfaces (31 of 36 ends). FlexiCubes ends at a spurious genus of 4–58 on targets of genus 0–15. 4Deform with its matching weight at zero ends at a silhouette IoU of 0.50–0.76. Transport alone and the F-control of Chapter 3 have pieces farther than five pitches in 76–81 % and 7–98 % of their frames. Ours has such pieces in 0–8 % of its frames with at most 14 particles, and its common occupancy reads 0.96–1.09 over every frame.

B.7.8 Thrown and Dropped Bodies

In the delivered throw ($2 \times$ 150k particles, $E \times 7$), the bodies first touch at 3.8 s with the morph 43 % complete, part at 5.7 s with a restitution of 0.95, land at 8.95 and 10.95 s and tumble by 68° and 127°; each ends at a silhouette IoU of 0.985 / 0.984 in its own frame, momentum is kept within 2.7×10^{-4} in every phase, and the run takes 50.5 min. The dropped bunny (300k) lands at 2.5–2.6 s at a fifth of the wave speed; eight particles at an ear tip move two to six spacings off between 2.9 and 5.2 s and rejoin, and the run takes 9.7 min.

B.7.9 The Render with Its Weight Held

These runs back Section 4.6.4. In each, λ is set by Eq. (4.5) at the first window and then held, or only the colour term's weight is held; everything else, including the cleanup at $w_c = 0.2$, is the pipeline of this chapter. Each is compared with its $\lambda = 0$ twin and with the same scene under the rule.

Where the render acts (bunny, 300k, seed 97). Held, λ stays at 0.321; under the rule it falls to 0.059 by window 5, 0.0030 by window 20 and 0.0014 after window 40. The render's share of the outer layer's pull, aggregated over

groups of windows, is 0.20, 0.15, 0.14, 0.18 and 0.23 under the rule (windows 2–5, 6–10, 11–20, 21–40, after 40) and 0.59, 0.78, 0.88, 0.89 and 0.90 held; the render changes the outer layer's control relative to the twin by 11–20 % under the rule after arrival and by 80–97 % held. At the end, the kinetic energy is 1.1×10^{-3} held against 1.0×10^{-6} under the rule, the uncovered thin target points 5.7 % against 4.1 %, the silhouette IoU 0.9872 against 0.9884, and the stop rule ends the held run at 7.8 s (54 windows) against 13–17 s. One seed was run held.

Relief that exists only in the pictures. The plaque spans $217 \times 67 \times 35$ pitches at 300k; the letters are 1.5 pitches high with 4.5-pitch strokes, and each face is seen in 9 of the 18 views, 8 of them oblique. We read the relief the particles carry by its amplitude and its correlation with the lettering; the lettered mesh, sampled at 300k, gives the ceiling, 1.16–1.23 pitches at a correlation of 0.59–0.60. Held ($\lambda = 1.34$), the full method carves 0.69 pitch (58 % of the ceiling; 0.67 on SIGGRAPH, 0.74 on VANDERBILT) at a correlation of 0.50, and its shading error on the letter pixels is 40 % below the twin's; the twin carries -0.05. After the sixth window the render takes 94–99 % of the step and pushes 40–100 times harder than the physics gradient. The cost: the end mean speed is 479 times the twin's, the Chamfer distance to the smooth plaque rises by 0.09 pitch, the silhouette IoU falls from 0.9919 to 0.9851, the uncovered thin points rise from 3.2 to 11 %, and the run stops at 4.0 s after three rejected candidates (24 windows, 6 min, against the twin's 112 windows and 23 min); the held-out silhouette and the pieces are unchanged (no piece of 10 or more particles). At 40k, the held run carves 91 % of its ceiling, while the rule leaves the plaque at the twin's level (0.26 of the ceiling, the 40k reading's own floor being 0.21–0.27). The 300k run under the rule did not finish: another user's job took its card after about 160 windows, by which point λ had fallen to 1/4800 of its first value. On a coin and a bunny lettered "PM" at 40k, the held weight carves 47–53 % and 54–58 % of the ceiling, from about one second on, with end speeds 3–11 times the twin's.

Placement by appearance (40k, seeds 97 and 99). The paint is checked to be invisible to the physics: with every particle white, the run is bit for bit the run without colour, and the twin's trajectory is bit for bit the same under any paint. Five scenes move the red material over a ladder of median distances: half (a control, 2.6 pitches), band (3.7), ears (9.8), turn (12.4) and a dragon band (7.0). We report the recovered share r of the twin's layout error (the share of the exterior with the wrong colour). Under the rule $r = 0.01$–0.09; with the whole render weight held, $r = 0.19$–0.93; with only the colour term held and the shape render terms under the rule, $r = 0.67$–1.00, with the turn at 0.75 and 0.69, and the held-out colour PSNR rises by 2.9–6.0 dB. The cost of the colour-held runs: 12 of 80 shape cells fall short of the twin, mostly the F-score at half a pitch (by 0.001–0.002 on half and band, 0.01–0.07 on turn and dragon); 7 of 10 runs miss the rest criterion fixed beforehand, by a factor of 1.1–4.2, exactly those whose colour is still moving at the end; and on half, band and ears the surface sits 0.15–0.19 pitch rms from the twin's, since the colour moves mostly by a circulation below the surface, red material sinking and white rising.

Which path carries the lettering (40k, "PM" on a coin and a bunny, seeds 97 and 99, render weight held). With u switched off, the coin carves 0.18–0.19 of the ceiling against 0.47–0.53 with u, and the bunny 0.57–0.62 against

0.54–0.58; the twins stay at -0.06–0.04. Reading the physics placement terms only at the loss cell (the held render with placement at the cell) carves 0.55–0.59 on the coin and 0.66–0.83 on the bunny. In every held arm the body is still moving at the stop, at 3–49 times its twin's mean speed.

When the controls are released. Released at the stop and left to settle, the held runs come to rest (mean speed about 2×10^{-5}, below the twins' at their stop), the colour layout stays (band: r 0.93–0.94 at the stop and 0.92–0.94 settled), and the carved relief falls to 0.17–0.29 of the ceiling from 0.56–0.75 at the stop, a third to two fifths of it kept (0.29–0.43). Two mechanisms can remove it and have not yet been separated: the elastic part of the stress the controls built up, and the outer-layer relaxation toward the smooth 3D target, which keeps acting in the settling windows.

The press. The finished plaque, letters down, is pressed into a slab of a twentieth of the base stiffness and lifted, with the controls off; the plaque is made seven times stiffer than the base. At 300k the plaque keeps its end state, elastic and plastic, and its print reaches a contrast of 0.13 particle spacings and a correlation of 0.46 with the letters, against -0.05 and -0.28 for the twin with a decal. At 500k the end shape is taken as the rest shape; the plaque carries a contrast of 0.68 spacings against the twin's 0.03, and its print, with the press's tilt and bend removed, correlates with the letters at 0.65 against the twin's -0.13. At 40k the letters are shallower than the particle spacing, and the print keeps only a band. In a still picture the twin's decal reads more crisply than the carving.

B.7.10 Further Measured Costs

Transient floaters. The early arrival leaves a few particles apart from the body, mostly relief standing half a pitch outside the displayed surface. None persists; on the display surface a few form separate specks of fewer than ten particles, which the figures do not draw (Appendix B.2.2). Their main source is u; a link that keeps each layer step near its neighbour removed most of them but moved the centre of mass, and the next form makes that link symmetric, as the minimum spacing is.

Angular momentum and the drag. The ambient drag damps the particle momentum but not APIC's affine part. A drag that damps the whole APIC momentum cut the largest angular momentum several-fold on a test body and kept shape and rest.

A long, slow tail. Most of the delivered frames come after the arrival. A target that moves along the transport's displacement interpolation over the body's elastic period spreads the deformation in time, lowers the kinetic peak and the early loose particles, keeps the render's lead, and ends at rest, but its end trails the fixed target's slightly on a few meshes; the next measurement tells a shorter refit from under-supplied extremities.

Appendix C

IntSim Cutting Implementation Details

This appendix records numerical details of IntSim's cutting resistance, damage thresholds, and time-step stabilization. These quantities are used inside the simulator and are not supplied as inputs to the evaluated VLA policies.

C.1 Contact-Strength Normalization and Resistance Update

A normalized contact-strength indicator $\hat{c} \in [0,1]$ is constructed using the characteristic speed $c_{\text{norm}} = 0.35\,\Delta x/\Delta t$. It gates blade-localized damage and modulates material-dependent resistance. With normalized speed $u = s/s_0$, the update is

$$u \leftarrow \frac{u}{1 + k_2\,\hat{c}\,u\,\Delta t}, \tag{C.1}$$

where k_2 is scaled using properties such as Young's modulus E and yield stress σ_y.

C.2 Resolution-Dependent Damage Thresholds and Time-Step Constraint

Damage is gated by proximity to the blade SDF, contact strength, approach speed, and stroke direction as detailed in Chapter 5. To adjust the band $|\phi| < b$ and the speed threshold across discretizations, the implementation uses

$$b = b_0 \left(\frac{\Delta x_{\text{ref}}}{\Delta x} \right)^{\gamma}, \qquad v_{\text{th}} = \frac{\Delta x}{\Delta t}\,\hat{v}. \tag{C.2}$$

Here b_0 and Δx_{ref} specify a reference band and grid spacing, while $\hat{v}$ is a dimensionless threshold. The exponent γ is distinct from the morphing smoothing factor in Appendix A.

With Lamé parameters λ, μ and material density ρ, explicit time stepping is constrained by

$$\Delta t \leq 0.2\,\frac{\Delta x}{\sqrt{(\lambda + 2\mu)/\rho}}. \tag{C.3}$$

Additional numerical safeguards support robust rollouts but do not establish resolution-independent fracture accuracy.

References

Alexa, M., Cohen-Or, D., and Levin, D. (2000). As-rigid-as-possible shape interpolation. In *Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 157–164.

An, X., Kopka, J., Rode, M., and Zude-Sasse, M. (2025). Relationship of cell size distribution and biomechanics of strawberry fruit under varying Ca and N supply. *Food and Bioprocess Technology*, 18(2):1244–1258.

Bansal, S. and Tatu, A. (2018). Lie bodies based 3D shape morphing and interpolation. In *Proceedings of the ACM SIGGRAPH European Conference on Visual Media Production (CVMP)*, pages 5:1–5:10.

Bao, Y., Guo, X., and Qin, H. (2005). Physically based morphing of point-sampled surfaces. *Computer Animation and Virtual Worlds*, 16(3–4):509–518.

Baraff, D. and Witkin, A. (1998). Large steps in cloth simulation. In *Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 43–54.

Bardenhagen, S. G., Brackbill, J. U., and Sulsky, D. (2000). The material-point method for granular materials. *Computer Methods in Applied Mechanics and Engineering*, 187(3–4):529–541.

Bardenhagen, S. G. and Kober, E. M. (2004). The generalized interpolation material point method. *Computer Modeling in Engineering & Sciences*, 5(6):477–496.

Beltran-Hernandez, C. C., Erbetti, N., and Hamaya, M. (2024). SliceIt! – a dual simulator framework for learning robot food slicing. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 4296–4302.

Black, K., Brown, N., Driess, D., Esmail, A., Equi, M. R., Finn, C., Fusai, N., Groom, L., Hausman, K., Ichter, B., et al. (2025). π_0: A vision-language-action flow model for general robot control. In *Proceedings of Robotics: Science and Systems (RSS)*.

Bonneel, N., van de Panne, M., Paris, S., and Heidrich, W. (2011). Displacement interpolation using Lagrangian mass transport. *ACM Transactions on Graphics*, 30(6):158:1–158:12.

Breen, D. E. and Whitaker, R. T. (2001). A level-set approach for the metamorphosis of solid models. *IEEE Transactions on Visualization and Computer Graphics*, 7(2):173–192.

Bridson, R., Marino, S., and Fedkiw, R. (2003). Simulation of clothing with folds and wrinkles. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA)*, pages 28–36.

Brohan, A., Brown, N., Carbajal, J., Chebotar, Y., Chen, X., Choromanski, K., Ding, T., Driess, D., Dubey, A., Finn, C., et al. (2023). RT-2: Vision-language-action models transfer web knowledge to robotic control. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 229 of *Proceedings of Machine Learning Research*, pages 2165–2183.

Buonomo, C., Digne, J., and Chaine, R. (2025). Volume preserving neural shape morphing. *Computer Graphics Forum*, 44(5):e70196.

Chen, H., Zhao, S., and Zhao, J. (2025a). A sparse-memory-encoding GPU-MPM framework for large-scale simulations of granular flows. *Computers and Geotechnics*, 180:107113.

Chen, J., Kala, V., Marquez-Razon, A., Gueidon, E., Hyde, D. A. B., and Teran, J. (2021). A momentum-conserving implicit material point method for surface tension with contact angles and spatial gradients. *ACM Transactions on Graphics*, 40(4):111:1–111:16.

Chen, X., Liang, C., Huang, D., Real, E., Wang, K., Pham, H., Dong, X., Luong, T., Hsieh, C.-J., Lu, Y., and Le, Q. V. (2023). Symbolic discovery of optimization algorithms. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 49205–49233.

Chen, Y., Ge, Y., Tang, W., Li, Y., Ge, Y., Ding, M., Shan, Y., and Liu, X. (2025b). Moto: Latent motion token as the bridging language for learning robot manipulation from videos. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 19752–19763.

Chen, Y., Levin, D. I. W., and Langlois, T. R. (2024). Fluid control with Laplacian eigenfunctions. In *ACM SIGGRAPH Conference Papers*, pages 44:1–44:11.

Cheng, B., Girshick, R., Dollár, P., Berg, A. C., and Kirillov, A. (2021). Boundary IoU: Improving object-centric image segmentation evaluation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15329–15337.

Coros, S., Macklin, M., Thomaszewski, B., and Thürey, N. (2021). Differentiable simulation. In *SIGGRAPH Asia Courses*, pages 3:1–3:142.

Coros, S., Martin, S., Thomaszewski, B., Schumacher, C., Sumner, R., and Gross, M. (2012). Deformable objects alive! *ACM Transactions on Graphics*, 31(4):69:1–69:9.

Cundall, P. A. and Strack, O. D. L. (1979). A discrete numerical model for granular assemblies. *Géotechnique*, 29(1):47–65.

Cuturi, M. (2013). Sinkhorn distances: Lightspeed computation of optimal transport. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 26.

Dasari, S., Ebert, F., Tian, S., Nair, S., Bucher, B., Schmeckpeper, K., Singh, S., Levine, S., and Finn, C. (2020). RoboNet: Large-scale multi-robot learning. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 100 of *Proceedings of Machine Learning Research*, pages 885–897.

de Vaucorbeil, A., Nguyen, V. P., Sinaie, S., and Wu, J. Y. (2020). Material point method after 25 years: Theory, implementation, and applications. *Advances in Applied Mechanics*, 53:185–398.

Deitke, M., VanderBilt, E., Herrasti, A., Weihs, L., Ehsani, K., Salvador, J., Han, W., Kolve, E., Kembhavi, A., and Mottaghi, R. (2022). ProcTHOR: Large-scale embodied AI using procedural generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 35, pages 5982–5994.

Dikshit, A., Bartsch, A., George, A., and Farimani, A. B. (2023). RoboChop: Autonomous framework for fruit and vegetable chopping leveraging foundational models. arXiv preprint arXiv:2307.13159.

Du, H. and He, Q. (2026). JAX-MPM: A learning-augmented differentiable meshfree framework for GPU-accelerated Lagrangian simulation and geophysical inverse modeling. *Engineering with Computers*, 42(3):85.

Eisenberger, M. and Cremers, D. (2020). Hamiltonian dynamics for real-world shape interpolation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 179–196.

Fang, Y., Qu, Z., Li, M., Zhang, X., Zhu, Y., Aanjaneya, M., and Jiang, C. (2020). IQ-MPM: An interface quadrature material point method for non-sticky strongly two-way coupled nonlinear solids and fluids. *ACM Transactions on Graphics*, 39(4):51:1–51:16.

Fattal, R. and Lischinski, D. (2004). Target-driven smoke animation. *ACM Transactions on Graphics*, 23(3):441–448.

Feydy, J., Séjourné, T., Vialard, F.-X., Amari, S.-i., Trouvé, A., and Peyré, G. (2019). Interpolating between optimal transport and MMD using Sinkhorn divergences. In *Proceedings of the International Conference on Artificial Intelligence and Statistics (AISTATS)*, volume 89 of *Proceedings of Machine Learning Research*, pages 2681–2690.

Gao, L., Lai, Y.-K., Huang, Q.-X., and Hu, S.-M. (2013). A data-driven approach to realistic shape morphing. *Computer Graphics Forum*, 32(2):449–457.

Gast, T., Fu, C., Jiang, C., and Teran, J. (2016). Implicit-shifted symmetric QR singular value decomposition of 3×3 matrices. Technical report, University of California, Los Angeles.

Gibou, F., Hyde, D., and Fedkiw, R. (2019). Sharp interface approaches and deep learning techniques for multiphase flows. *Journal of Computational Physics*, 380:442–463.

Gong, R., Huang, J., Zhao, Y., Geng, H., Gao, X., Wu, Q., Ai, W., Zhou, Z., Terzopoulos, D., Zhu, S.-C., Jia, B., and Huang, S. (2023). ARNOLD: A benchmark for language-grounded task learning with continuous states in realistic 3D scenes. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 20426–20438.

Gu, J., Xiang, F., Li, X., Ling, Z., Liu, X., Mu, T., Tang, Y., Tao, S., Wei, X., Yao, Y., Yuan, X., Xie, P., Huang, Z., Chen, R., and Su, H. (2023). ManiSkill2: A unified benchmark for generalizable manipulation skills. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Guo, Q., Han, X., Fu, C., Gast, T., Tamstorf, R., and Teran, J. (2018). A material point method for thin shells with frictional contact. *ACM Transactions on Graphics*, 37(4):147:1–147:15.

Heiden, E., Macklin, M., Narang, Y., Fox, D., Garg, A., and Ramos, F. (2021). DiSECt: A differentiable simulation engine for autonomous robotic cutting. In *Proceedings of Robotics: Science and Systems (RSS)*.

Hochreiter, S. (1998). The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(2):107–116.

Hu, S.-M., Li, C.-F., and Zhang, H. (2004). Actual morphing: A physics-based approach to blending. In *Proceedings of the ACM Symposium on Solid Modeling and Applications (SM)*, pages 309–314.

Hu, Y., Anderson, L., Li, T.-M., Sun, Q., Carr, N., Ragan-Kelley, J., and Durand, F. (2020). DiffTaichi: Differentiable programming for physical simulation. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Hu, Y., Fang, Y., Ge, Z., Qu, Z., Zhu, Y., Pradhana, A., and Jiang, C. (2018). A moving least squares material point method with displacement discontinuity and two-way rigid body coupling. *ACM Transactions on Graphics*, 37(4):150:1–150:14.

Hu, Y., Li, T.-M., Anderson, L., Ragan-Kelley, J., and Durand, F. (2019a). Taichi: A language for high-performance computation on spatially sparse data structures. *ACM Transactions on Graphics*, 38(6):201:1–201:16.

Hu, Y., Liu, J., Spielberg, A., Tenenbaum, J. B., Freeman, W. T., Wu, J., Rus, D., and Matusik, W. (2019b). ChainQueen: A real-time differentiable physical simulator for soft robotics. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 6265–6271.

Hyde, D. A. B., Gagniere, S. W., Marquez-Razon, A., and Teran, J. (2020). An implicit updated Lagrangian formulation for liquids with large surface energy. *ACM Transactions on Graphics*, 39(6):183:1–183:13.

Jahanbakhshi, A., Yeganeh, R., and Shahgoli, G. (2020). Determination of mechanical properties of banana fruit under quasi-static loading in pressure, bending, and shearing tests. *International Journal of Fruit Science*, 20(3):314–322.

Jamdagni, P. and Jia, Y.-B. (2021). Robotic slicing of fruits and vegetables: Modeling the effects of fracture toughness and knife geometry. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 6607–6613.

Jiang, C., Schroeder, C., Selle, A., Teran, J., and Stomakhin, A. (2015). The affine particle-in-cell method. *ACM Transactions on Graphics*, 34(4):51:1–51:10.

Jiang, C., Schroeder, C., and Teran, J. (2017). An angular momentum conserving affine-particle-in-cell method. *Journal of Computational Physics*, 338:137–164.

Jiang, C., Schroeder, C., Teran, J., Stomakhin, A., and Selle, A. (2016). The material point method for simulating continuum materials. In *ACM SIGGRAPH Courses*, pages 24:1–24:52.

Johnson, D., Maxfield, T., Jin, Y., and Fedkiw, R. (2025). Software-based automatic differentiation is flawed. *Journal of Computational Physics*, 542:114319.

Kabas, O. and Vladut, V. (2015). Determination of drop-test behavior of a sample peach using finite element method. *International Journal of Food Properties*, 18(11):2584–2592.

Kakouris, E. G., Chatzis, M. N., and Triantafyllou, S. (2025). An extended B-spline-based material point method for contact problems. *International Journal for Numerical Methods in Engineering*, 126(3):e70003.

Karamcheti, S., Nair, S., Balakrishna, A., Liang, P., Kollar, T., and Sadigh, D. (2024). Prismatic VLMs: Investigating the design space of visually-conditioned language models. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 235 of *Proceedings of Machine Learning Research*, pages 23123–23144.

Kazhdan, M., Solomon, J., and Ben-Chen, M. (2012). Can mean-curvature flow be modified to be non-singular? *Computer Graphics Forum*, 31(5):1745–1754.

Keetha, N., Karhade, J., Jatavallabhula, K. M., Yang, G., Scherer, S., Ramanan, D., and Luiten, J. (2024). SplaTAM: Splat, track & map 3D Gaussians for dense RGB-D SLAM. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 21357–21366.

Kerbl, B., Kopanas, G., Leimkühler, T., and Drettakis, G. (2023). 3D Gaussian splatting for real-time radiance field rendering. *ACM Transactions on Graphics*, 42(4):139:1–139:14.

Khazatsky, A., Pertsch, K., Nair, S., Balakrishna, A., Dasari, S., Karamcheti, S., Nasiriany, S., Srirama, M. K., Chen, L. Y., Ellis, K., et al. (2024). DROID: A large-scale in-the-wild robot manipulation dataset. In *Proceedings of Robotics: Science and Systems (RSS)*.

Kheiralipour, K., Tabatabaeefar, A., Mobli, H., Rafiee, S., Sahraroo, A., Rajabipour, A., and Jafari, A. (2008). Some physical properties of apple. *Pakistan Journal of Nutrition*, 7(5):667–672.

Kilian, M., Mitra, N. J., and Pottmann, H. (2007). Geometric modeling in shape space. *ACM Transactions on Graphics*, 26(3):64:1–64:8.

Kim, M. J., Pertsch, K., Karamcheti, S., Xiao, T., Balakrishna, A., Nair, S., Rafailov, R., Foster, E. P., Sanketi, P. R., Vuong, Q., Kollar, T., Burchfiel, B., Tedrake, R., Sadigh, D., Levine, S., Liang, P., and Finn, C. (2025). OpenVLA: An open-source vision-language-action model. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 270 of *Proceedings of Machine Learning Research*, pages 2679–2713.

Kingma, D. P. and Ba, J. (2015). Adam: A method for stochastic optimization. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Knapitsch, A., Park, J., Zhou, Q.-Y., and Koltun, V. (2017). Tanks and Temples: Benchmarking large-scale scene reconstruction. *ACM Transactions on Graphics*, 36(4):78:1–78:13.

Koh, H., Song, C.-Y., Choi, Y., Viveiros, M., Hyde, D., and Kim, H. (2026). CulinaryCut: A physics-grounded cutting benchmark for vision-language-action models. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 397–415.

Kolve, E., Mottaghi, R., Han, W., VanderBilt, E., Weihs, L., Herrasti, A., Deitke, M., Ehsani, K., Gordon, D., Zhu, Y., Kembhavi, A., Gupta, A., and Farhadi, A. (2017). AI2-THOR: An interactive 3D environment for visual AI. arXiv preprint arXiv:1712.05474.

Kurniawati, E., Rakhmadevi, A. G., Fadhila, P. T., and Lailah, S. (2023). Determination of physical properties of Siamese oranges (*Citrus nobilis* var. *Microcarpa*): Sphericity, roundness, volume, and density. *Journal of Agritechnology and Food Processing*, 3(2):67–76.

Laine, S., Hellsten, J., Karras, T., Seol, Y., Lehtinen, J., and Aila, T. (2020). Modular primitives for high-performance differentiable rendering. *ACM Transactions on Graphics*, 39(6):194:1–194:14.

Li, Y., Du, T., Wu, K., Xu, J., and Matusik, W. (2023). DiffCloth: Differentiable cloth simulation with dry frictional contact. *ACM Transactions on Graphics*, 42(1):2:1–2:20.

Li, Z., He, J., Börcsök, B., Zhang, T., Chen, D., Du, T., Lin, M. C., Turk, G., and Zhu, B. (2025). An adjoint method for differentiable fluid simulation on flow maps. In *SIGGRAPH Asia Conference Papers*, pages 128:1–128:12.

Liang, J. and Lin, M. C. (2020). Differentiable physics simulation. In *ICLR Workshop on Integration of Deep Neural Models and Differential Equations*.

Liang, J., Lin, M. C., and Koltun, V. (2019). Differentiable cloth simulation for inverse problems. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32.

Lin, Y., Lin, C., Xu, J., and Mu, Y. (2025). OmniPhysGS: 3D constitutive Gaussians for general physics-based dynamics generation. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Liu, B., Zhu, Y., Gao, C., Feng, Y., Liu, Q., Zhu, Y., and Stone, P. (2023a). LIBERO: Benchmarking knowledge transfer for lifelong robot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 44776–44791.

Liu, H., Li, C., Wu, Q., and Lee, Y. J. (2023b). Visual instruction tuning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pages 34892–34916.

Liu, S., Wu, L., Li, B., Tan, H., Chen, H., Wang, Z., Xu, K., Su, H., and Zhu, J. (2025). RDT-1B: A diffusion foundation model for bimanual manipulation. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Liu, Y.-S., Yan, H.-B., and Martin, R. R. (2011). As-rigid-as-possible surface morphing. *Journal of Computer Science and Technology*, 26(3):548–557.

Lorensen, W. E. and Cline, H. E. (1987). Marching cubes: A high resolution 3D surface construction algorithm. In *Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 163–169.

Macklin, M. (2022). Warp: A high-performance Python framework for GPU simulation and graphics. NVIDIA GPU Technology Conference (GTC), https://github.com/NVIDIA/warp.

Mattos, F. R., Fasina, O. O., Reina, L. D., Fleming, H. P., Breidt, Jr., F., Damasceno, G. S., and Passos, F. V. (2005). Heat transfer and microbial kinetics modeling to determine the location of microorganisms within cucumber fruit. *Journal of Food Science*, 70(5):E324–E330.

McCann, R. J. (1997). A convexity principle for interacting gases. *Advances in Mathematics*, 128(1):153–179.

McNamara, A., Treuille, A., Popović, Z., and Stam, J. (2004). Fluid control using the adjoint method. *ACM Transactions on Graphics*, 23(3):449–456.

Mescheder, L., Oechsle, M., Niemeyer, M., Nowozin, S., and Geiger, A. (2019). Occupancy networks: Learning 3D reconstruction in function space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4455–4465.

Müller, M., Keiser, R., Nealen, A., Pauly, M., Gross, M., and Alexa, M. (2004). Point based animation of elastic, plastic and melting objects. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA)*, pages 141–151.

Museth, K. (2013). VDB: High-resolution sparse volumes with dynamic topology. *ACM Transactions on Graphics*, 32(3):27:1–27:22.

Nicolet, B., Jacobson, A., and Jakob, W. (2021). Large steps in inverse rendering of geometry. *ACM Transactions on Graphics*, 40(6):248:1–248:13.

Niu, D., Sharma, Y., Biamby, G., Quenum, J., Bai, Y., Shi, B., Darrell, T., and Herzig, R. (2025). LLARVA: Vision-action instruction tuning enhances robot learning. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 270 of *Proceedings of Machine Learning Research*, pages 3333–3355.

Nourain, J., Ying, Y.-b., Wang, J.-p., Rao, X.-q., and Yu, C.-g. (2005). Firmness evaluation of melon using its vibration characteristic and finite element analysis. *Journal of Zhejiang University-SCIENCE B*, 6(6):483–490.

Novello, T., da Silva, V., Schardong, G., Schirmer, L., Lopes, H., and Velho, L. (2023). Neural implicit surface evolution. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 14233–14243.

NVIDIA, Agarwal, N., Ali, A., Bala, M., Balaji, Y., Barker, E., Cai, T., Chattopadhyay, P., Chen, Y., Cui, Y., Ding, Y., et al. (2025). Cosmos world foundation model platform for physical AI. arXiv preprint arXiv:2501.03575.

O'Brien, J. F., Bargteil, A. W., and Hodgins, J. K. (2002). Graphical modeling and animation of ductile fracture. *ACM Transactions on Graphics*, 21(3):291–294.

Octo Model Team, Ghosh, D., Walke, H. R., Pertsch, K., Black, K., Mees, O., Dasari, S., Hejna, J., Kreiman, T., Xu, C., Luo, J., Tan, Y. L., Chen, L. Y., Sanketi, P. R., Vuong, Q., Xiao, T., Sadigh, D., Finn, C., and Levine, S. (2024). Octo: An open-source generalist robot policy. In *Proceedings of Robotics: Science and Systems (RSS)*.

Open X-Embodiment Collaboration, O'Neill, A., Rehman, A., Maddukuri, A., Gupta, A., Padalkar, A., Lee, A., Pooley, A., Gupta, A., Mandlekar, A., Jain, A., et al. (2024). Open X-Embodiment: Robotic learning datasets and RT-X models. In *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pages 6892–6903.

OpenAI, Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, F. L., Almeida, D., Altenschmidt, J., Altman, S., Anadkat, S., et al. (2023). GPT-4 technical report. arXiv preprint arXiv:2303.08774.

Perazzi, F., Pont-Tuset, J., McWilliams, B., Van Gool, L., Gross, M., and Sorkine-Hornung, A. (2016). A benchmark dataset and evaluation methodology for video object segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 724–732.

Qiao, Y.-L., Liang, J., Koltun, V., and Lin, M. C. (2020). Scalable differentiable physics for learning and control. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 119 of *Proceedings of Machine Learning Research*, pages 7847–7856.

Qiao, Y.-L., Liang, J., Koltun, V., and Lin, M. C. (2021). Differentiable simulation of soft multi-body systems. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 17123–17135.

Radford, A., Kim, J. W., Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., and Sutskever, I. (2021). Learning transferable visual models from natural language supervision. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 8748–8763.

Sang, L., Canfes, Z., Cao, D., Bernard, F., and Cremers, D. (2025a). Implicit neural surface deformation with explicit velocity fields. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Sang, L., Canfes, Z., Cao, D., Marin, R., Bernard, F., and Cremers, D. (2025b). 4Deform: Neural surface deformation for robust shape interpolation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 6542–6551.

Sharp, N. et al. (2019). Polyscope. https://polyscope.run.

Shen, T., Munkberg, J., Hasselgren, J., Yin, K., Wang, Z., Chen, W., Gojcic, Z., Fidler, S., Sharp, N., and Gao, J. (2023). Flexible isosurface extraction for gradient-based mesh optimization. *ACM Transactions on Graphics*, 42(4):37:1–37:16.

Shi, H., Xu, H., Clarke, S., Li, Y., and Wu, J. (2023). RoboCook: Long-horizon elasto-plastic object manipulation with diverse tools. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 229 of *Proceedings of Machine Learning Research*, pages 642–660.

Solenthaler, B., Schläfli, J., and Pajarola, R. (2007). A unified particle model for fluid–solid interactions. *Computer Animation and Virtual Worlds*, 18(1):69–82.

Sołowski, W. T., Berzins, M., Coombs, W. M., Guilkey, J. E., Möller, M., Tran, Q. A., Adibaskoro, T., Seyedan, S., Tielen, R., and Soga, K. (2021). Material point method: Overview and challenges ahead. *Advances in Applied Mechanics*, 54:113–204.

Song, C.-Y. and Hyde, D. (2025). PhysMorph-GS: Render-guided volumetric morphing with differentiable physics. arXiv preprint arXiv:2511.16988.

Stomakhin, A., Howes, R., Schroeder, C., and Teran, J. M. (2012). Energetically consistent invertible elasticity. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA)*, pages 25–32.

Stomakhin, A., Schroeder, C., Chai, L., Teran, J., and Selle, A. (2013). A material point method for snow simulation. *ACM Transactions on Graphics*, 32(4):102:1–102:10.

Sulsky, D., Chen, Z., and Schreyer, H. L. (1994). A particle method for history-dependent materials. *Computer Methods in Applied Mechanics and Engineering*, 118(1–2):179–196.

Sumner, R. W. and Popović, J. (2004). Deformation transfer for triangle meshes. *ACM Transactions on Graphics*, 23(3):399–405.

Szot, A., Clegg, A., Undersander, E., Wijmans, E., Zhao, Y., Turner, J., Maestre, N., Mukadam, M., Chaplot, D. S., Maksymets, O., et al. (2021). Habitat 2.0: Training home assistants to rearrange their habitat. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 251–266.

Tatarchenko, M., Richter, S. R., Ranftl, R., Li, Z., Koltun, V., and Brox, T. (2019). What do single-view 3D reconstruction networks learn? In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 3400–3409.

Terzopoulos, D., Platt, J., Barr, A., and Fleischer, K. (1987). Elastically deformable models. In *Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 205–214.

Tobin, J., Fong, R., Ray, A., Schneider, J., Zaremba, W., and Abbeel, P. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. In *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 23–30.

Treuille, A., McNamara, A., Popović, Z., and Stam, J. (2003). Keyframe control of smoke simulations. *ACM Transactions on Graphics*, 22(3):716–723.

Turk, G. and O'Brien, J. F. (1999). Shape transformation using variational implicit functions. In *Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 335–342.

van Kaick, O., Zhang, H., Hamarneh, G., and Cohen-Or, D. (2011). A survey on shape correspondence. *Computer Graphics Forum*, 30(6):1681–1707.

Walke, H., Black, K., Lee, A., Kim, M. J., Du, M., Zheng, C., Zhao, T., Hansen-Estruch, P., Vuong, Q., He, A., Myers, V., Fang, K., Finn, C., and Levine, S. (2023). BridgeData V2: A dataset for robot learning at scale. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 229 of *Proceedings of Machine Learning Research*, pages 1723–1736.

Wang, L., Bian, J., Heiden, E., and Garg, A. (2025). TopoCut: Learning multi-step cutting with spectral rewards and discrete diffusion policies. In *Proceedings of the Conference on Robot Learning (CoRL)*, volume 305 of *Proceedings of Machine Learning Research*, pages 1282–1310.

Wang, S., Ding, M., Gast, T. F., Zhu, L., Gagniere, S., Jiang, C., and Teran, J. M. (2019). Simulation and visualization of ductile fracture with the material point method. *Proceedings of the ACM on Computer Graphics and Interactive Techniques*, 2(2):18:1–18:20.

Wang, X., Li, M., Fang, Y., Zhang, X., Gao, M., Tang, M., Kaufman, D. M., and Jiang, C. (2020a). Hierarchical optimization time integration for CFL-rate MPM stepping. *ACM Transactions on Graphics*, 39(3):21:1–21:16.

Wang, X., Qiu, Y., Slattery, S. R., Fang, Y., Li, M., Zhu, S.-C., Zhu, Y., Tang, M., Manocha, D., and Jiang, C. (2020b). A massively parallel and scalable multi-GPU material point method. *ACM Transactions on Graphics*, 39(4):30:1–30:15.

Wang, Z., Bovik, A. C., Sheikh, H. R., and Simoncelli, E. P. (2004). Image quality assessment: From error visibility to structural similarity. *IEEE Transactions on Image Processing*, 13(4):600–612.

Witkin, A. and Kass, M. (1988). Spacetime constraints. In *Proceedings of the Annual Conference on Computer Graphics and Interactive Techniques (SIGGRAPH)*, pages 159–168.

Wolper, J., Chen, Y., Li, M., Fang, Y., Qu, Z., Lu, J., Cheng, M., and Jiang, C. (2020). AnisoMPM: Animating anisotropic damage mechanics. *ACM Transactions on Graphics*, 39(4):37:1–37:16.

Wolper, J., Fang, Y., Li, M., Lu, J., Gao, M., and Jiang, C. (2019). CD-MPM: Continuum damage material point methods for dynamic fracture animation. *ACM Transactions on Graphics*, 38(4):119:1–119:15.

Wu, G., Yi, T., Fang, J., Xie, L., Zhang, X., Wei, W., Liu, W., Tian, Q., and Wang, X. (2024). 4D Gaussian splatting for real-time dynamic scene rendering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20310–20320.

Wu, S., Basu, S., Brödermann, T., Van Gool, L., and Sakaridis, C. (2025). PBR-NeRF: Inverse rendering with physics-based neural fields. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 10974–10984.

Xie, T., Zong, Z., Qiu, Y., Li, X., Feng, Y., Yang, Y., and Jiang, C. (2024). PhysGaussian: Physics-integrated 3D Gaussians for generative dynamics. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 4389–4398.

Xu, D., Zhang, H., Wang, Q., and Bao, H. (2005). Poisson shape interpolation. In *Proceedings of the ACM Symposium on Solid and Physical Modeling (SPM)*, pages 267–274.

Xu, J., Makoviychuk, V., Narang, Y., Ramos, F., Matusik, W., Garg, A., and Macklin, M. (2022). Accelerated policy learning with parallel differentiable simulation. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Xu, M. and Levin, D. I. W. (2023). Deformation gradient control of physically simulated amorphous solids. In *Proceedings of the ACM SIGGRAPH/Eurographics Symposium on Computer Animation (SCA)*, pages 2:1–2:2.

Xu, M., Song, C.-Y., Levin, D., and Hyde, D. (2025). A differentiable material point method framework for shape morphing. *IEEE Transactions on Visualization and Computer Graphics*, 31(10):9140–9153.

Yang, G., Belongie, S., Hariharan, B., and Koltun, V. (2021). Geometry processing with neural fields. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34, pages 22483–22497.

Yang, Z., Gao, X., Zhou, W., Jiao, S., Zhang, Y., and Jin, X. (2024). Deformable 3D Gaussians for high-fidelity monocular dynamic scene reconstruction. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 20331–20341.

Ye, S., Jang, J., Jeon, B., Joo, S. J., Yang, J., Peng, B., Mandlekar, A., Tan, R., Chao, Y.-W., Lin, B. Y., Liden, L., Lee, K., Gao, J., Zettlemoyer, L., Fox, D., and Seo, M. (2025). Latent action pretraining from videos. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Yifan, W., Serena, F., Wu, S., Öztireli, C., and Sorkine-Hornung, O. (2019). Differentiable surface splatting for point-based geometry processing. *ACM Transactions on Graphics*, 38(6):230:1–230:14.

Yu, J., Liu, H., Yu, Q., Ren, J., Hao, C., Ding, H., Huang, G., Huang, G., Song, Y., Cai, P., Zhang, W., and Lu, C. (2025). ForceVLA: Enhancing VLA models with a force-aware MoE for contact-rich manipulation. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 38, pages 93409–93439.

Yu, T., Kumar, S., Gupta, A., Levine, S., Hausman, K., and Finn, C. (2020). Gradient surgery for multi-task learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 5824–5836.

Zamora Mora, M. A., Peychev, M., Ha, S., Vechev, M., and Coros, S. (2021). PODS: Policy optimization via differentiable simulation. In *Proceedings of the International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 7805–7817.

Zhang, R., Isola, P., Efros, A. A., Shechtman, E., and Wang, O. (2018). The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 586–595.

Zhang, T., Yu, H.-X., Wu, R., Feng, B. Y., Zheng, C., Snavely, N., Wu, J., and Freeman, W. T. (2024). PhysDreamer: Physics-based interaction with 3D objects via video generation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pages 388–406.

Zhang, Y., Yu, W., Liu, C. K., Kemp, C. C., and Turk, G. (2020). Learning to manipulate amorphous materials. *ACM Transactions on Graphics*, 39(6):189:1–189:11.

Zheng, R., Liang, Y., Huang, S., Gao, J., Daumé III, H., Kolobov, A., Huang, F., and Yang, J. (2025). TraceVLA: Visual trace prompting enhances spatial-temporal awareness for generalist robotic policies. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Zhu, D., Chen, J., Shen, X., Li, X., and Elhoseiny, M. (2024). MiniGPT-4: Enhancing vision-language understanding with advanced large language models. In *Proceedings of the International Conference on Learning Representations (ICLR)*.

Zhu, Y. and Bridson, R. (2005). Animating sand as a fluid. *ACM Transactions on Graphics*, 24(3):965–972.

Zhu, Y., Popović, J., Bridson, R., and Kaufman, D. M. (2017). Planar interpolation with extreme deformation, topology change and dynamics. *ACM Transactions on Graphics*, 36(6):213:1–213:15.